Stanford | ONLINE

AI Agent Test-Time Scaling and Self-Improvement Techniques

Aakanksha Chowdhery
Adjunct Professor of Computer Science

Azalia Mirhoseini
Assistant Professor of Computer Science

This document is for your personal, educational, non-commercial use only.

Copyright © Stanford University. All Rights Reserved.

Stanford | ONLINE

Aakanksha Chowdhery

Adjunct Professor of Computer Science

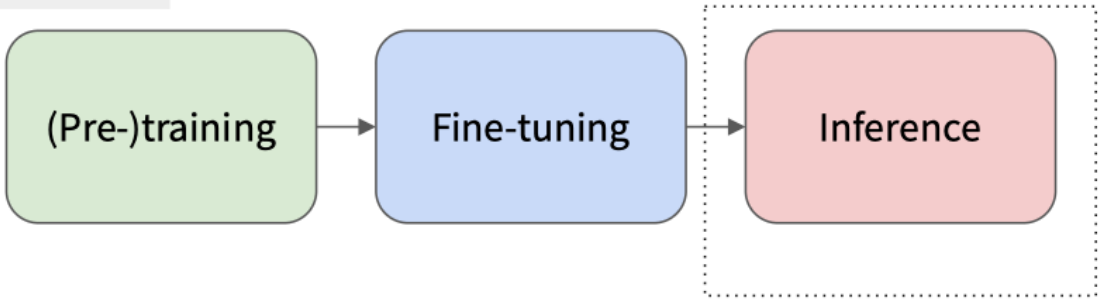
Azalia Mirhoseini

Assistant Professor of Computer Science

Test-Time Scaling

Stanford | ONLINE

What's New?
A New Frontier for Scaling is **Inference!**



What We Will Cover

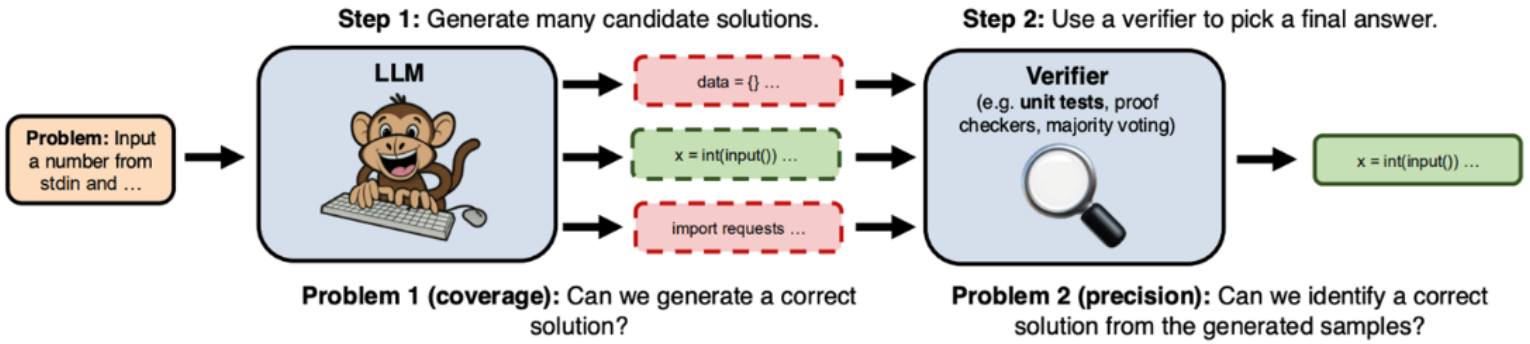
- Large Language Monkeys: Scaling Inference Compute with Repeated Sampling
- Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
- Archon: An Architecture Search Framework for Inference-Time Techniques

The Infinite Monkey Theorem

The ***infinite monkey theorem*** says a monkey hitting a typewriter keyboard randomly for an infinite number of times will almost surely type any text, including the works of William Shakespeare. (source: wiki/Infinite_monkey_theorem)

Large Language Monkeys

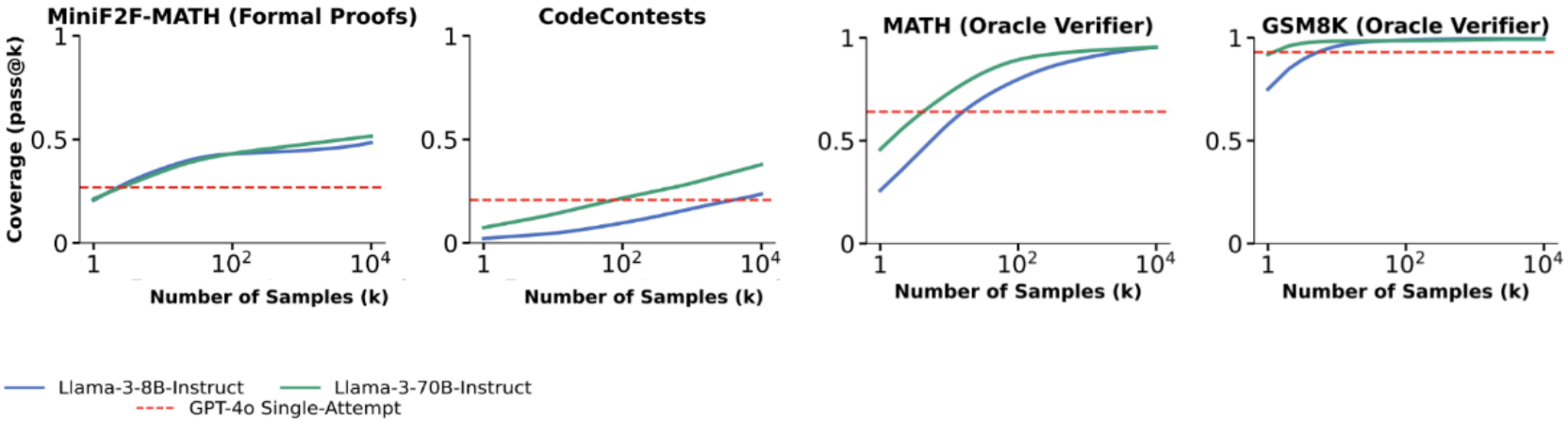
The ***infinite monkey theorem*** says a monkey hitting a typewriter keyboard randomly for an infinite number of times will almost surely type any text, including the works of William Shakespeare. (source: wiki/Infinite_monkey_theorem)



Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

Models Improve Drastically with Just Repeated Sampling!

Llama-3-8B outperforms GPT-4o on various reasoning benchmarks if sampled more!

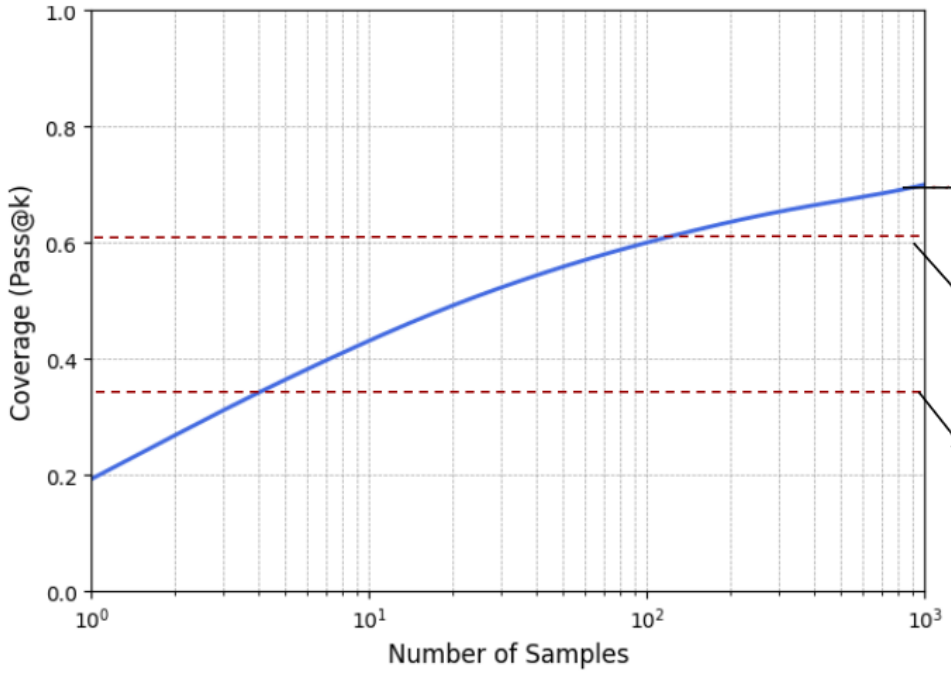


Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

*Due to the large compute cost, we generated results on random subsets of datasets.

Inference Scaling also Works for Agentic Benchmarks Like SWE-Bench Verified!

In domains with automated verification such as formal proofs for math and unit tests for programming, scaling sampling readily leads to more capability



70+% coverage on SWE-bench Verified with open-source DeepSeek + repeated sampling

62.2% SOTA as of 12/13/2024, used Claude-3-5-sonnet-new as backend (by Aide)

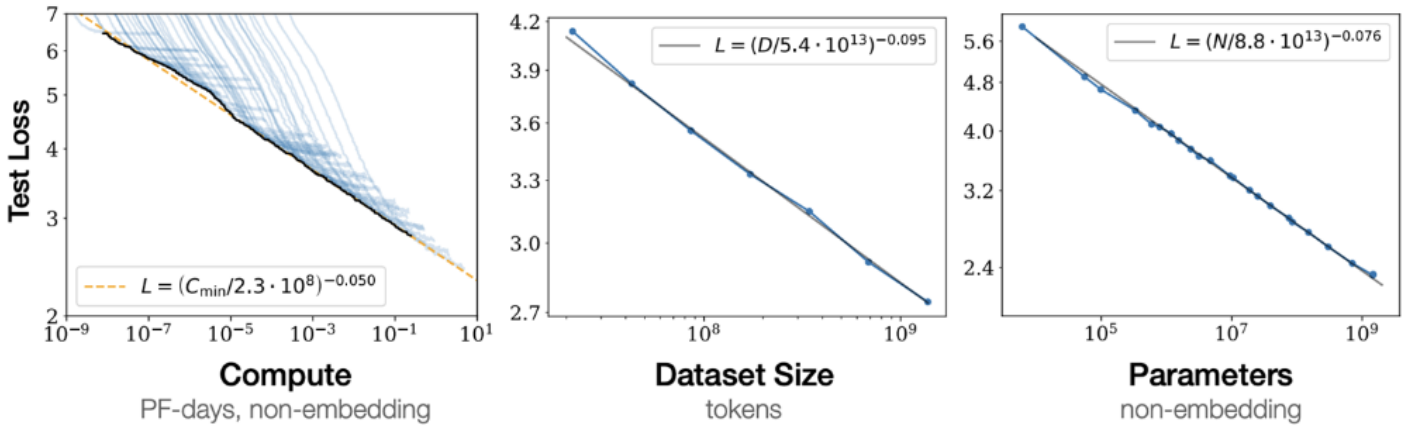
38.4% - O1 Preview

Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

(Pre-Training) Scaling Laws for Large Language Models (LLMs)

There is a power law relationship between LLMs test loss and:

- Amount of compute used
- Dataset size
- Number of parameters in the model



Kaplan et al., Scaling Laws for Neural Language Models, 2020

Scaling Laws for Inference

- The relationship between coverage (accuracy) and the number of samples can be modelled using an *exponentiated power law*

- Similar to pretraining, this allows us to predict the performance as a function of scaling compute, but this time scaling happens only at inference time

$$c = \exp(ak^b)$$

k: number of samples
a & b: curve fitting parameters
c: coverage or fraction of problems solved by at least one sample

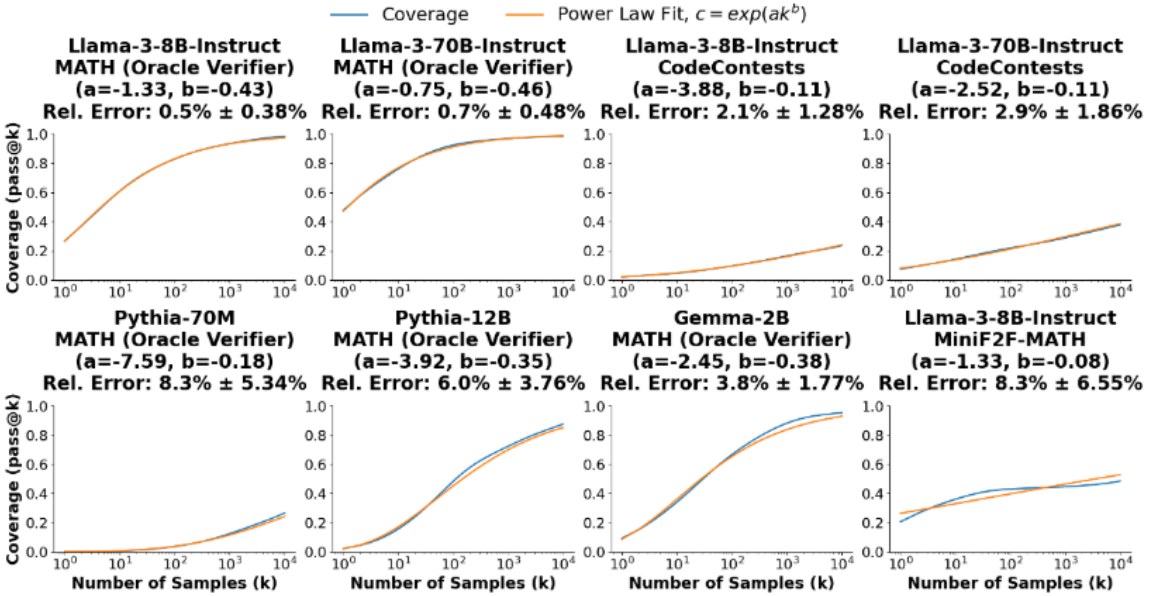
Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

Scaling Laws for Inference

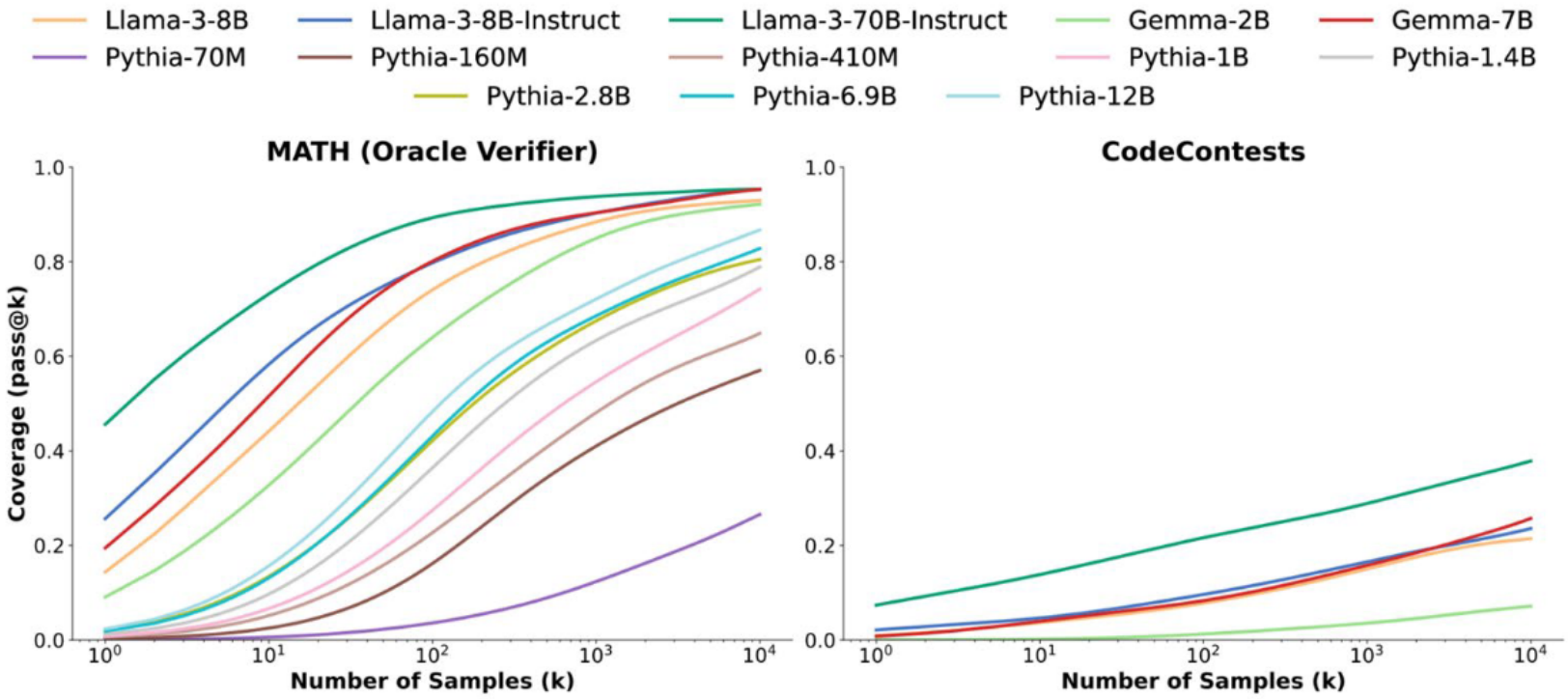
- The relationship between coverage (accuracy) and the number of samples can be modelled using an *exponentiated power law*

$$c = \exp(ak^b)$$

k: number of samples
a & b: curve fitting parameters
c: fraction of problems solved by at least one sample



Scaling Inference Is Generally Applicable: From 70M to 70B Parameter Size Models



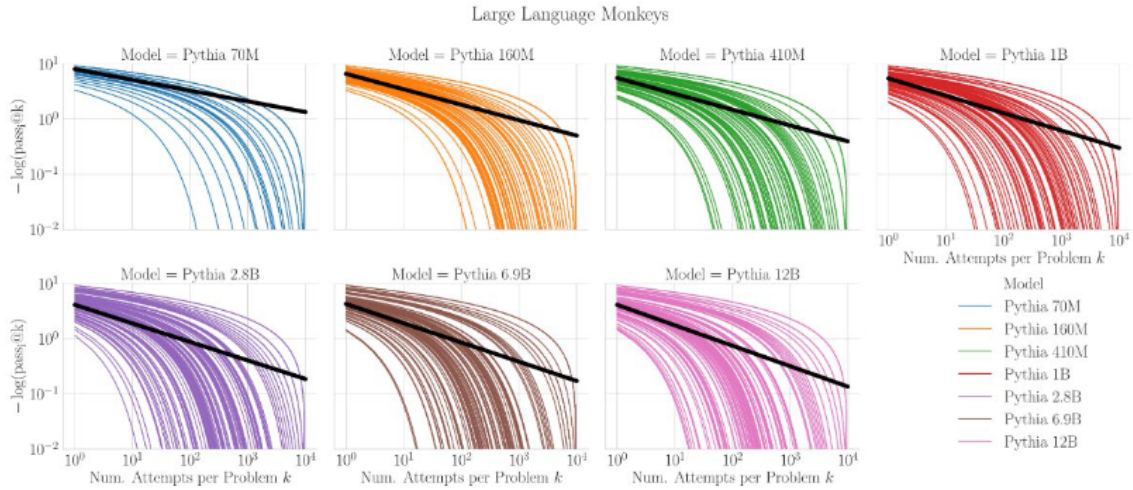
How Do Large Language Monkeys Get Their Power (Laws)?

For a SINGLE problem, we should expect -log(pass@k) to scale exponentially with k:

$$\begin{aligned}\text{pass}_\text{i}@\text{k} &= \mathop{\mathbb{E}}_{k \text{ Attempts}}\Big[1 - \mathbb{I}[\text{All } k \text{ Attempts Fail}]\Big] \\ &= 1 - \prod_{j=1}^{k} \mathop{\mathbb{E}}_{1 \text{ Attempt}}\Big[\mathbb{I}[j\text{-th Attempt Fails}]\Big].\end{aligned}$$

$$\text{pass}_\text{i}@\text{k} = 1 - (1 - \text{pass}_\text{i}@1)^k.$$

$$-\log(\text{pass}_\text{i}@\text{k}) \approx (1 - \text{pass}_\text{i}@1)^k.$$



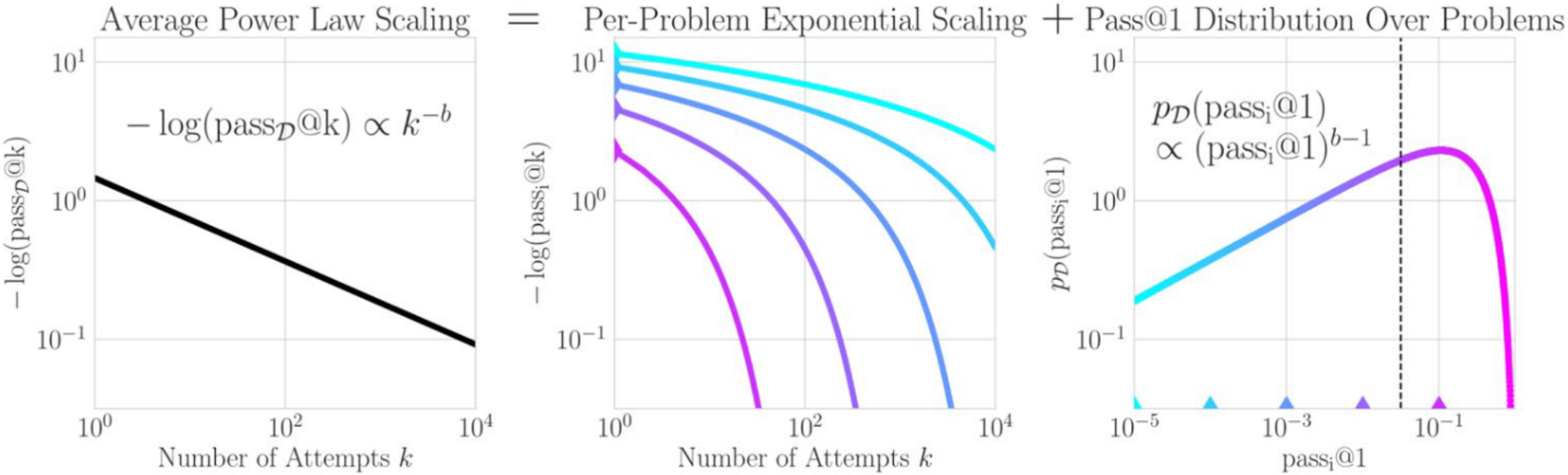
-log(avg pass@k): Power Law (Polynomial) Scaling with k

-log(pass_i@k): Exponential Scaling with k

How can we reconcile these?

How Do Large Language Monkeys Get Their Power (Laws)?
Rylan Schaeffer, Joshua Kazdan, John Hughes, Jordan Juravsky, Sara Price, Aengus Lynch, Erik Jones, Robert Kirk, Azalia Mirhoseini, Sanmi Koyejo, ICML 2025

Theorems: Long Tail of Hard Problems Is Sufficient and Necessary to Create Power Law Scaling in Avg Pass@k

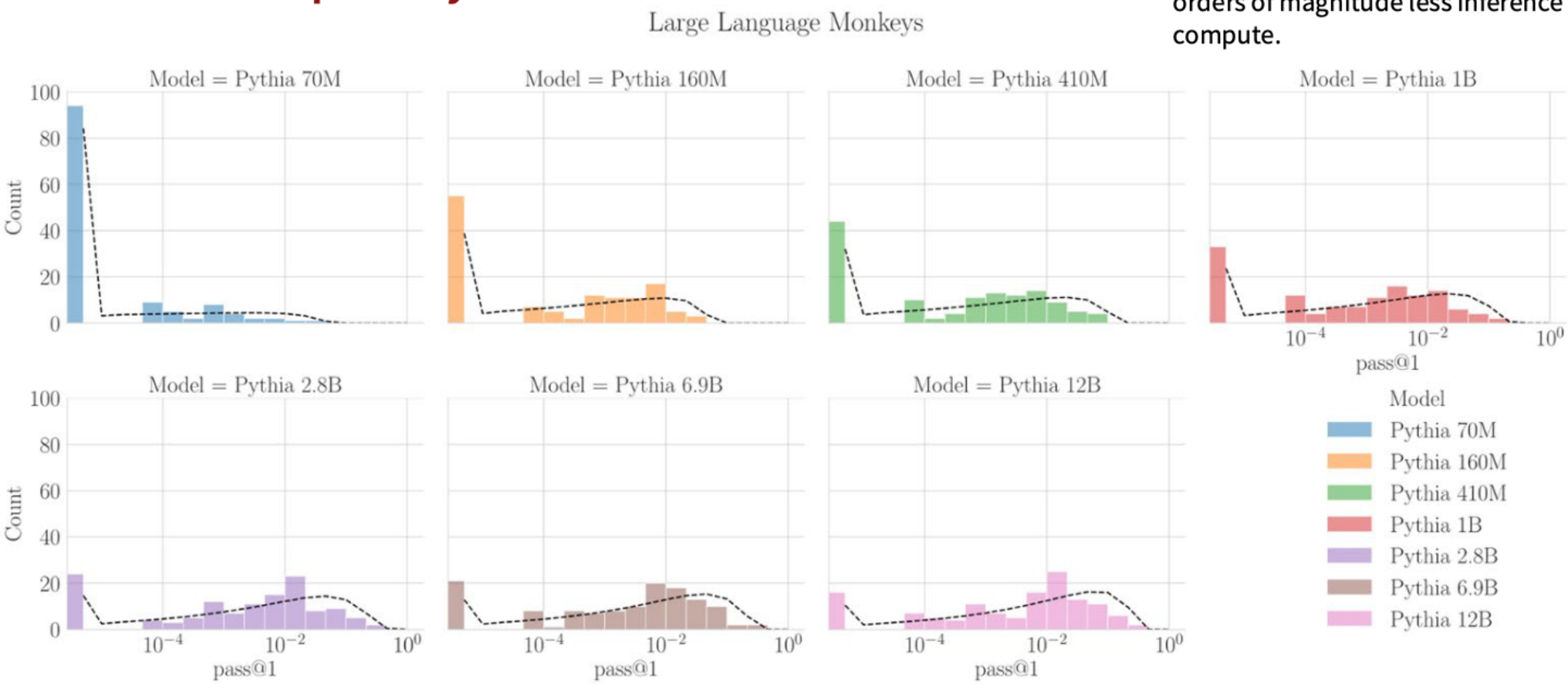


Intuition: For -log(avg pass@k) to fall polynomially with the number of attempts k, if each problem's -log(pass_i@k) falls exponentially with k, then a long tail of extremely hard problems is necessary to "drag out" the exponential curves

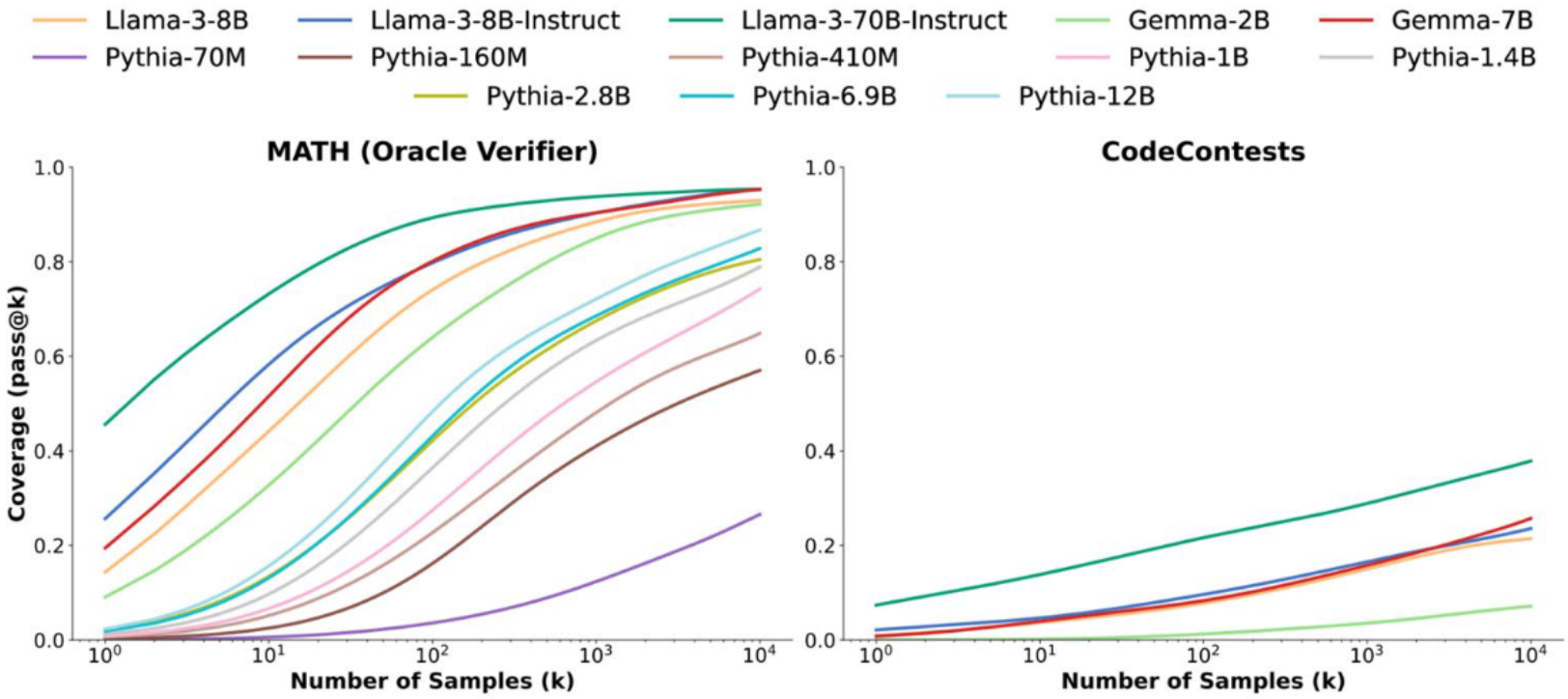
How Do Large Language Monkeys Get Their Power (Laws)?
Rylan Schaeffer, Joshua Kazdan, John Hughes, Jordan Juravsky, Sara Price, Aengus Lynch, Erik Jones, Robert Kirk, Azalia Mirhoseini, Sanmi Koyejo, ICML 2025

Does Data Empirically Have This Power Law Left Tail?

Provides a method for predicting the power law exponent with ~2-4 orders of magnitude less inference compute.

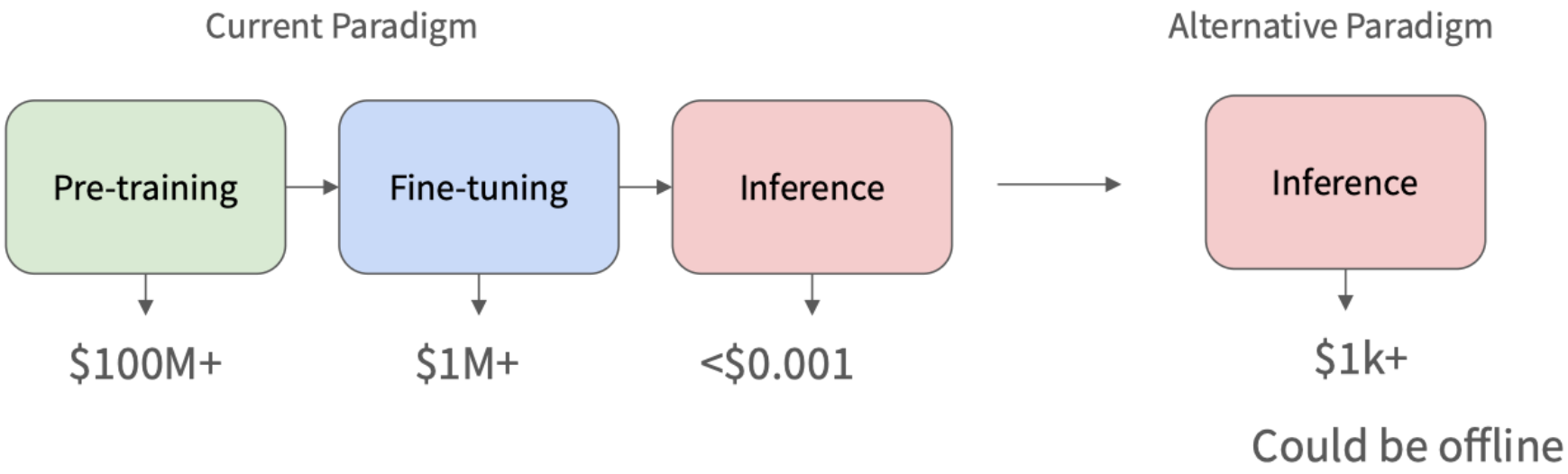


Scaling Inference Is Generally Applicable: From 70M to 70B Parameter Size Models



Inference Time Compute Predictably Translates to Intelligence

- This motivates a shift in compute allocations, especially for domains with automated verifiers



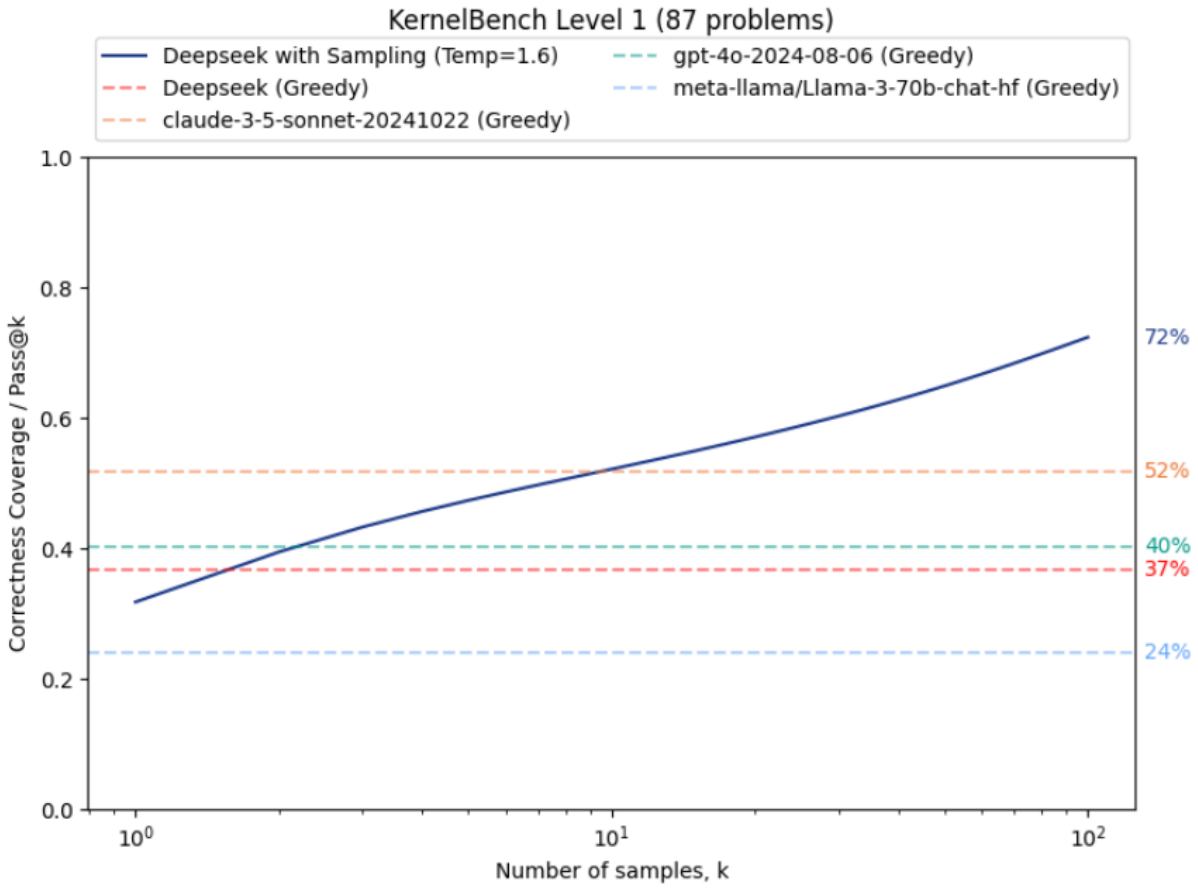
Need for Automated Verification!

Inference-Time Scaling Requires Robust Verification

- In domains with automated verification, repeated sampling directly leads to more capability
  - Formal proofs for math
  - Unit tests for coding
  - AI as a compiler: e.g. generate CUDA and use source Pytorch as verifier

Repeated Sampling Works for CUDA Generation Too

- Using the same ideas in Large Language Monkeys
- Scaling law when applying sampling
  - Coverage (correctness of generated code) scales log-linearly with number of samples
- The advantage?
  - We have automatic unit tests: matching source torch code for any input



Inference-Time Scaling Requires Robust Verification

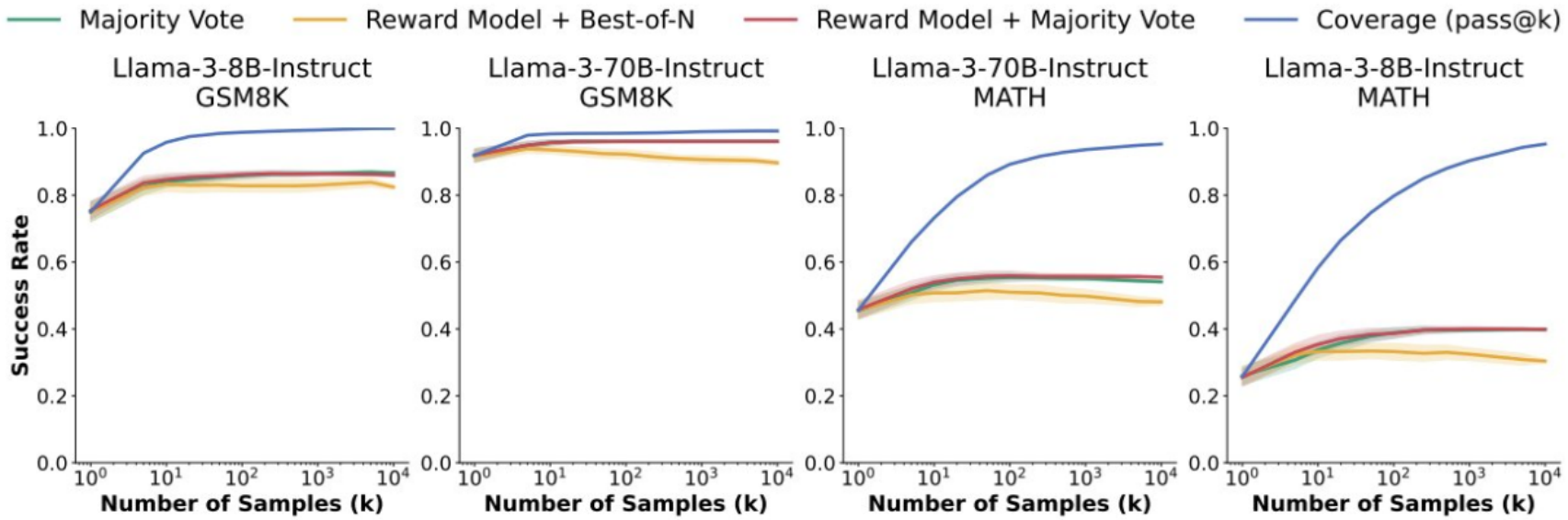
- In domains with automated verification, repeated sampling directly leads to more capability
  - Formal proofs for math
  - Unit tests for coding
  - AI as a compiler: e.g. generate CUDA and use source Pytorch as verifier

Inference-Time Scaling Requires Robust Verification

- In domains with automated verification, repeated sampling directly leads to more capability
  - Formal proofs for math
  - Unit tests for coding
  - AI as a compiler: e.g. generate CUDA and use source Pytorch as verifier

- For domains without verification, there is a large gap between “best-of-N” and methods such as majority voting and model-based rankers (i.e. reward models)

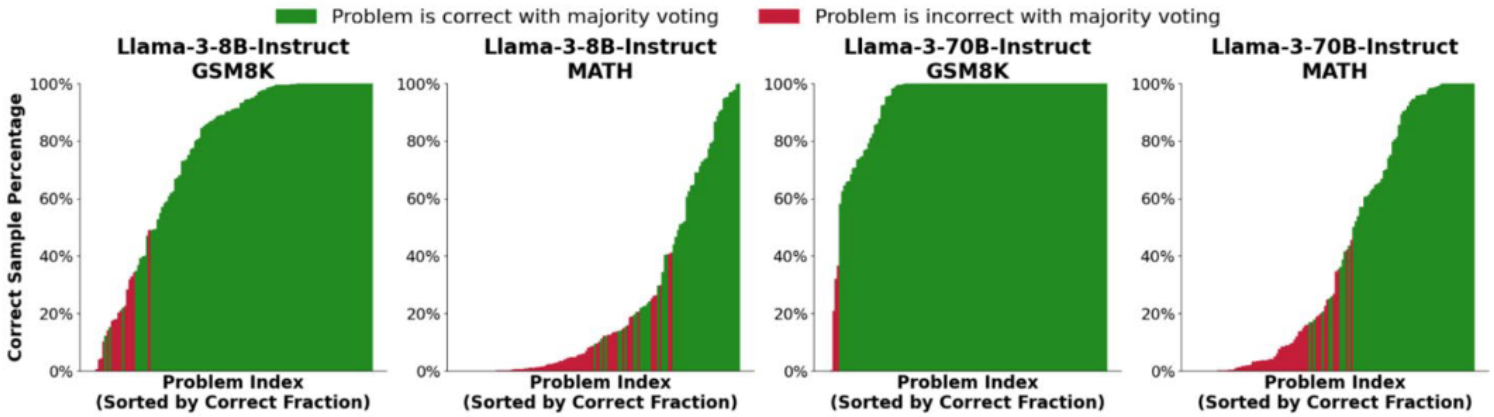
Existing Verification Methods Need Drastic Improvements



Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

The Correct Generations Could Be Very Rare

So, it makes sense that majority voting doesn't work well across the board!



Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Bradley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

Monkeys showed repeated sampling is very effective!

What are other ways to scale compute at inference?

What We Will Cover

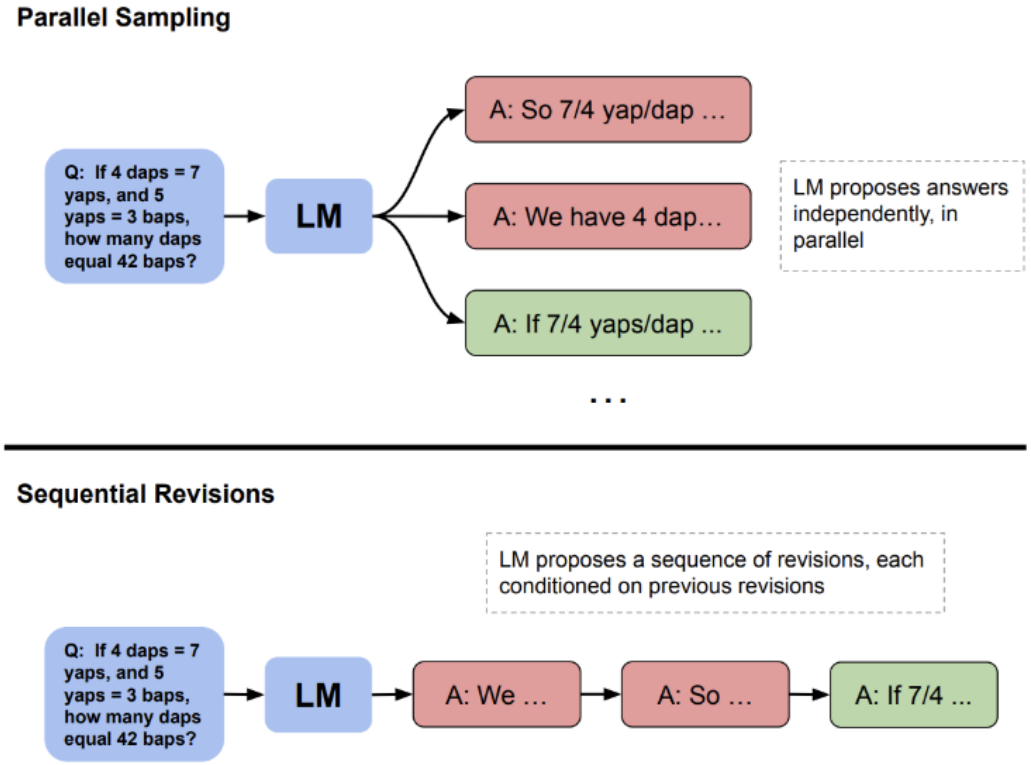
- Large Language Monkeys: Scaling Inference Compute with Repeated Sampling
- Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
- Archon: An Architecture Search Framework for Inference-Time Techniques

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
Charlie Snell, Jaehoon Lee, Kelvin Xu, Aviral Kumar, 2024

Test Time Scaling Techniques

- Sequential revisions: by asking the base model to revise its original responses “sequentially”
- Parallel sampling: by sampling multiple responses in parallel
- We can also alter how a verifier is used to select the correct response

Parallel Sampling vs Sequential Revisions

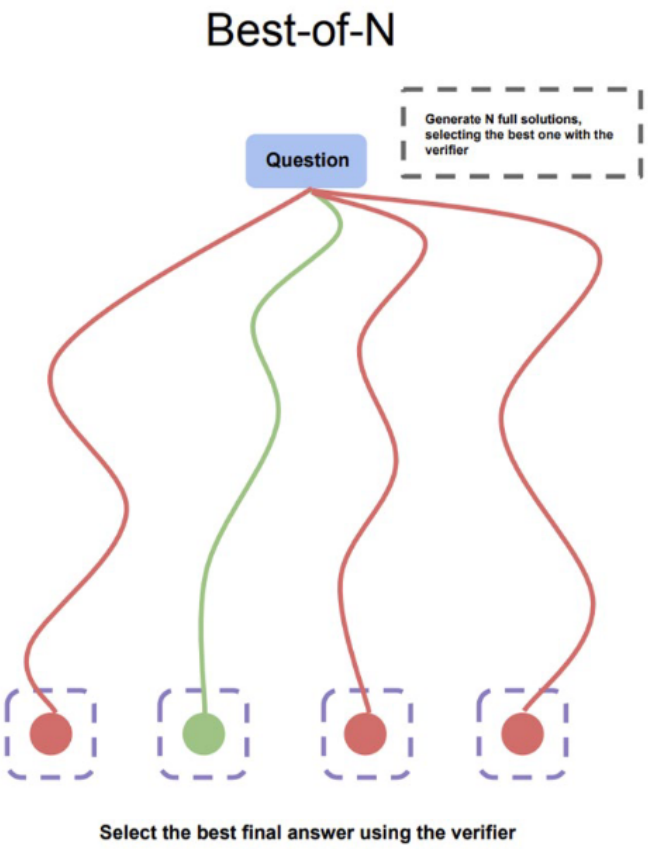


Type of Reward Models

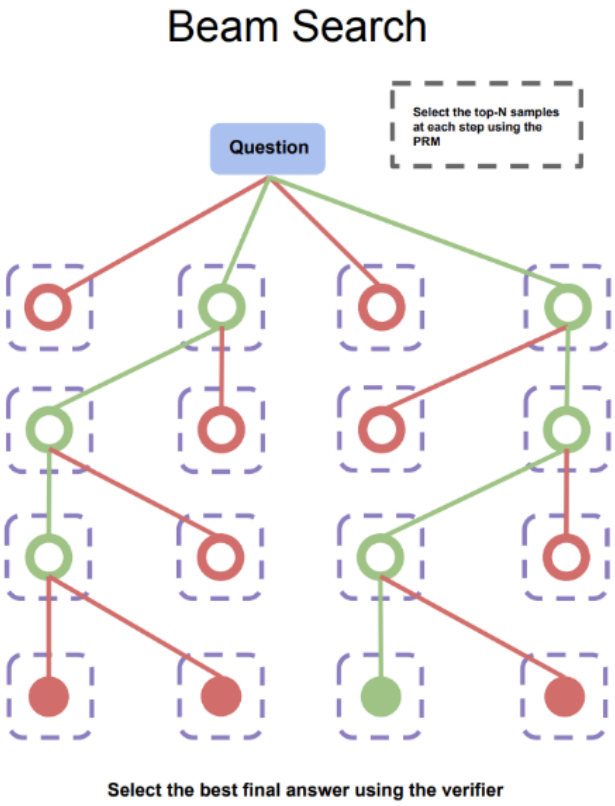
When we evaluate the model's responses during test time:

- Outcome Reward Model assigns a score to the final answer when it is correct.
- Process Reward Model assigns a score for each step in the model's response and not just the final answer.

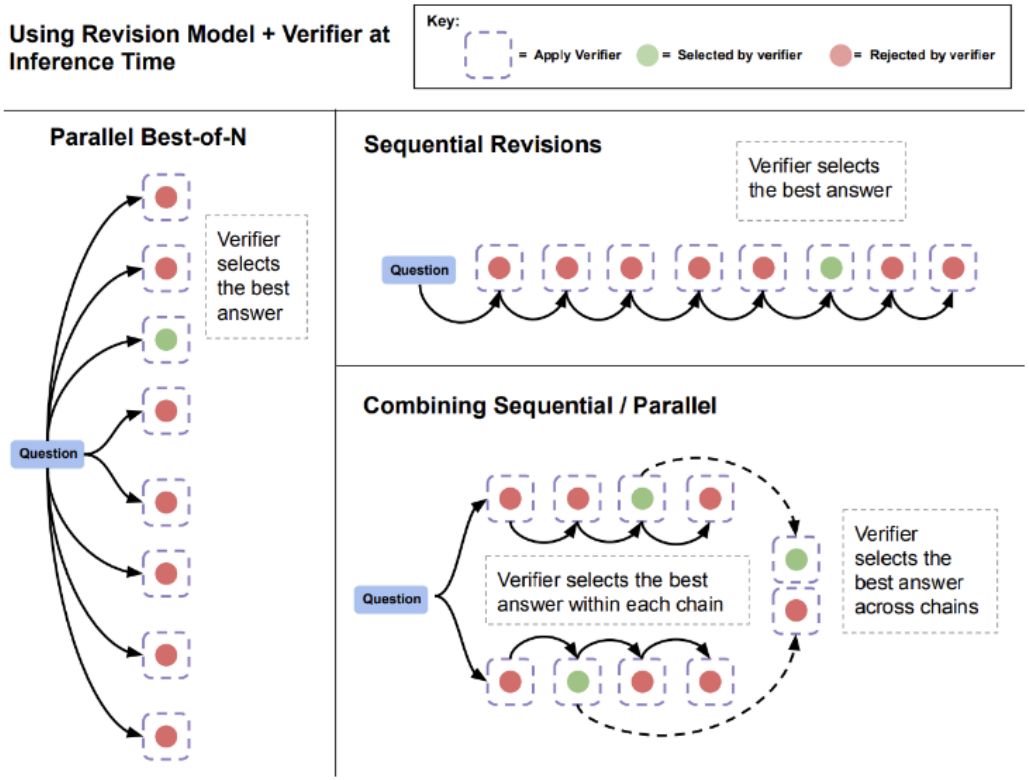
Best-of-N Sampling with Verifier



Beam Search: Process Reward + Verifiers



Compute Optimal Strategy:
Combine Parallel Sampling + Sequential Revisions

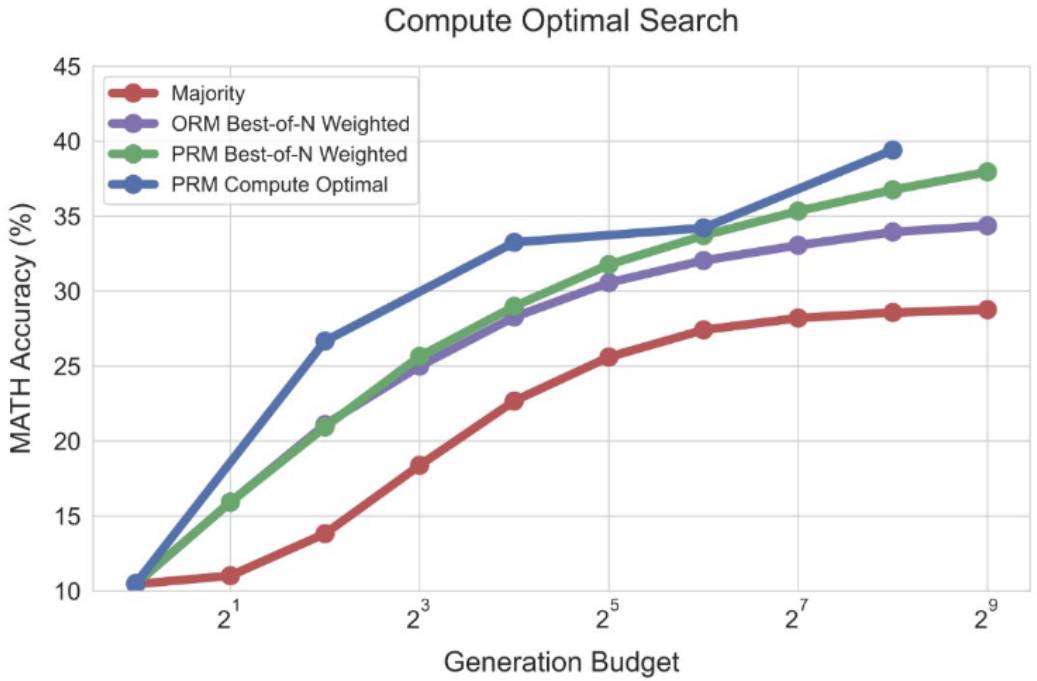


Experiment Methodology

- Dataset: MATH benchmark, which consists of high-school competition level math problems with a range of difficulty levels.
    - MATH dataset consists of 12k train and 500 test questions.
- Model: PaLM 2-S* (Codey) base model.
- Question Difficulty Estimation for Compute-Optimal Scaling
    - bin the model's pass@1 rate (estimated from 2048 samples) on each question in the test set into five quantiles of increasing difficulty levels.
- Process reward model trained with the approach from MATH-Shephard paper
- Revision model is finetuned on trajectories consisting of a sequence of incorrect answers followed by a correct answer

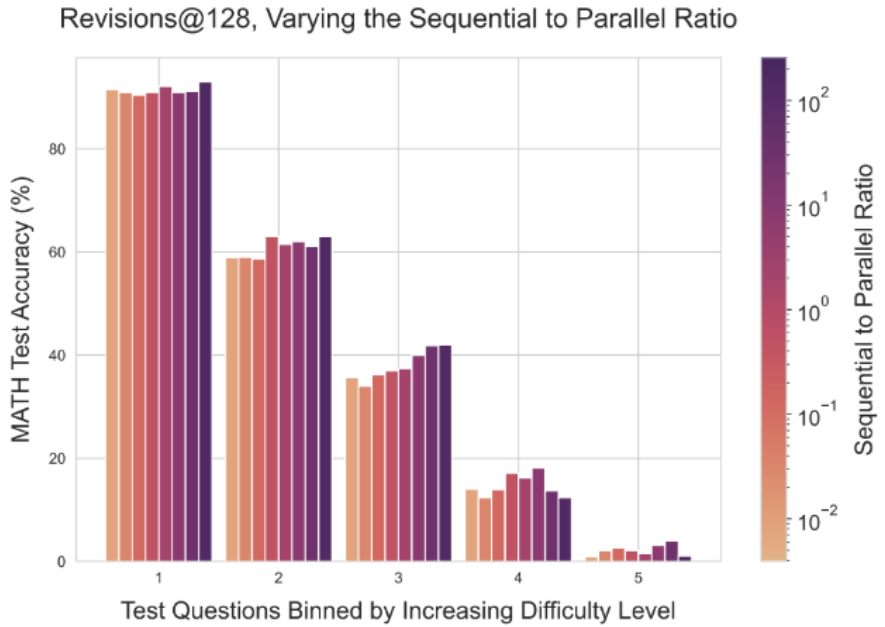
Experimental Results: Test-time Search Against a PRM Verifier

Significant early improvements over best-of-N from compute-optimal scaling



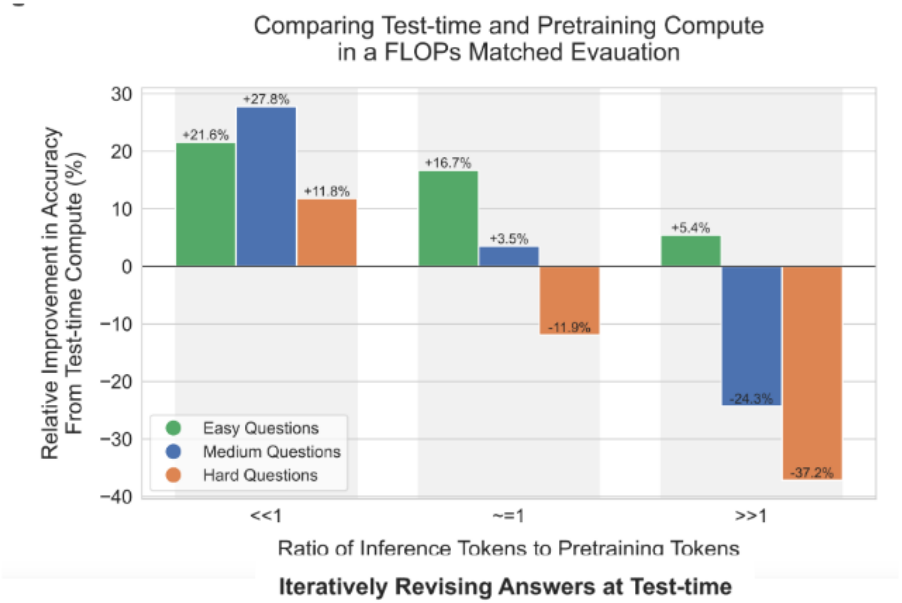
Experimental Results: Varying the Sequential/Parallel Ratio

Using verifier-based selection, we see that the easier questions attain the best performance with full sequential compute. On the harder questions, there is an ideal ratio of sequential to parallel test-time compute.



Experimental Results: Test-time vs Pretraining FLOPs

- Easy, medium questions: additional test-time compute often preferable to scaling pretraining.
- Most challenging questions: very little benefits from scaling up test-time compute.



What We Will Cover

- Large Language Monkeys: Scaling Inference Compute with Repeated Sampling
- Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
- Archon: An Architecture Search Framework for Inference-Time Techniques

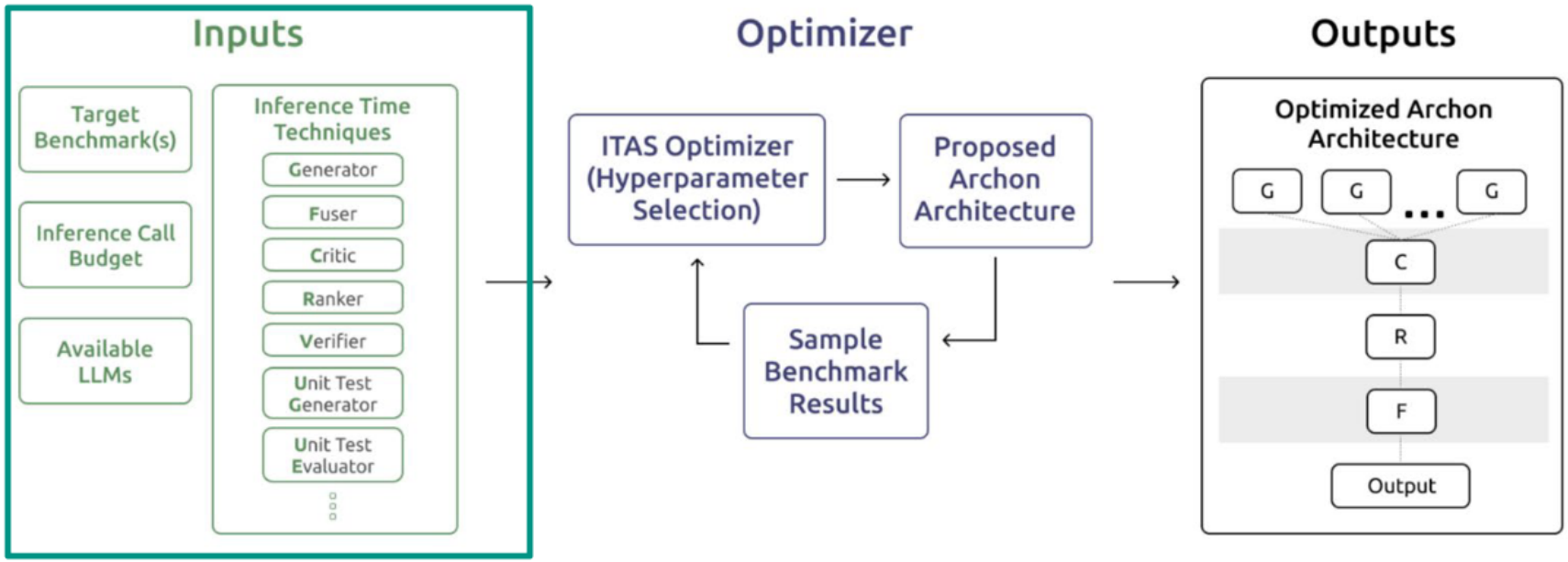
Problem Statement

How can we build capable models by mixing and matching different inference time compute scaling operations?

How can we effectively leverage different models' cost and capability trade-offs to optimize the pareto frontier?

How to optimize assigning inference time compute to tasks?

Archon: An Architecture Search Framework for Inference-Time Systems



Archon: An Architecture Search Framework for Inference-Time Techniques, Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, Etash Guha, E Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, Azalia Mirhoseini, 2024

Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

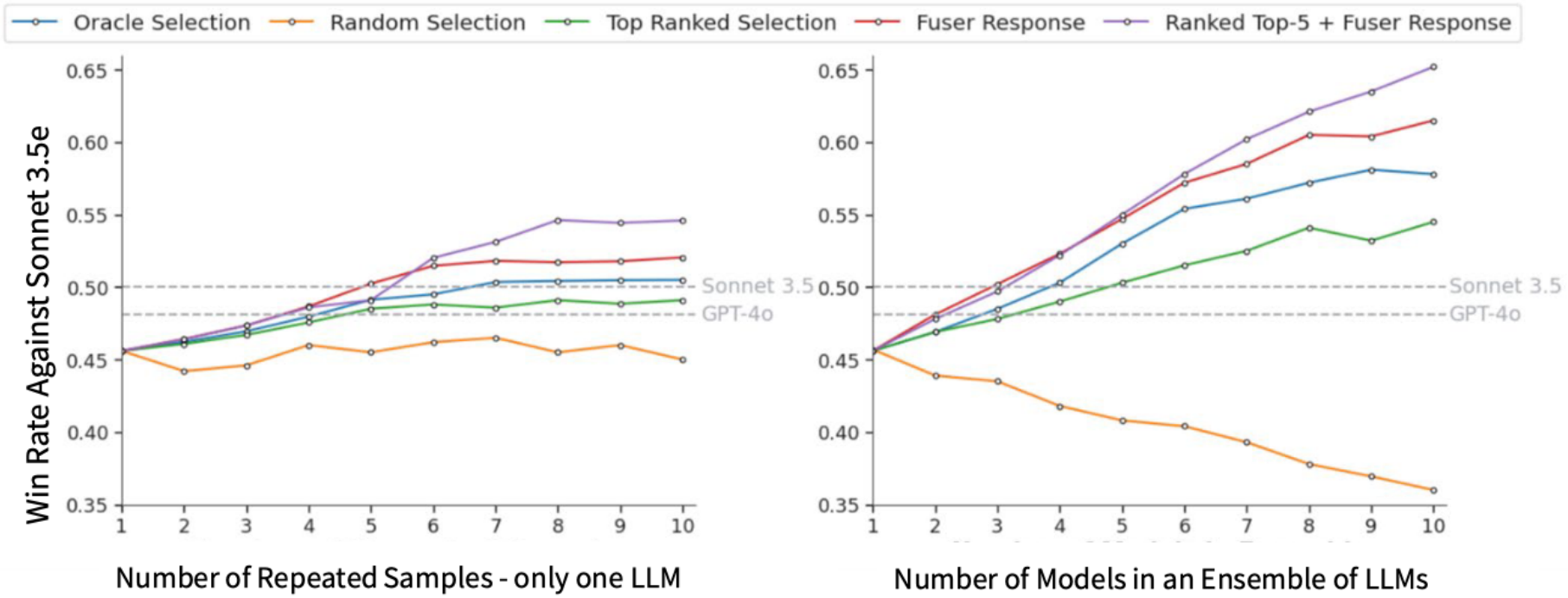
Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

Inference Time Operations

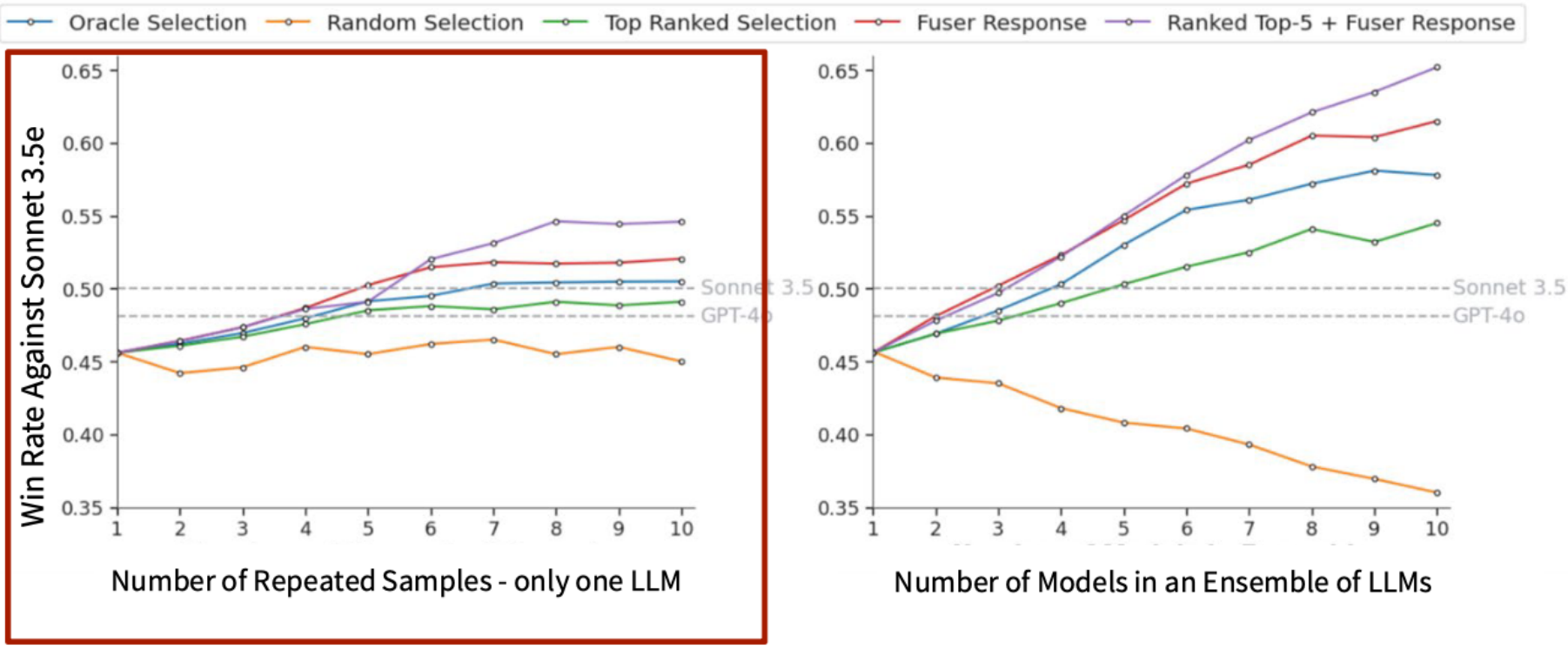
Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

Inference Time Operations Improve Results over Oracle-Based Verifiers

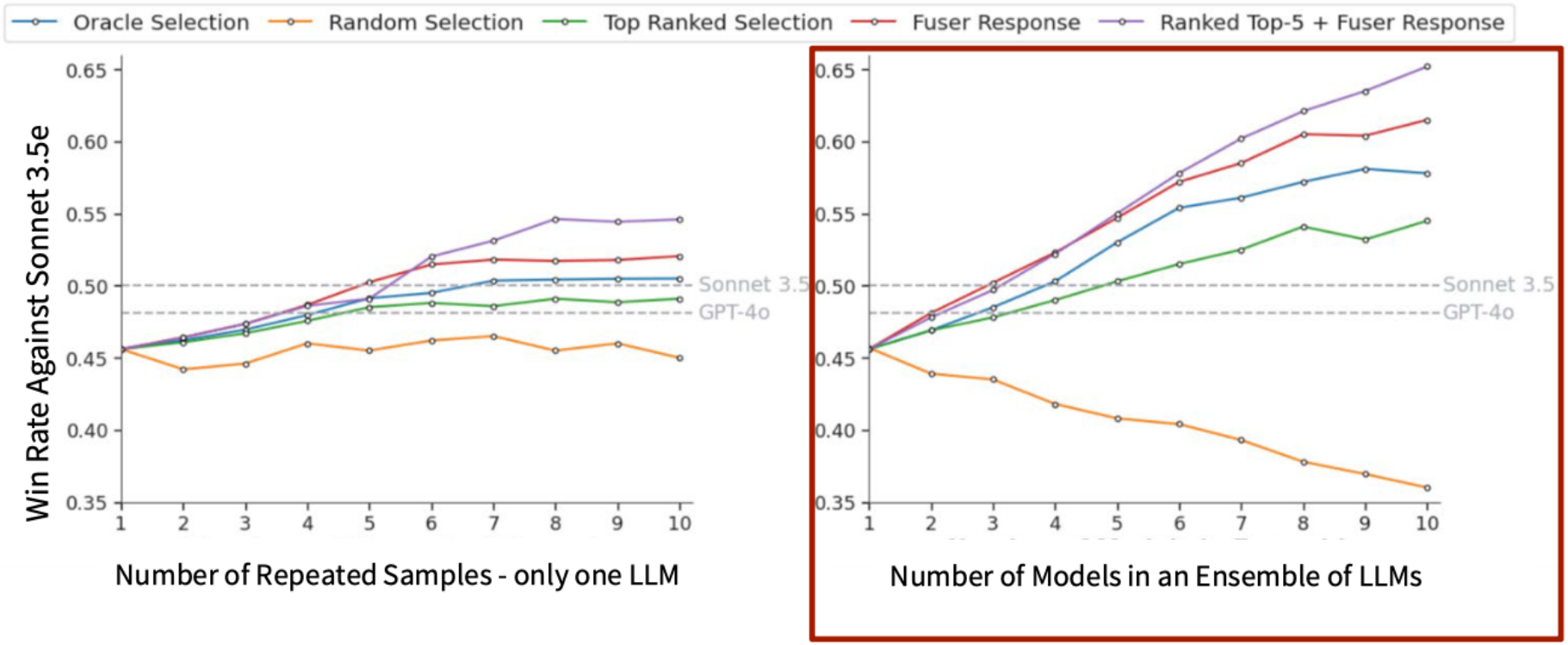


Archon: An Architecture Search Framework for Inference-Time Techniques, Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, Etash Guha, E Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, Azalia Mirhoseini, 2024

Inference Time Operations Improve Results over Oracle-Based Verifiers



Inference Time Operations Improve Results over Oracle-Based Verifiers



Archon Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

Natural Language Unit Tests

Problem: Check for balanceness of round brackets in an input string.
Example Input: "Some random text (with nested (brackets [inside] and even more)) closing brackets)) here!"

LLM-Based Unit Tests:

- Given a string that starts with a closing bracket like ")(", the solution should output "NO" immediately
- Given a string with an odd number of brackets, the solution should output "NO"
- Given a string where brackets match until the very end but then has an extra bracket like "())", the solution should output "NO"
- Given a string that has correct pairs but in wrong order like ")(()", the solution should output "NO"
- When a closing bracket is encountered, it must match the most recently opened bracket that hasn't been matched yet

Natural Language Unit Tests

Problem: Check for balanceness of round brackets in an input string.
Example Input: "Some random text (with nested (brackets [inside] and even more)) closing brackets)) here!"

LLM-Based Unit Tests:

- Given a string that starts with a closing bracket like ")(", the solution should output "NO" immediately
- Given a string with an odd number of brackets, the solution should output "NO"
- Given a string where brackets match until the very end but then has an extra bracket like "())", the solution should output "NO"
- Given a string that has correct pairs but in wrong order like ")((", the solution should output "NO"
- When a closing bracket is encountered, it must match the most recently opened bracket that hasn't been matched yet

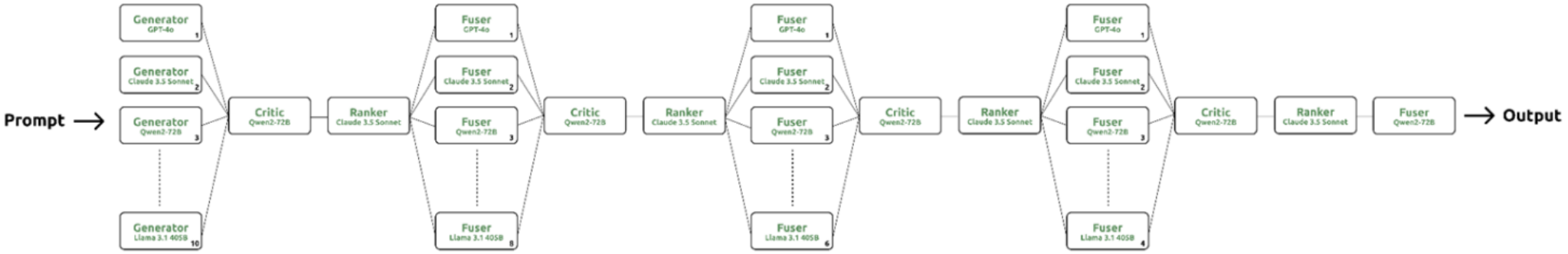
Natural Language Unit Tests

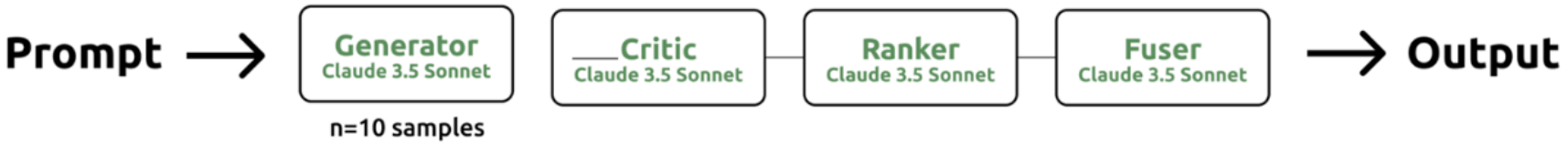
Problem: Check for balanceness of round brackets in an input string.
Example Input: "Some random text (with nested (brackets [inside] and even more)) closing brackets)) here!"

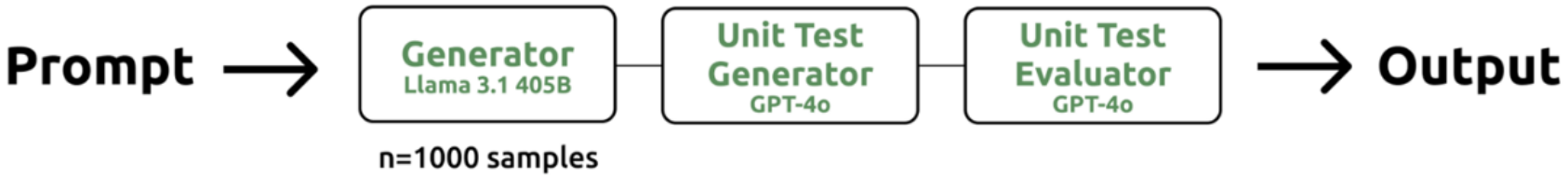
LLM-Based Unit Tests:

- Given a string that starts with a closing bracket like ")(", the solution should output "NO" immediately
- Given a string with an odd number of brackets, the solution should output "NO"
- Given a string where brackets match until the very end but then has an extra bracket like "())", the solution should output "NO"
- Given a string that has correct pairs but in wrong order like ")((", the solution should output "NO"
- When a closing bracket is encountered, it must match the most recently opened bracket that hasn't been matched yet

Diverse Range of Archon Architectures

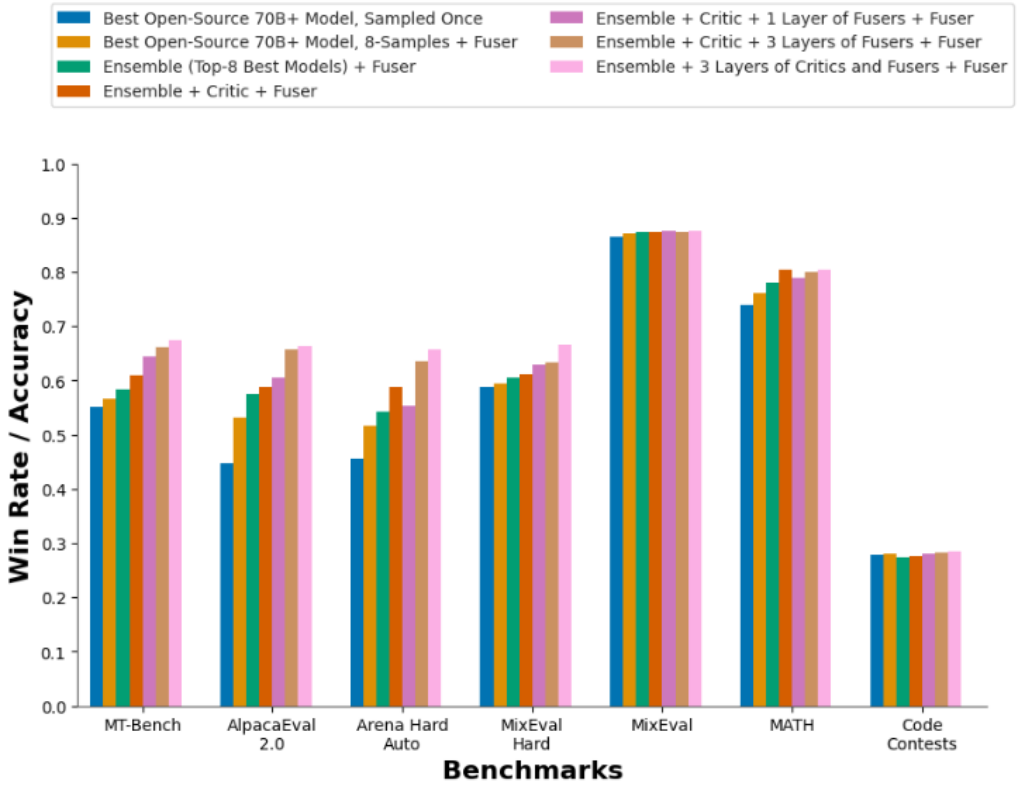




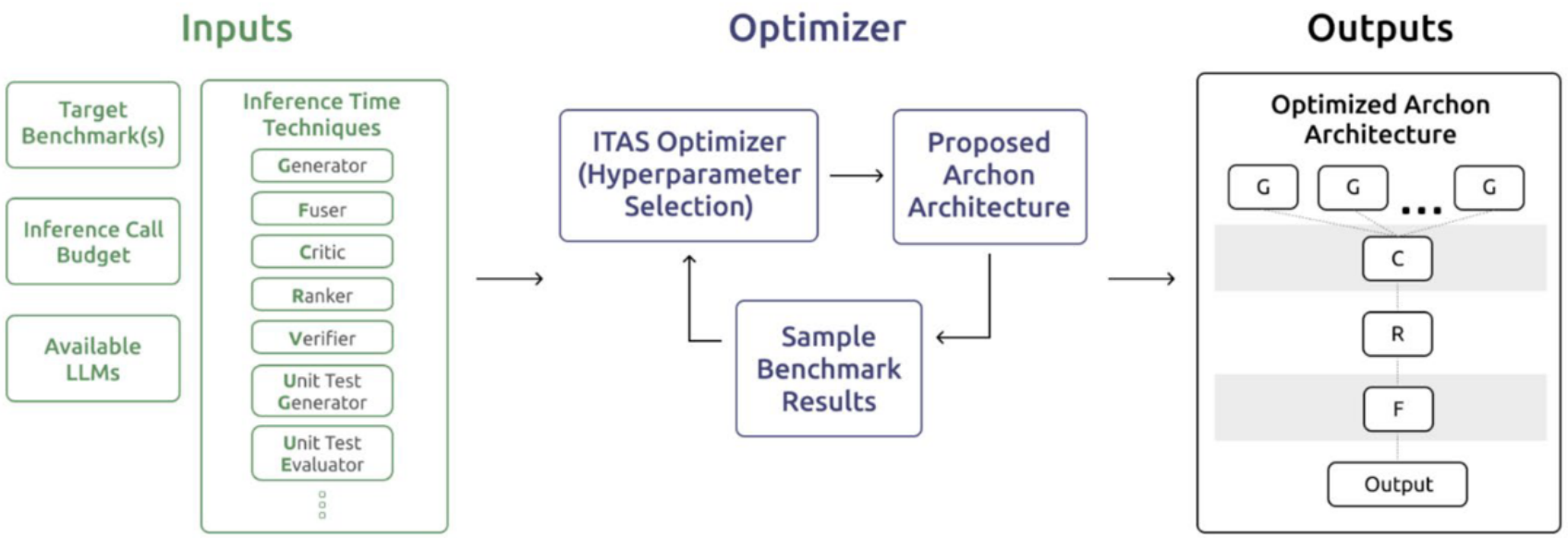


“Deep” Inference-time Architectures

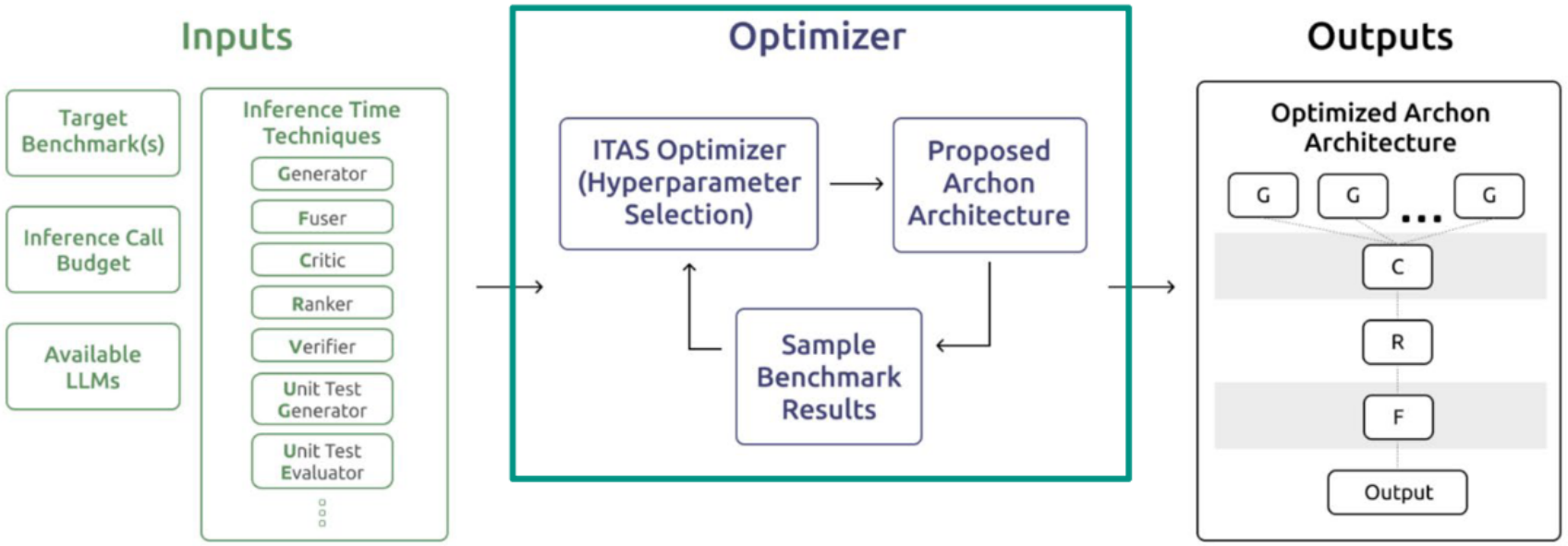
Archon architectures resemble neural architectures in that scaling the number of layers of inference-time methods improves performance!



How to Find the Best Inference Time Architectures?



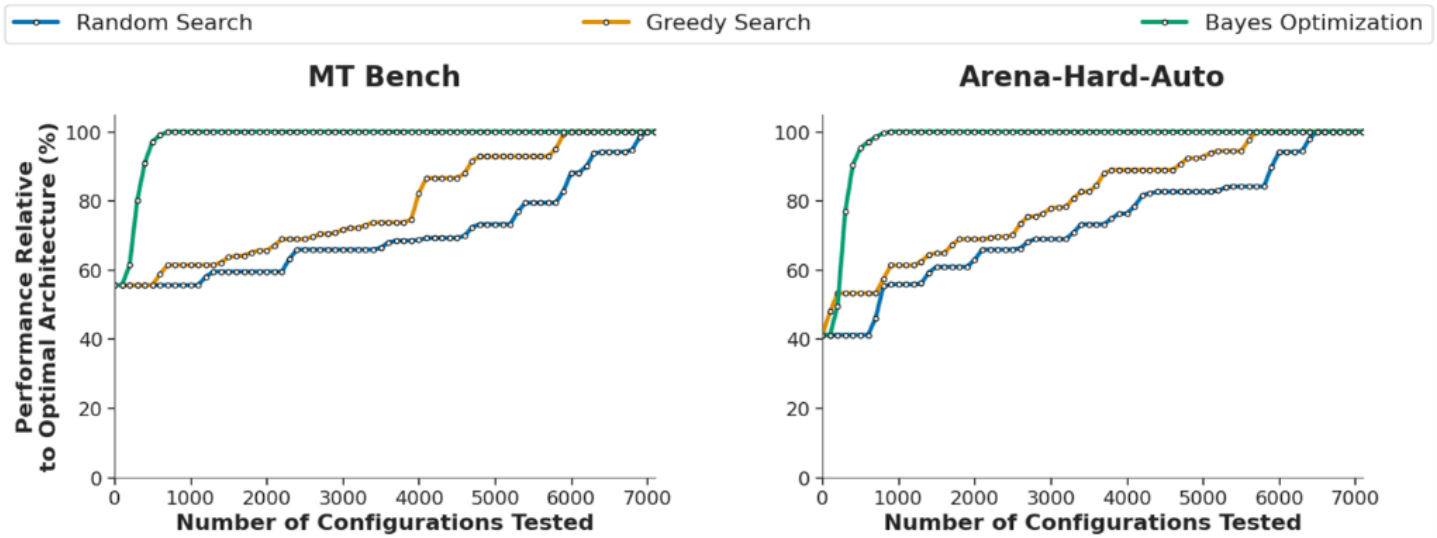
How to Find the Best Inference Time Architectures?



Rules for Archon's Architecture Search Optimizer

- Only one type of inference-time op can be present in any given layer
- Rules about ordering the ops:
  - First layer is for Generator ops only
  - Ranker, Critic, Verifier, Fuser, and Unit Test Generator/Evaluator ops can go anywhere (besides the first layer), but:
  - Critic op must come before a Ranker or a Fuser
  - Unit Test Generators must be followed by an Evaluator op

Inference Time Architecture Search with Bayesian Optimization



Objective: Find the optimal configuration of LLMs for target benchmark(s), given:

- Available LLM(s)
- Inference Time Ops
- Inference Call Budget

Archon's Strong Performance with Open Source, Closed Source, and All Source Models

Archon outperforms GPT-4o and Claude-3.5-Sonnet in **pass@1** accuracy/win-rate by an average of 14.1% on instruction-following, reasoning, math, and coding tasks.

			MT Bench	Alpaca Eval 2.0	Arena Hard Auto	MixEval Hard	MixEval	MATH
	LLM Systems / ARCHON	Infer. Calls	W.R.	L.C. W.R.	W.R	Acc.	Acc.	Pass @1
	GPT-4o	1	44.2%	57.8%	80.6%	63.4%	87.5%	73.2%
	Claude 3.5 Sonnet	1	N/A	52.7%	81.4%	68.7%	89.1%	73.1%
	Llama 3.1 405B	1	44.1%	40.7%	64.5%	66.0%	88.2%	75.2%
	MoA	19	51.6%	65.4%	84.5%	62.3%	86.9%	73.9%
	MoA Lite	7	45.6%	59.6%	88.3%	60.9%	86.4%	71.8%
Archon – Open Src.	General Purpose	35	67.2%	63.3%	85.6%	65.3%	86.2%	76.6%
Archon – Open Src.	Task Specific	44	71.1%	67.1%	89.6%	67.5%	88.8%	81.9%
Archon – Closed Src.	General Purpose	32	72.7%	63.9%	86.2%	67.5%	87.2%	77.9%
Archon – Closed Src.	Task Specific	40	77.0%	68.9%	90.5%	72.6	89.5%	81.6%
Archon – All Src.	General Purpose	35	76.2%	66.4%	89.8%	69.8%	87.3%	79.3%
Archon – All Src.	Task Specific	39	**79.5%**	**69.0%**	**92.5%**	**72.7%**	**89.7%**	**82.1%**

TextGrad: Automatic "Differentiation" via Text Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, James Zou

Mixture-of-Agents (MoA) Enhances Large Language Model Capabilities Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, James Zou, 2024

Self-Improvement : Rewards & Verification

Stanford | ONLINE

What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

Training Verifiers to Solve Math Word Problems

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, John Schulman, OpenAI, 2021

Background and Motivation

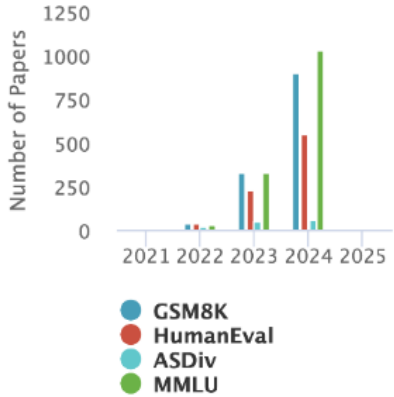
1. LLMs can hallucinate and confidently present wrong solutions for math problems.
2. Lack of suitable math reasoning benchmarks
   a. Language understanding (e.g. SquAD), question answering (e.g. OpenBookQA)
   b. Advanced competition-level problems (e.g. MATH)
   c. Lack of intermediate reasoning steps

The cheese pizza is cut into 12 slices and the pepperoni pizza is cut into 8 slices. If Kate's 6 friends each eat 6 cheese pizza slices and 4 pepperoni pizza slices, how many pizza pies does she need to buy?

6B Finetuning: [incorrect]
Kate's 6 friends eat a total of 6 friends * 6 slices = <<6*6=36>>36 slices.
This is a total of 36 slices + 8 slices = <<36+8=44>>44 slices needed to buy.
Therefore, Kate needs 44 slices / 12 slices/pizza = <<44/12=3.6666666666666665>>3 pizza pies.
A: 3

Contributions

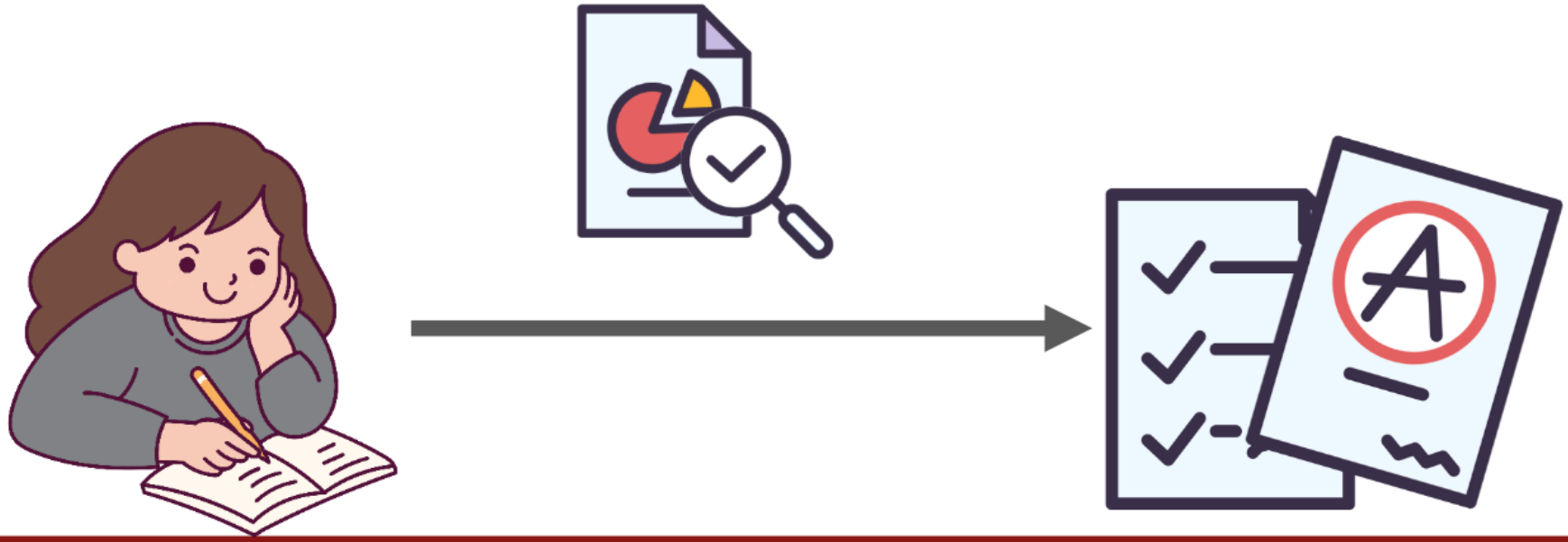
1. GSM8K dataset
   - Consist of 8.5K **grade school** math problems
   - Why GSM8K Stands Out?
     - High Quality: Rely on human workers to create problems
     - High Diversity: Cover various mathematical concepts
     - Focus on **multi-step reasoning**
     - Collect natural language solution rather than as pure math expressions



Contributions

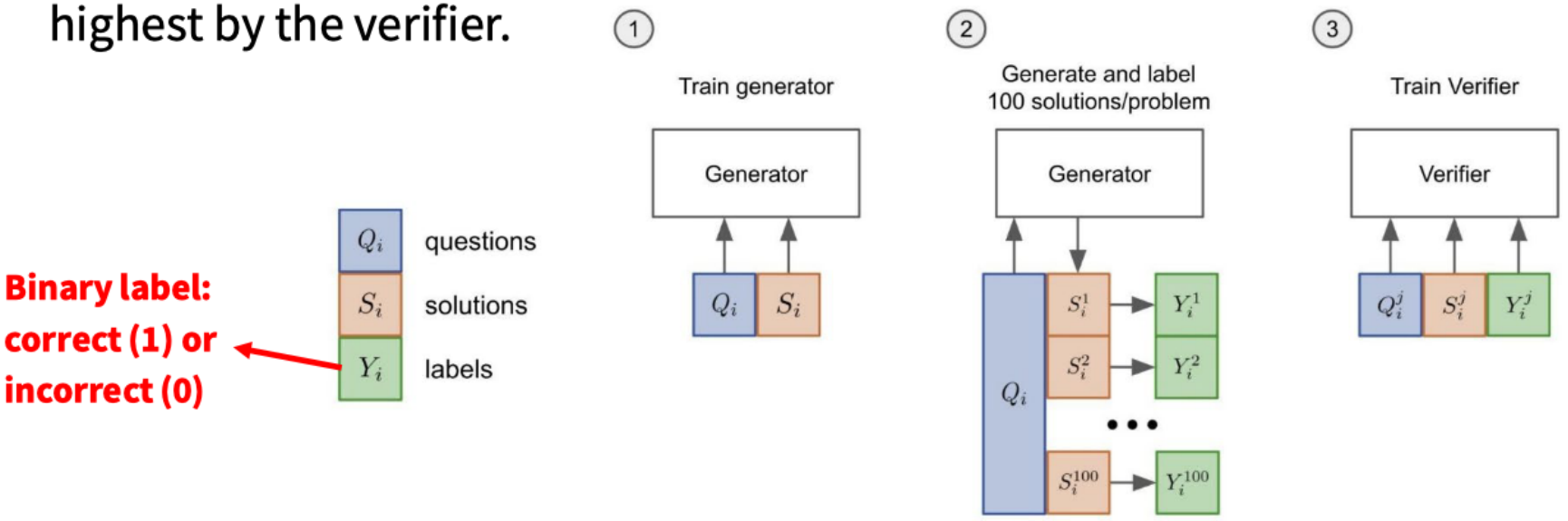
2. Trained a verification model

- The verifier outputs the probability that the solution is correct.



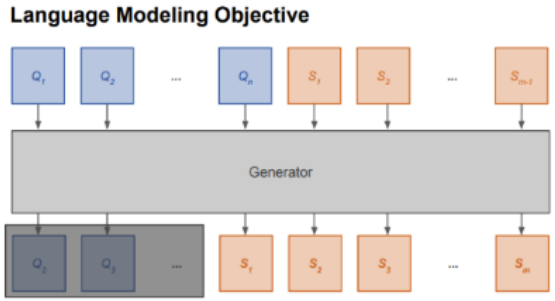
Methodology

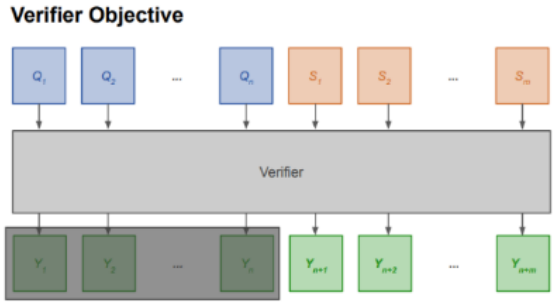
- Train verifiers to judge the correctness of model completions.
- At test time, generate many candidate solutions and select the one ranked highest by the verifier.



Verifier Architecture

- The verifier is trained with a joint objective:
  - Correctness prediction (correctness or incorrectness)
  - Original language modeling objective (next token prediction loss)
- The verifier is a language model, with a small scalar head that outputs predictions on a per-token basis





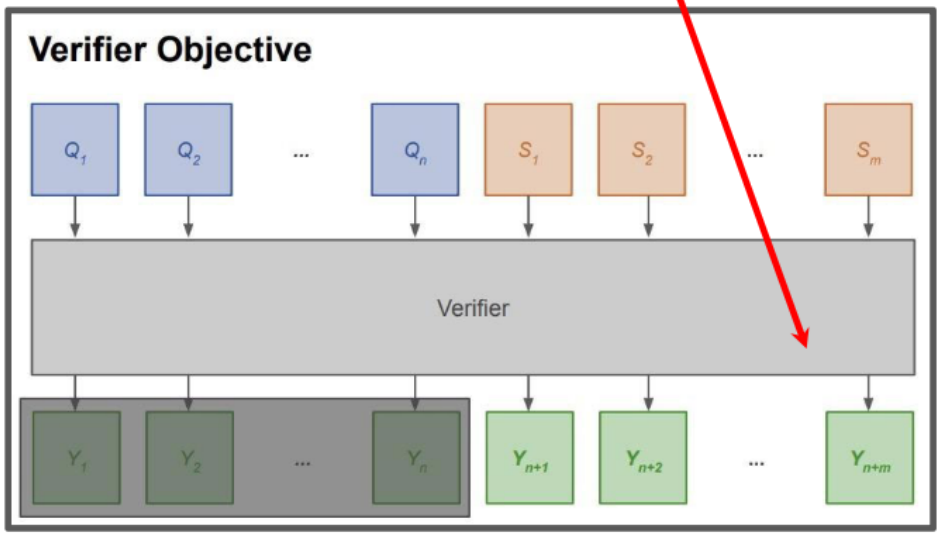
The tokens in questions are masked out and only considers the loss corresponding to tokens in the solution

Verifier Training Process

- Finetune a model (the “generator”) for 2 epochs on the training set
- Sample 100 completions from the generator for each training problem and label each solution as correct or incorrect.
- Train a verifier for a single epoch on this dataset.

Methodology: Verification Design Ablation

- Verification Ablations
  - Sentence-level: make a single scalar prediction conditioned on **the entire generated solution**
  - Token-level: make a scalar prediction **after each token in the solution**
    - The final solution score is the score after last token



Methodology: Verification Example

Question & Model Generation	Verifier Pred.	Actual Score
Claire makes a 3 egg omelet every morning for breakfast. How many dozens of eggs will she eat in 4 weeks? Claire makes 3 omelets every morning, so she eats 3*7=<<3*7=21>>21 omelets per week She eats 21 omelets per week so over 4 weeks she will eat 4*21=<<21*4=84>>84 omelets There are 12 in 1 dozen so she will eat 84/12=<<84/12=7.0>>7 dozens of eggs #### 7	✅	✅
Howard spends $8 dollars at the arcade on Monday. On Tuesday, he spends twice as much at the arcade as he did on Monday. On Wednesday, he spends 4 times as much at the arcade as he spent on Tuesday. If he originally had $100, how much money does he have left? He spent $8 dollars on Monday and $8*2= $<<8*2=16>>16 dollars on Tuesday. He spent $16 dollars on Tuesday and $16*4= $<<16*4=64>>64 dollars on Wednesday. He has $100 dollars and spent $64 dollars so he has 100-64=<<100-64=36>>36 dollars left. #### 36	❌	❌

Verification vs Finetuning

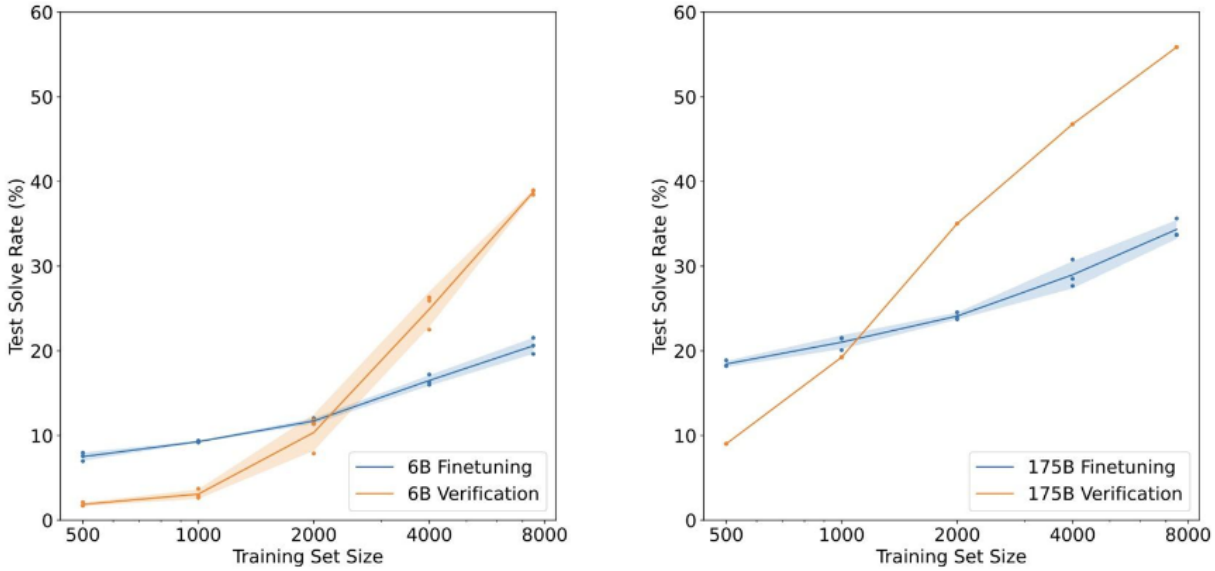
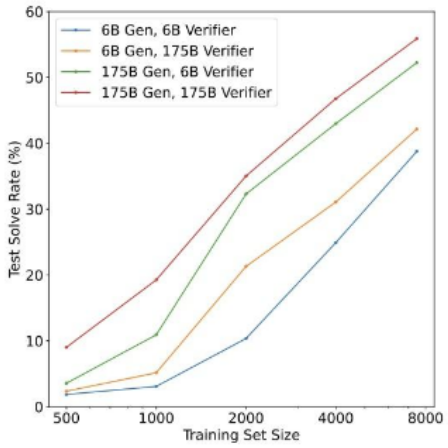


Figure 5: A comparison between finetuning and verification using 6B and 175B model sizes. Verification considers 100 solutions per problem. Mean and standard deviation is shown across 3 runs, except for 175B verification which shows only a single run.

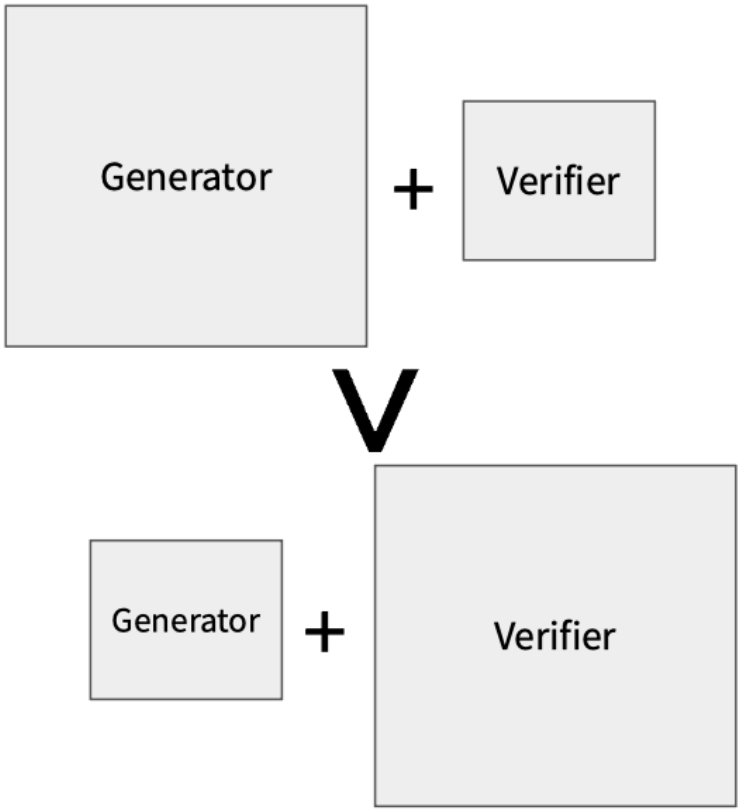
Verification is not beneficial at smaller dataset sizes (overfitting).

With sufficiently large datasets, verifiers provide a significant performance boost.

Experimental Result: Verification Ablations

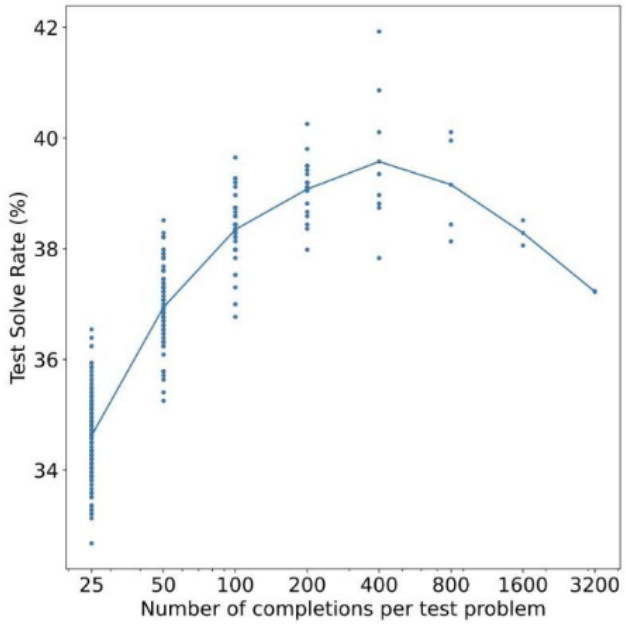


(c) Performance when varying the size of the generator and the verifier in isolation. Increasing the size of the generator has a larger impact than increasing the size of the verifier.



Using a large generator with a small verifier performs significantly better than using a small generator with a large verifier.

Experimental Result: Test Time Compute



Performance improves as we increase the number of completions up to 400.

What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

Let’s Verify Step by Step

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, Karl Cobbe, OpenAI, 2023

Methodology

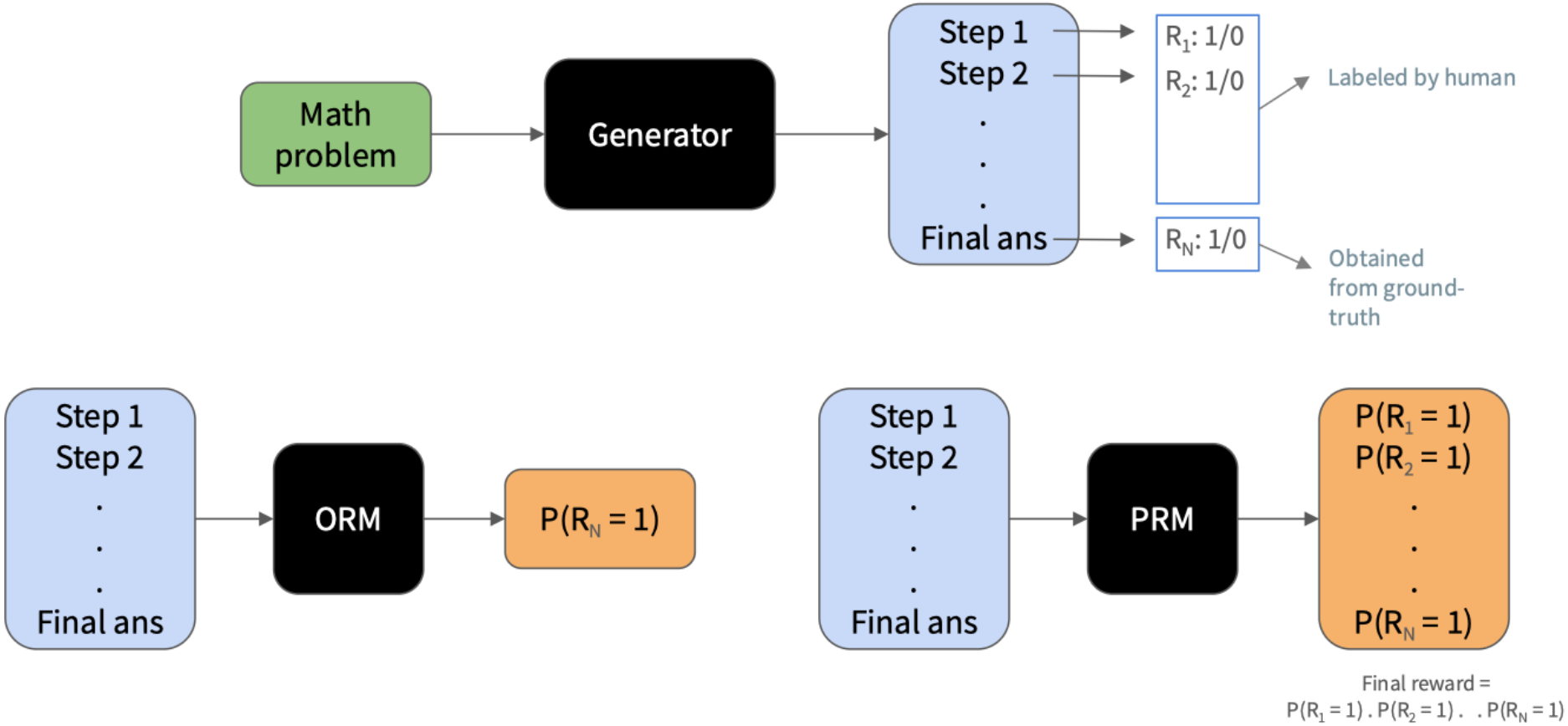
Challenge

- LLMs hallucinate and make logical errors
- A **mistake in a single step** can derail a math solution

Solution

- Verify LLM-generated solutions using one of:
  - Outcome Reward Model (**ORM**) – Assign reward to entire solution
  - Process Reward Model (**PRM**) – Assign reward to each step of the solution
- Choose sample with highest reward

Method


Math problem
Generator
Step 1
Step 2
.
.
.
Final ans
R_1: 1/0
R_2: 1/0
R_N: 1/0
Labeled by human
Obtained from ground-truth
Step 1
Step 2
.
.
.
Final ans
ORM
$P(R_N = 1)$
Step 1
Step 2
.
.
.
Final ans
PRM
$P(R_1 = 1)$
$P(R_2 = 1)$
.
.
.
$P(R_N = 1)$
Final reward =
$P(R_1 = 1) . P(R_2 = 1) . \ . P(R_N = 1)$

Why Process Supervision?

Credit Assignment: Provides precise feedback on the correctness of each reasoning step and location of errors.

False Positive Avoidance: Outcome supervision may assign a high reward to a solution that arrives at the correct answer with wrong reasoning.

AI Alignment: Encourages interpretable reasoning and human-endorsed process of solution generation.

Human Feedback Collection

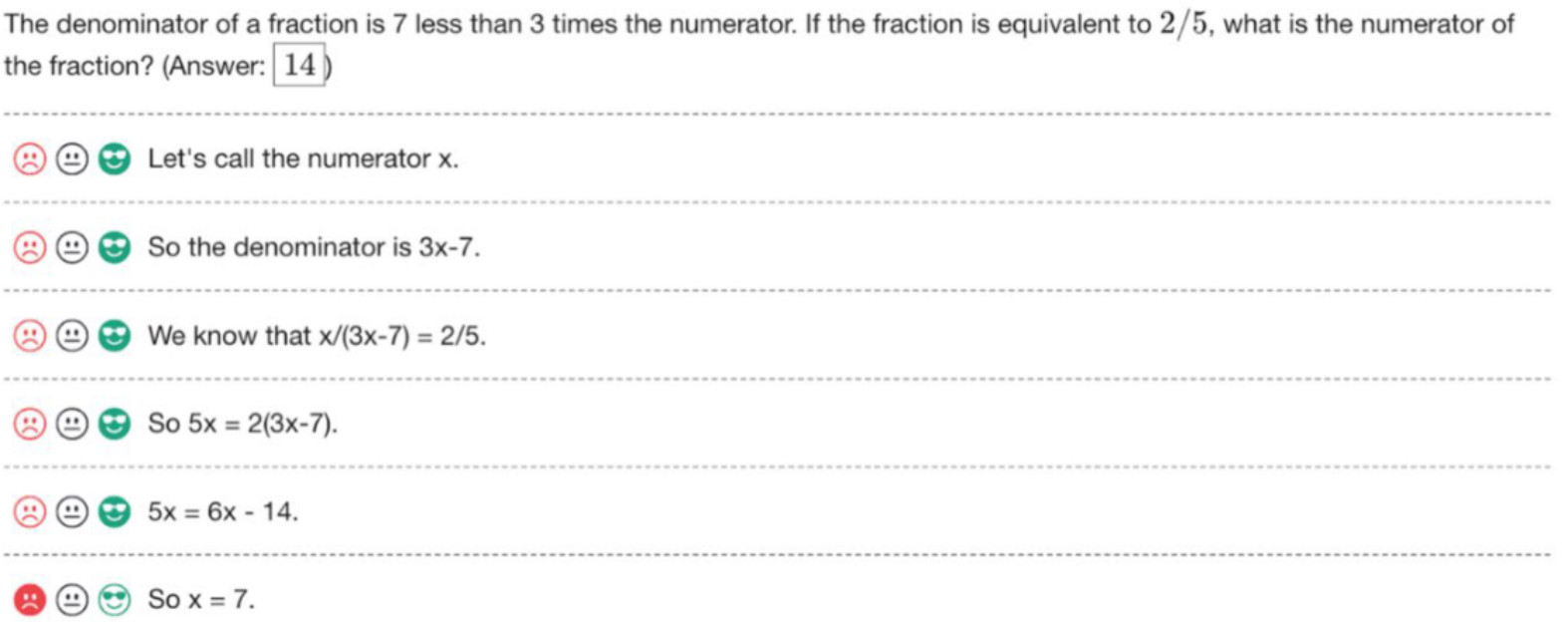
A large amount of human feedback is collected to train the **PRM**.

- A generator model generates a large number of samples per math problem
- **Active Learning:** *Convincing wrong answer solutions* are chosen to be labeled by humans.
  - Solutions that are rated highly by the current best PRM but result in the wrong answer.
  - Found to be **2.6x more data efficient** than choosing random samples for labeling.
- Each step is labeled as positive/negative/neutral
- **PRM800K:** Open-source dataset of 800k step-level labels for 12k math problems

Human Feedback Collection

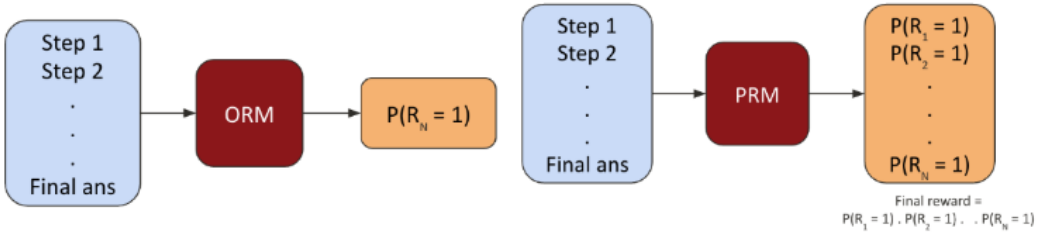
A large amount of human feedback is collected to train the **PRM**.

- A gene
- **Active** ed by humar
  - Sc
  - Fo
- Each s
- **PRM8(** lems



Training and Eval

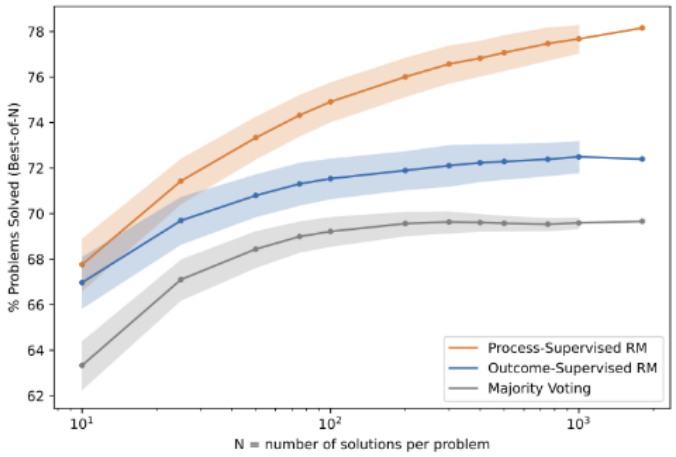
- **Generator:** GPT-4 base model
- **ORM and PRM:** Fine-tuned from GPT-4



- **Fine-tuning data:**
  - ORM: Pairs of (generator sample, final answer correctness ("correct"/"wrong"))
  - PRM: Pairs of (generator sample, step-wise correctness from PRM800K)
- **ORM final score:** Score of the final token in the completion
- **PRM final score:** Product of "correct" probabilities across steps

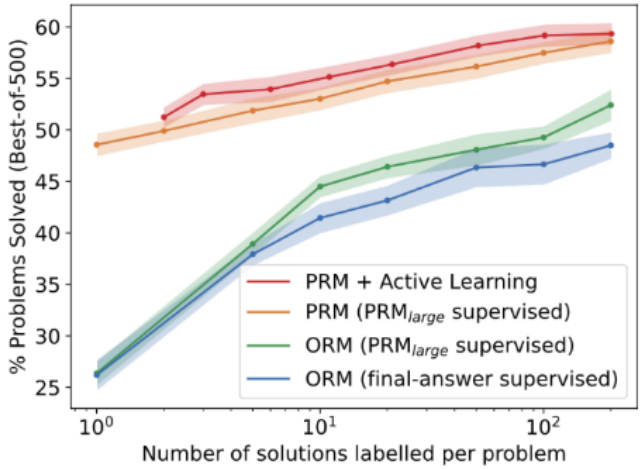
Results: PRM vs ORM

- PRM outperforms ORM and majority voting
- Performance gap widens as N increases
- PRM detects correct solutions for some problems with <5% correct answers.



Results: Benefits of Active learning & Large data

- PRM trained with data from active learning outperforms data from randomly selected samples
- Small amount of process supervision and large amount of outcome supervision leads to similar results



Results: *OOD Generalization*

- Trained PRM generalizes well to out-of-distribution (OOD) datasets and outperforms ORM/majority voting.
- PRM can tolerate a moderate amount of distribution shift

	ORM	PRM	Majority Vote	# Problems
AP Calculus	68.9%	**86.7%**	80.0%	45
AP Chemistry	68.9%	**80.0%**	71.7%	60
AP Physics	77.8%	**86.7%**	82.2%	45
AMC10/12	49.1%	**53.2%**	32.8%	84
Aggregate	63.8%	**72.9%**	61.3%	234

What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

Math-Shepherd: Verify and Reinforce LLMs Step-by-Step Without Human Annotations

Peiyi Wang, Lei Li, Zhihong Shao, R.X. Xu, Damai Dai, Yifei Li, Deli Chen, Y.Wu, Zhifang Sui, 2023

Motivation

- Verifying the generated solution has shown potential to improve a model's ability to reason mathematically
- Process-Level Supervision (PRM) has shown more potential for improvement as compared to Outcome-Level Supervision (ORM)
- Improvement has typically been tied to creating **more data**, either by using humans or generative models
    - ***Gathering PRM data is hard, so can we automate this too?***

Contributions

1) Automatic Annotation

1) Evaluate Math-Shepherd as a verifier, as a reward model for reinforcement learning, and as a verifier + reward model

Automatic Annotation

*We define the quality of a **reasoning step** as its potential to reach the **correct answer**.*

For a given reasoning step, we:

1. *Sample N "reasoning processes" that follows it*
2. *Estimate the potential of this step based on the correctness of decoded answers*

Hard Estimate (HE): does it *ever* get the right answer

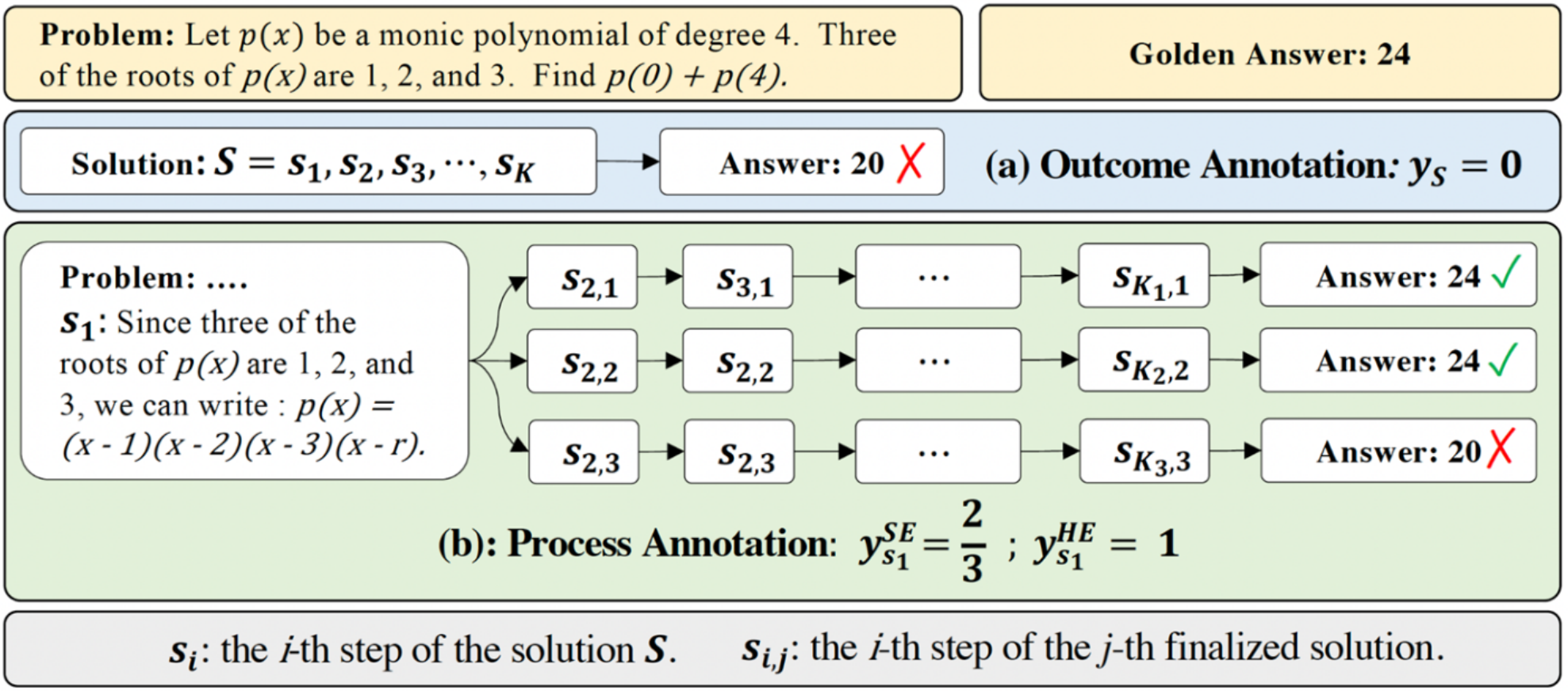
$$y_{s_i}^{HE} = \begin{cases} 1 & \exists a_j \in A, a_j = a^* \\ 0 & \text{Otherwise} \end{cases}$$

Soft Estimate (SE): frequency with which it gets the right answer

$$y_{s_i}^{SE} = \frac{\sum_{j=1}^{N} \mathbb{I}(a_j = a^*)}{N}.$$

a_j: decoded answer
y_{si}: label for step s_i

Automatic Annotation



Generates N reasoning processes (N=3 above) for an intermediate step (s1 above), then uses hard estimation (HE) or soft estimation (SE) to annotate this step based on the decoded answer

Evaluation

Verification

Re-rank multiple outputs generated by LLM

Given problem p:

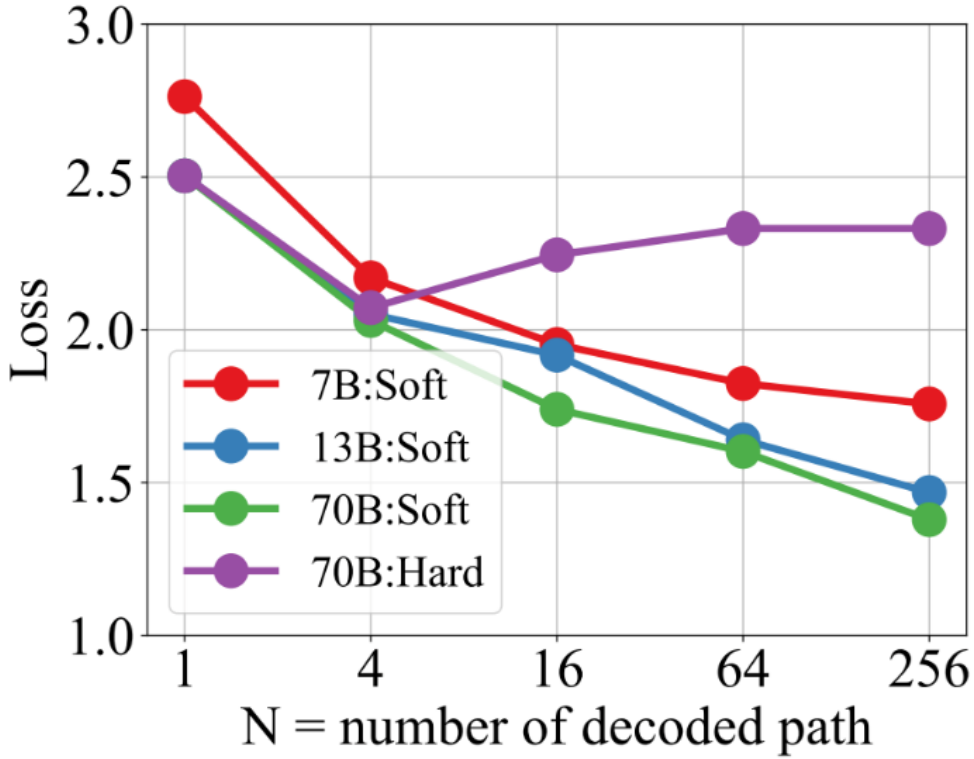
1. Sample N candidate solutions
2. Score them using PRM model
3. Select highest

Reinforcement Learning

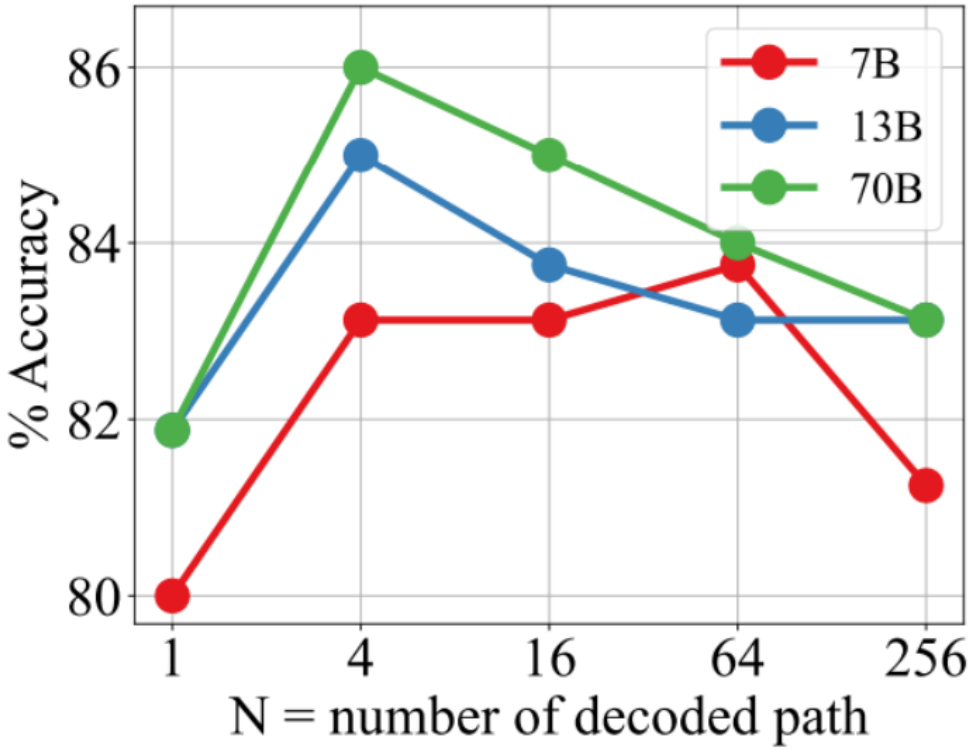
Reinforce with Proximal Policy Optimization (PPO)

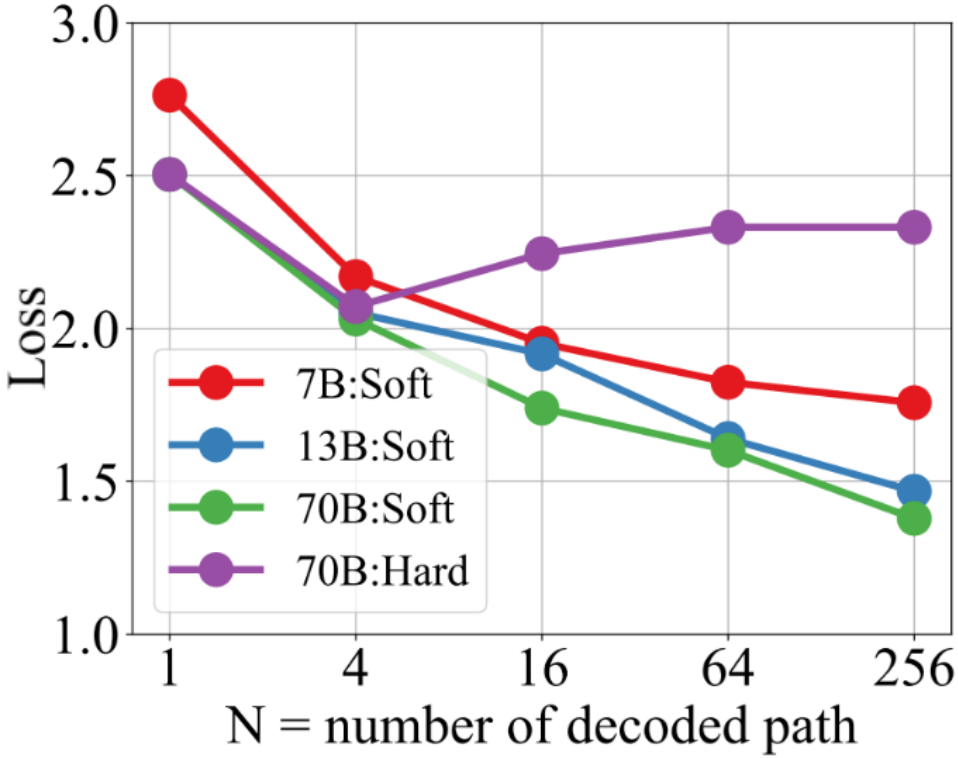
Step-by-step PPO provides rewards @ every step.

Hard vs Soft Annotations



Hard vs Soft Annotations

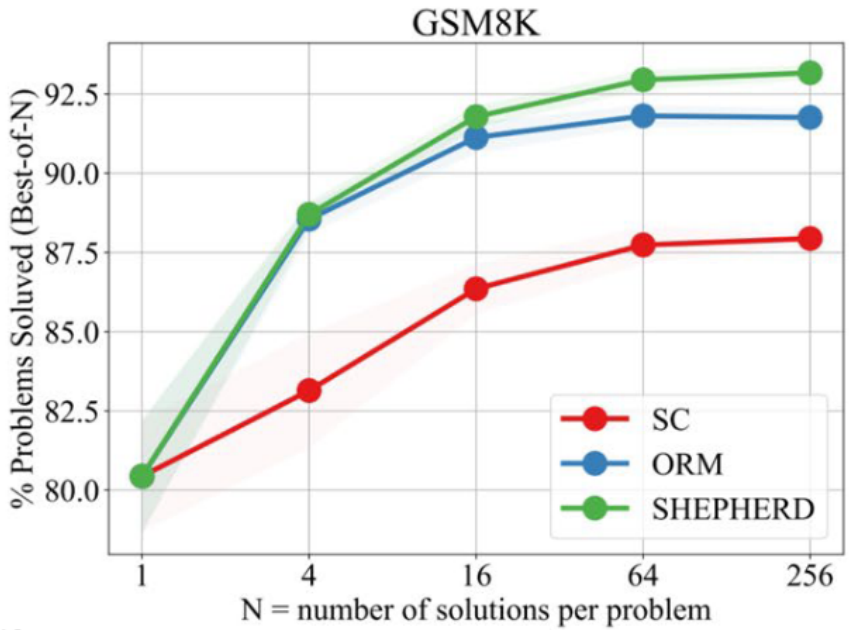


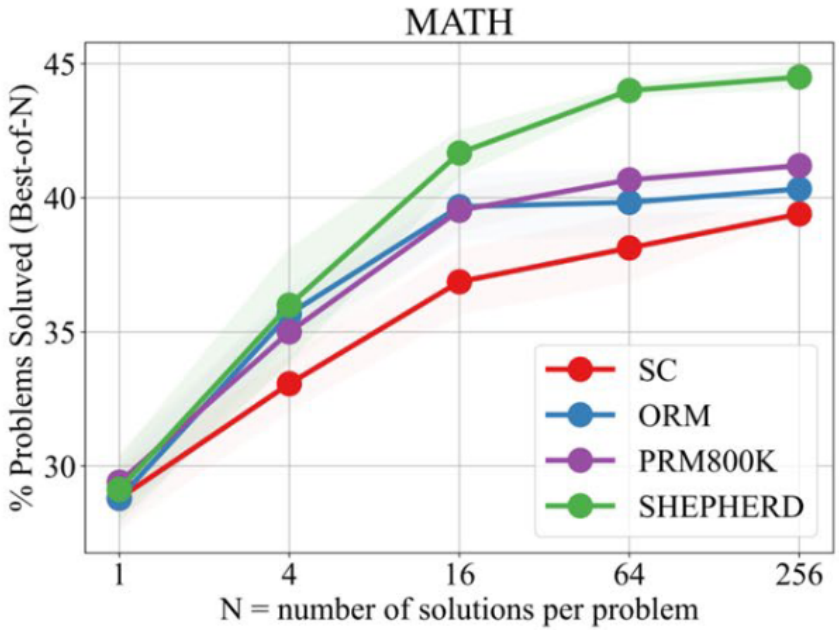


Best performance is achieved at N=4:

Performance of the verifier shows no substantial divergence based on SE or HE

Comparison with Baselines





SC: self-consistency,
ORM: Outcome reward,
PRM800k: PRM with human annotations
SHEPHERD: Step-wise PRM with auto-annotations

Results - Verification

Models	Verifiers	GSM8K	MATH500
LLaMA2-70B: MetaMATH	Self-Consistency	88.0	39.4
	ORM	91.8	40.4
	Self-Consistency + ORM	92.0	42.0
	MATH-SHEPHERD (Ours)	93.2	44.5
	Self-Consistency + MATH-SHEPHERD (Ours)	92.4	45.2
LLemma-34B: MetaMATH	Self-Consistency	82.6	44.2
	ORM	90.0	43.7
	Self-Consistency + ORM	89.6	45.4
	MATH-SHEPHERD (Ours)	90.9	46.0
	Self-Consistency + MATH-SHEPHERD (Ours)	89.7	47.3
DeepSeek-67B: MetaMATH	Self-Consistency	88.2	45.4
	ORM	92.6	45.3
	Self-Consistency + ORM	92.4	47.0
	MATH-SHEPHERD (Ours)	**93.3**	47.0
	Self-Consistency + MATH-SHEPHERD (Ours)	92.5	**48.1**

Table 1: Performances of different LLMs on GSM8K and MATH with different verification strategies. The reward models are trained based on LLama2-70B and LLemma-34B on GSM8K and MATH, respectively. The verification is based on 256 outputs.

Results - RL via PPO

Models	GSM8K	MATH
LLaMA2-7B: MetaMATH	66.6	19.2
+ RFT	68.5	19.9
+ ORM-PPO	70.8	20.8
+ MATH-SHEPHERD-step-by-step-PPO (Ours)	73.2	21.6
Mistral-7B: MetaMATH	77.9	28.6
+ RFT	79.0	29.9
+ ORM-PPO	81.8	31.3
+ MATH-SHEPHERD-step-by-step-PPO (Ours)	**84.1**	**33.0**

Table 2: Performances of different 7B models on GSM8K and MATH with greedy decoding. We use the questions in MetaMATH for RFT and PPO training. Both LLaMA2-7B and Mistral-7B are supervised by Mistral-7B-ORM and -MATH-SHEPHERD.

Results - RL via PPO + verification

Models	Verifiers	GSM8K	MATH500
Mistral-7B: MetaMATH	Self-Consistency	83.9	35.1
	ORM	86.2	36.4
	Self-Consistency + ORM	86.6	38.0
	MATH-SHEPHERD (Ours)	87.1	37.3
	Self-Consistency + MATH-SHEPHERD (Ours)	86.3	38.3
Mistral-7B: MetaMATH +step-by-step PPO (Ours)	Self-Consistency	87.4	42.3
	ORM	87.6	41.3
	Self-Consistency + ORM	89.0	43.1
	MATH-SHEPHERD (Ours)	88.4	41.1
	Self-Consistency + MATH-SHEPHERD (Ours)	**89.1**	**43.5**

Table 3: Results of reinforcement learning and verification combination. The reward models are trained based on Mistral-7B. The verification is based on 256 outputs.

What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

Shrinking the Generation-Verification Gap with Weak Verifiers

Jon Saad-Falcon*, E. Kelly Buchanan*, Mayee F Chen*, Tzu-Heng Huang, Brendan McLaughlin, Tanvir Bhathal, Shang Zhu, Ben Athiwaratkun, Frederic Sala, Scott Linderman, Azalia Mirhoseini, Christopher Ré, 2025

Motivation

- There have been different verification methods proposed, but none of them could close the generation-verification gap of the models.

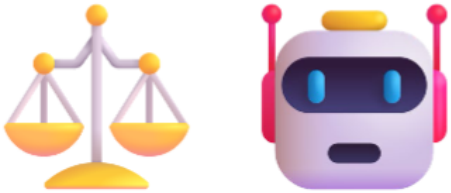
- Can we **reduce the generation-verification gap by scaling the inference compute**, in particular ensembling multiple weak verifiers?

Can Weak Verifiers Help Us Close the Gap?

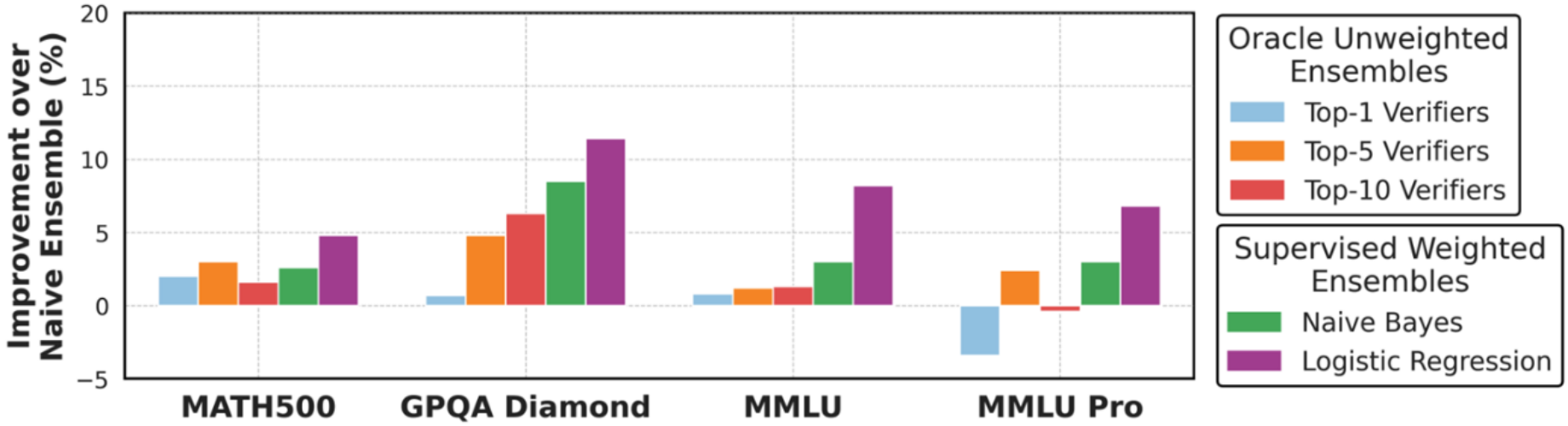
Weak Verifiers = Any function that scores responses imperfectly but provides positive signal

Reward Models (PRMs and ORMs)

LM Judges

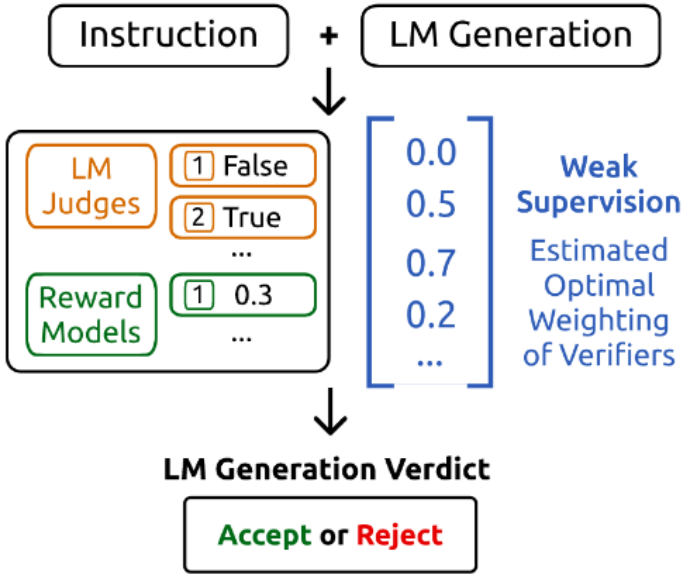


Weighted Verifier Ensembles Outperform Naive Verifier Ensembles



How Weaver Works: *Score⁄Weight⁄Select*

- **Score**: Collect outputs from multiple verifiers, normalize to common scale, and filter low-quality verifiers
- **Weight**: Use weak supervision to estimate verifier accuracies from minimal labeled data (~1%)
- **Select**: Combine weighted scores to choose the highest-confidence response



Weaver Methodology

Weak Supervision: Creating Signal from Noisy Voters

- **Weak Supervision** = A family of statistical methods that aggregate multiple weak, noisy labeling sources to infer true labels by modeling their accuracies and correlations. Weaver was particularly inspired by Snorkel, Ratner et al. (2017)

- Setup: 1<=i<=n queries, 1<=j<=k generated solutions y_{ij}, and "m" verifiers
- Weaver's goal is to predict the probability that a response is correct $\Pr(y_{ij}=1|s_{ij1},...,s_{ijm})$

Weak Supervision: Creating Signal from Noisy Voters

- **Weak Supervision** = A family of statistical methods that aggregate multiple weak, noisy labeling sources to infer true labels by modeling their accuracies and correlations. Weaver was particularly inspired by Snorkel, Ratner et al. (2017)

- Setup: 1<=i<=n queries, 1<=j<=k generated solutions y_{ij}, and "m" verifiers
- Weaver's goal is to predict the probability that a response is correct $\Pr(y_{ij}=1|s_{ij1},...,s_{ijm})$
- Assumption: Each verifier is assumed to capture independent aspects of the correctness
- Under this assumption, we can write the posterior probability of a correct generation as:

$$\Pr(Y=1|S_1=\bar{s}_1,...,S_m=\bar{s}_m)=\frac{\prod_{i=1}^{m}\Pr(S_i=\bar{s}_i|Y=1)\Pr(Y=1)}{\Pr(S_1=\bar{s}_1,...,S_m=\bar{s}_m)}.$$

Weak Supervision: Creating Signal from Noisy Voters

- **Weak Supervision** = A family of statistical methods that aggregate multiple weak, noisy labeling sources to infer true labels by modeling their accuracies and correlations. Weaver was particularly inspired by Snorkel, Ratner et al. (2017)

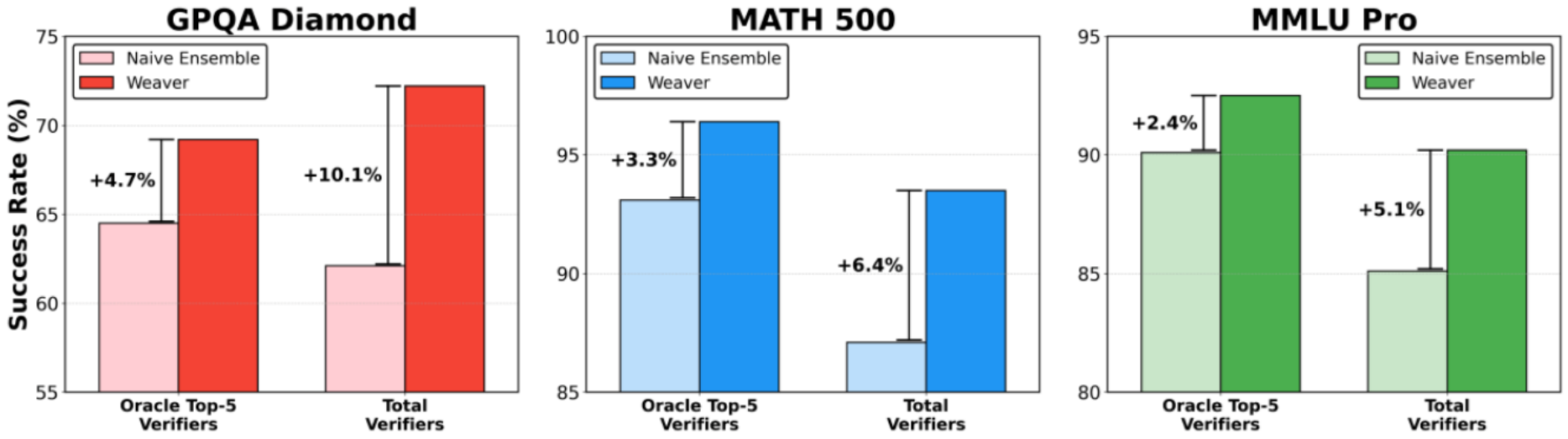
- Setup: 1<=i<=n queries, 1<=j<=k generated solutions y_{ij}, and "m" verifiers
- Weaver's goal is to predict the probability that a response is correct $\Pr(y_{ij}=1|s_{ij1},...,s_{ijm})$
- Assumption: Each verifier is assumed to capture independent aspects of the correctness
- Under this assumption, we can write the posterior probability of a correct generation as:

$$\Pr(Y=1|S_1=\bar{s}_1,...,S_m=\bar{s}_m)=\frac{\prod_{i=1}^{m}\Pr(S_i=\bar{s}_i|Y=1)\Pr(Y=1)}{\Pr(S_1=\bar{s}_1,...,S_m=\bar{s}_m)}.$$

$$\begin{aligned}\Pr(S_i,S_j)&=\Pr(S_i,S_j|Y=1)\Pr(Y=1)+\Pr(S_i,S_j|Y=0)\Pr(Y=0)\\&=\Pr(S_i|Y=1)\Pr(S_j|Y=1)\Pr(Y=1)+\Pr(S_i|Y=0)\Pr(S_j|Y=0)\Pr(Y=0).\end{aligned}$$

$$\Pr(S_i=1)=\Pr(S_i=1|Y=1)\Pr(Y=1)+\Pr(S_i=1|Y=0)\Pr(Y=0).$$

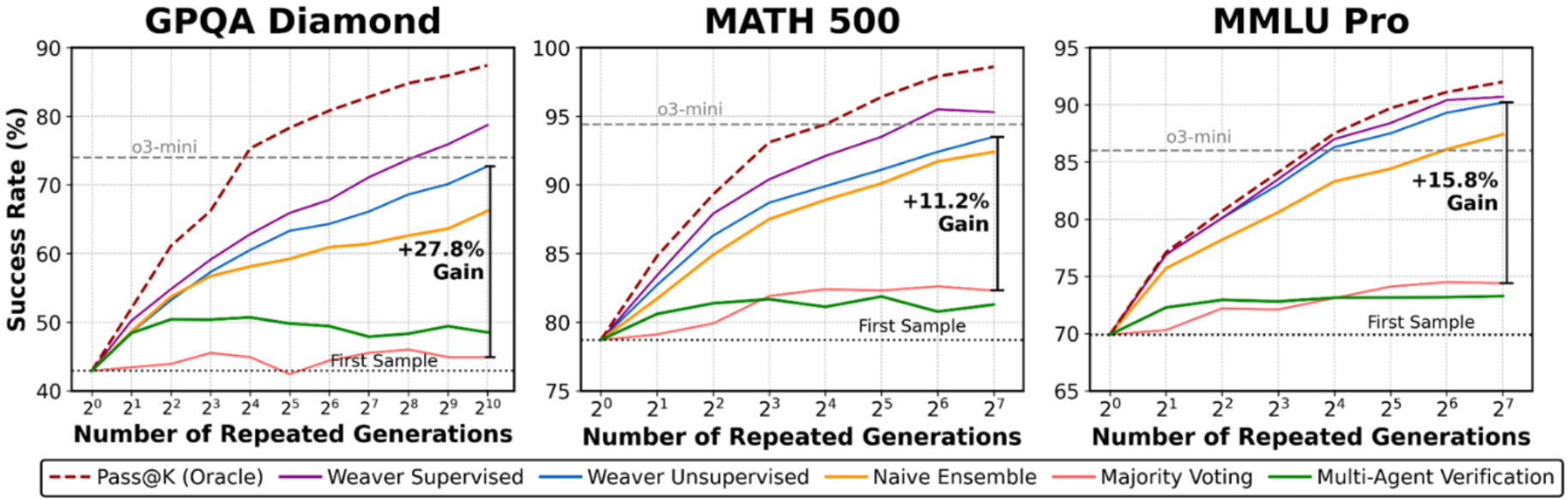
Naive ensemble vs. Weaver's Weak-to-Strong Ensemble



Scaling Verification Compute

- **Sample Count** → More Generations
- **Model Size** → Larger Models for Generation and Verification
- **Verifier Count** → More Models used for Verification
- **Inference Compute** → More FLOPs for Generation / Verification

Scaling Generations Boosts Performance with Weaver

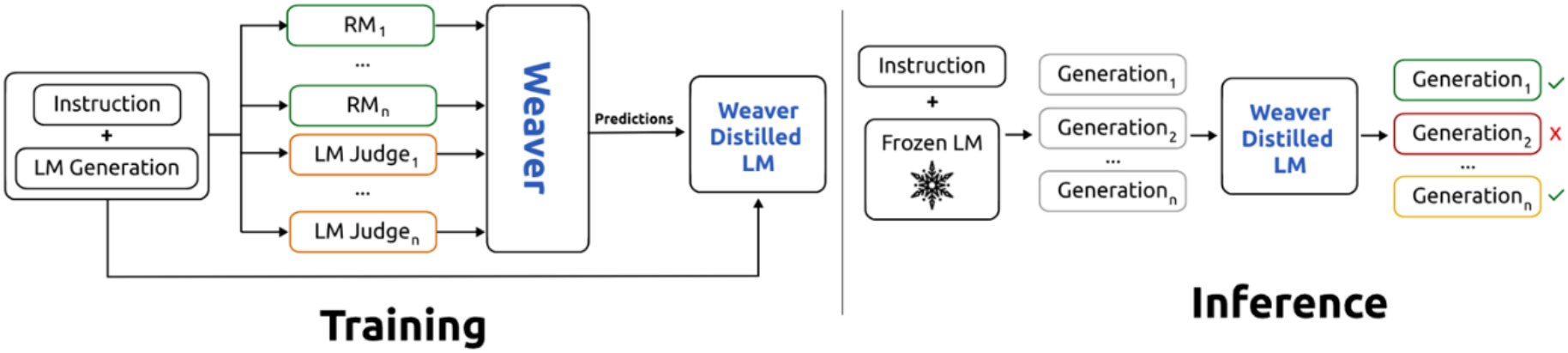


Multi-Agent Verification (MAV) combines multiple "Aspect Verifiers" - off-the shelf LLMs prompted to verify specific aspects of candidate outputs through binary votes. No training is required.

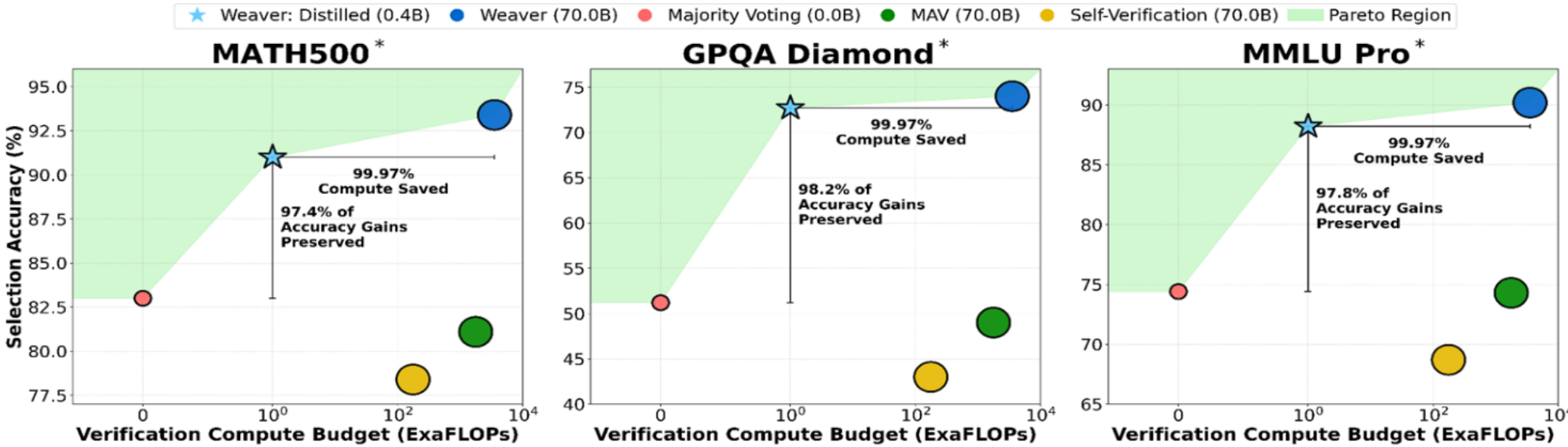
Weaver Reduces Gap between Model Classes: 8B and 70B, 70B and Frontier LM

Generator Model	Verifier Model	Aggregation Strategy	Datasets				Average
			MATH	GPQA Diamond	MMLU College	MMLU Pro	
Llama 3.1 8B Instruct	N/A	Majority Vote	69.0%	30.5%	72.7%	56.4%	57.2%
	8B and below	WEAVER	80.0%	47.1%	85.7%	67.2%	**70.0%**
		Δ w. WEAVER	+11.0%	+16.6%	+13.0%	+10.2%	+12.8%
Llama 3.3 70B Instruct	N/A	Majority Vote	83.0%	47.4%	84.1%	74.4%	72.2%
	72B and below	WEAVER	93.4%	66.4%	94.9%	90.2%	**86.2%**
		Δ w. WEAVER	+10.4%	+9.0%	+10.8%	+15.8%	+14.0%
o3-mini	N/A	First Sample	94.4%	74.0%	92.2%	86.0%	**86.7%**

Weaver Distillation: From 70B to 400M

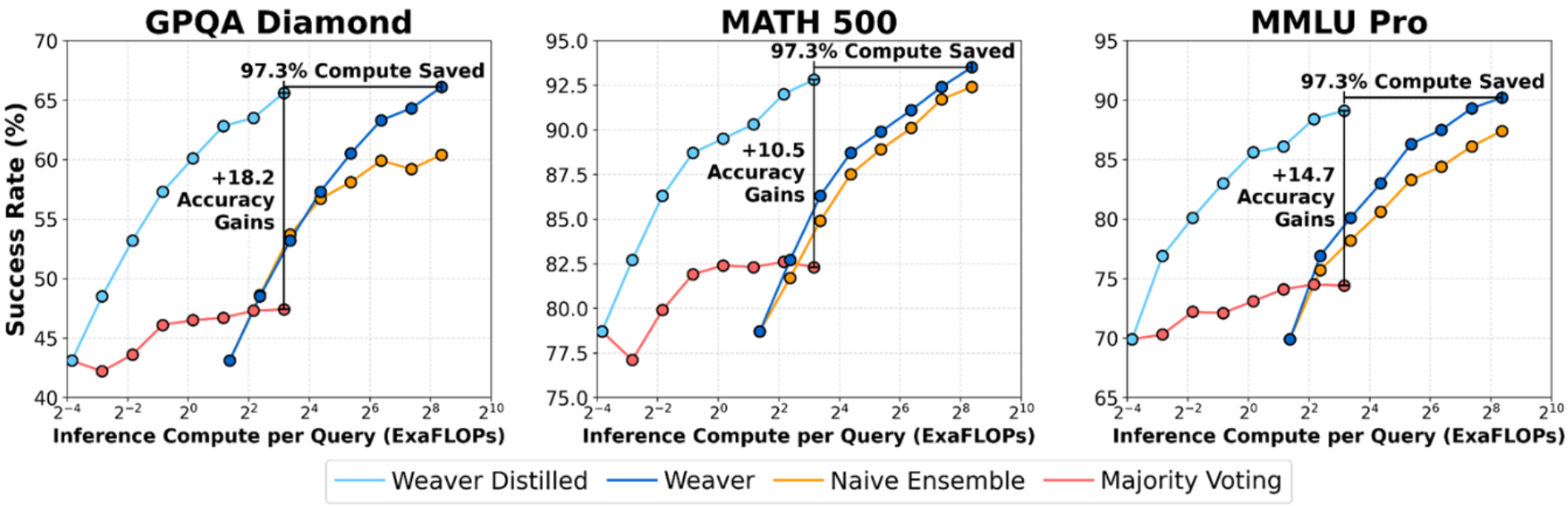


Distilled Weaver:
>97% of Accuracy Gains with 99.97% Less Compute



Multi-Agent Verification (MAV) combines multiple "Aspect Verifiers" - off-the shelf LLMs prompted to verify specific aspects of candidate outputs through binary votes. No training is required.

Weaver Improves the Accuracy-Compute Performance Trade-Offs



What We Will Cover

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

Let's Verify step by step (Lightman et al. 2023)

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2025)

Recap and Takeaways

Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021)

- Verification (outcome-based reward) improves performance during both training and inference.

Let's Verify step by step (Lightman et al. 2023)

- Process supervision is more effective than outcome supervision, but outcome reward also scales with more data.

Recap and Takeaways

Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations (Wang et al. 2023)

- Scaling verifier training data is crucial, and data labeling can be automated.
- A strong reward model is highly effective during RL/PPO training.

Shrinking the Generation-Verification Gap with Weak Verifiers (Saad-Falcon et al. 2023)

- An ensemble of weak verifiers can outperform a larger single model.
- A distilled model from multiple verifiers can surpass the performance of all individual verifiers.

Self-improvement Techniques

Stanford | ONLINE

Learning from Feedback with Tools/Code

What We Will Cover

ReAct: Synergizing Reasoning and Acting in Language Models

RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning

Constitutional AI: Harmlessness from AI Feedback

What We Will Cover

ReAct: Synergizing Reasoning and Acting in Language Models

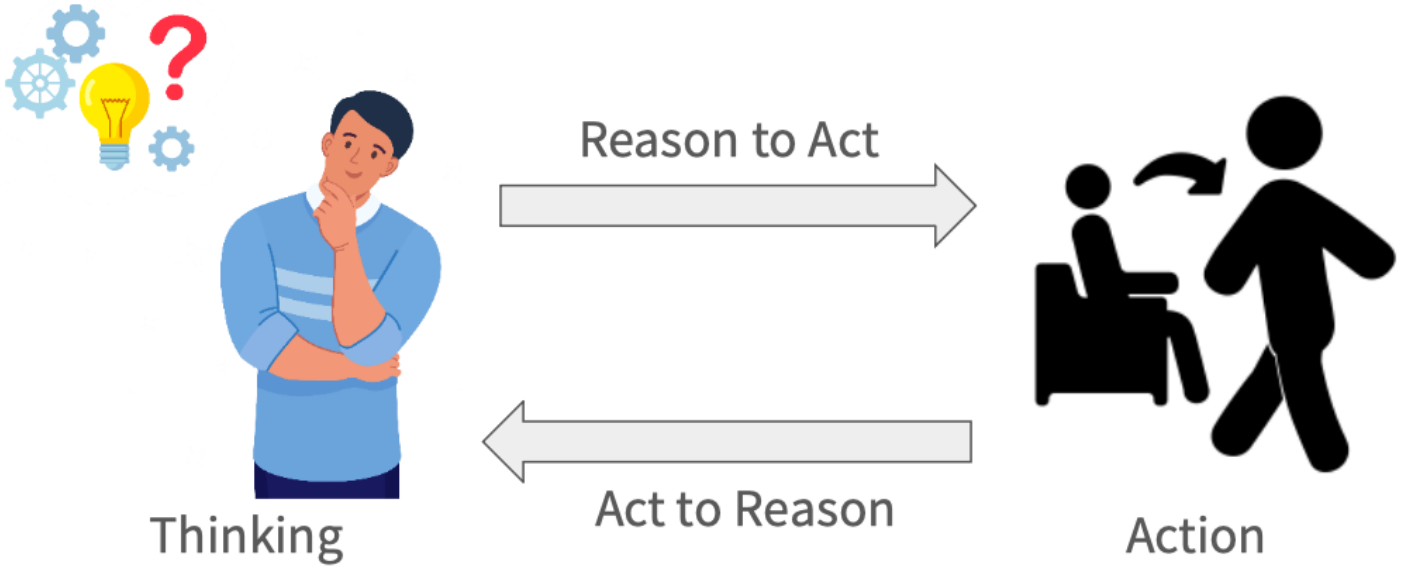
RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning

Constitutional AI: Harmlessness from AI Feedback

Motivation

Thinking drives action, guiding decisions.

Action provides feedback, **refining thought**.



Motivation

Challenge

- LLMs often struggle to combine **reasoning** (thinking) and **acting** (doing), treating them as isolated processes.
- Hallucinations and lack of contextual updates lead to errors in interactive decision-making.
- Lack of studies on how reasoning and acting can be combined for general task solving

Solution:

- ReAct, a framework where reasoning and acting occur in synergy, improving model effectiveness and human interpretability.

Prior Work

- Chain of Thought (CoT) Reasoning
  - **Problem:** The model's reasoning is an internal state that is not grounded (a static black box) → frequent hallucinations

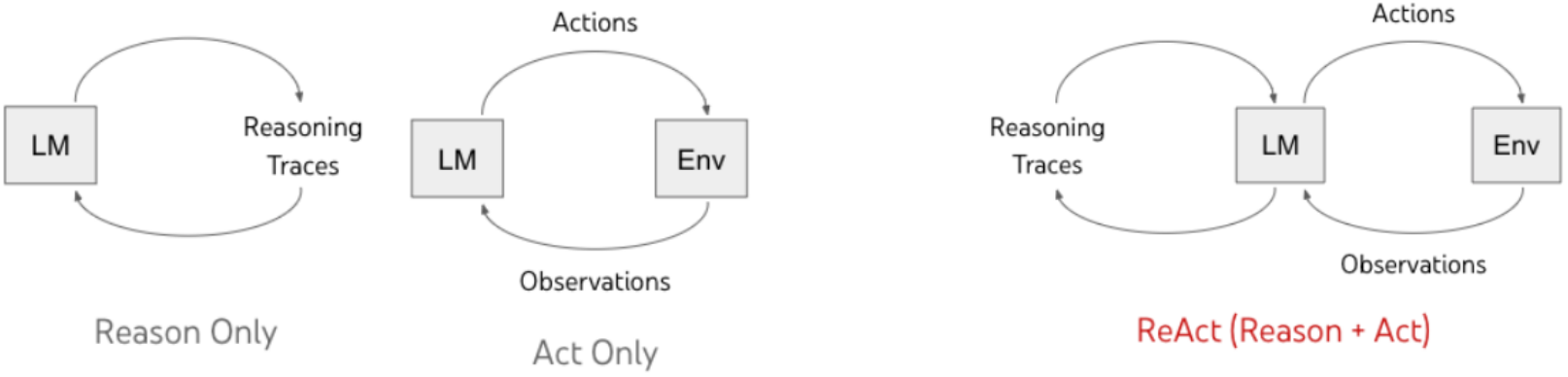
- LLMs for Decision Making
  - **Example:** WebGPT – LLM that interacts with web browsers
    - **Problem:** No explicit reasoning procedure
      - Reliance on expensive datasets and human feedback for RL

Contributions

- **ReAct:**
  - prompt LLMs to generate verbal reasoning traces and actions in an interleaved manner

- **Improves Performance:**
  - Outperforms SOTA in benchmarks like HotpotQA (QA), Fever (fact-checking), and WebShop (navigation).
- **Human Interpretability**
  - Provides detailed, grounded reasoning for each decision → increasing trustworthiness

Methodology



Methodology - Setup

Background:

- Agent observes $o_t \in \mathcal{O}$ and acts $a_t \in \mathcal{A}$ based on context c_t
  $$c_t = (o_1, a_1, \cdots, o_{t-1}, a_{t-1}, o_t)$$
- Learning a policy $\pi(a_t|c_t)$ is complex and expensive

Idea:

- Augment the space of language $\mathcal{L}$ to the space of actions $\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$
  - A **thought** is an action in the language space $\hat{a}_t \in \mathcal{L}$.
    - Thoughts **do not** affect the environment
    - They help an agent reason over current context
    - And results in $c_{t+1} = (c_t, \hat{a}_t)$ for future action + reasoning

Methodology - Implementation

Implementation:

- Prompt a frozen LLM (PaLM-540B) with few-shot in-context learning
  - Each example is a trajectory of actions, thoughts, and observations

Task Domain

- For reasoning tasks, **alternate** the generation of actions and thoughts
- For decision tasks, thoughts only need to appear **sparsely**
  - The LLM determines when to reason versus take action.

Reason-Only v.s. Act-Only v.s. ReAct

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

Reason-Only v.s. Act-Only v.s. ReAct

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes] ✗

Reason-Only v.s. Act-Only v.s. ReAct

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software)', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

Knowledge Intensive Reasoning Tasks

Domains

- **HotPotQA** - multi-hop question answering involving reasoning over Wikipedia passages
- **FEVER** - fact verification (supports, refutes, not enough info) based on Wikipedia passage's existence

Action Space

- **search[entity]** - returns first 5 sentences from *entity*'s wiki page or suggests 5 similar entities
- **lookup[string]** - simulates ctrl+F (*string*)
- **finish[answer]** - finish current task with *answer*

Purposefully weak action space to simulate human interaction with Wikipedia

Knowledge Intensive Reasoning Tasks

Baselines

- Ablate ReAct trajectories to build prompts for multiple baselines
  - **Standard** - removes all thoughts, actions, observations
  - **CoT** - removes actions and observations
  - **CoT-SC** - sample 21 CoT trajectories and adopt majority answer
  - **Act** - removes thoughts

Combination - combining internal and external knowledge

- **ReAct -> CoT-SC** - when ReAct fails to answer within given steps, back off to CoT-SC
- **CoT-SC -> ReAct** - when majority answer occurs less than half the time, back off to ReAct

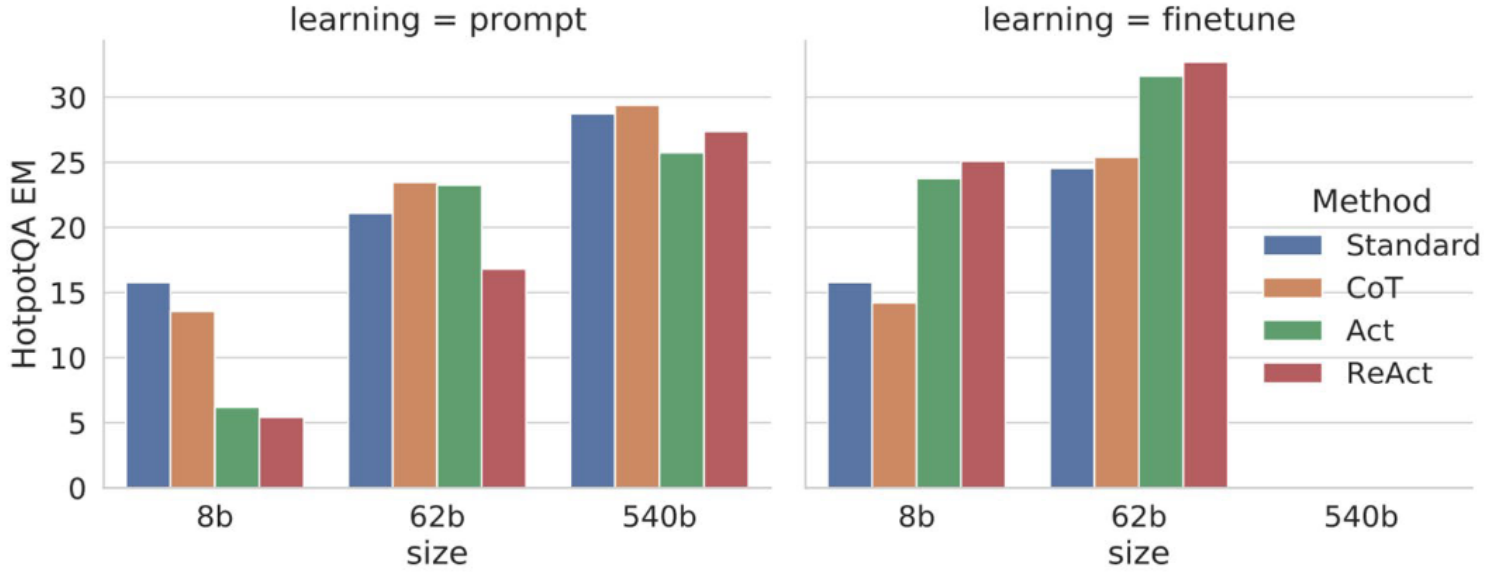
Knowledge Intensive Reasoning Tasks

Prompt Method[a]	**HotpotQA** (EM)	**Fever** (Acc)
`Standard`	28.7	57.1
`CoT` (Wei et al., 2022)	29.4	56.3
`CoT-SC` (Wang et al., 2022a)	33.4	60.4
`Act`	25.7	58.9
`ReAct`	27.4	60.9
`CoT-SC` → `ReAct`	34.2	**64.6**
`ReAct` → `CoT-SC`	**35.1**	62.0
Supervised SoTA[b]	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

Knowledge Intensive Reasoning Tasks

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%



Decision Making Tasks

WebShop - online shopping website environment

- Requires agent to purchase a product based on user instructions
- *Ex. "I am looking for a nightstand with drawers. It should have a nickel finish, and priced lower than $140."*
- Evaluated by **average score** (percentage of desired attributes covered) and **success rate** (percentage of all-requirements satisfaction)
- **Act** - actions to search, choose product, choose options, buy
- **ReAct** - additionally reason to determine what to explore, when to buy, what product options are relevant to the instruction
- Use **IL** (imitation learning) and **IL + RL** as baselines

Decision Making Tasks

Method	Score	SR
`Act`	62.3	30.1
`ReAct`	**66.6**	**40.0**
IL	59.9	29.1
IL+RL	62.4	28.7
Human Expert	82.1	59.6

Table 4: Score and success rate (SR) on Webshop. IL/IL+RL taken from Yao et al. (2022).

Strengths & Weaknesses

Strengths

- **Intuitive & Easy to Design**
- **Flexible**: Works for diverse tasks with a distinct set of actions and thoughts
- **Performant**: Outperforms SOTA with strong generalization to new tasks
- **Human Controllable**: Interpretable + correctable by humans

Weaknesses

- Large complex tasks with large action space require more demonstration, which can exceed in-context learning
- Requires multiple reasoning-action steps, increasing inference time and cost compared to direct-action models

ReAct Approach

- Integrates reasoning and acting in large language models
- Demonstrates superior performance across question-answering, fact-checking, and decision-making tasks
- Generates interpretable decision traces
- Reduces hallucinations and improves task success rates

What We Will Cover

ReAct: Synergizing Reasoning and Acting in Language Models

RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning

Constitutional AI: Harmlessness from AI Feedback

Background and Motivation

Growing demand for autonomous code generation

- Software development automation
- Focus more on creation than routine engineer

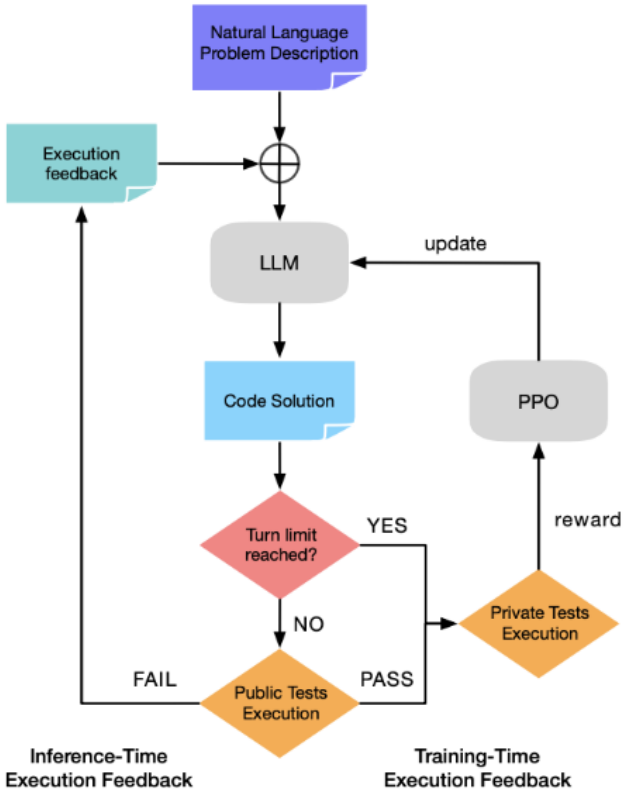
LLMs as coding agents need two key capabilities

- Accurately deduce user's intent
- Effective feedback from intermediate results

Contributions

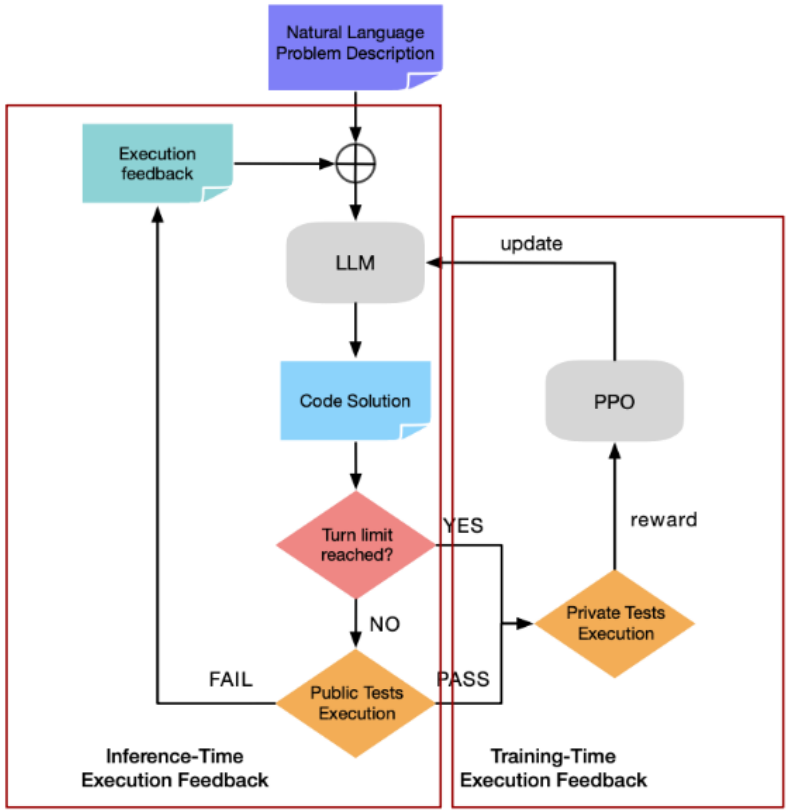
1. End-to-end RL **fine tuning** Framework
   - Actions = code; observations = execution feedback
   - Binary reward on whether or pass held-out test
2. Iterative refinement at fine tuning time
   - Iteratively repair the code with feedback before used for training
   - High quality data for PPO
3. Techniques: Two-tier Test Strategy + Hybrid Token/Turn-level Policy

Methodology: Framework



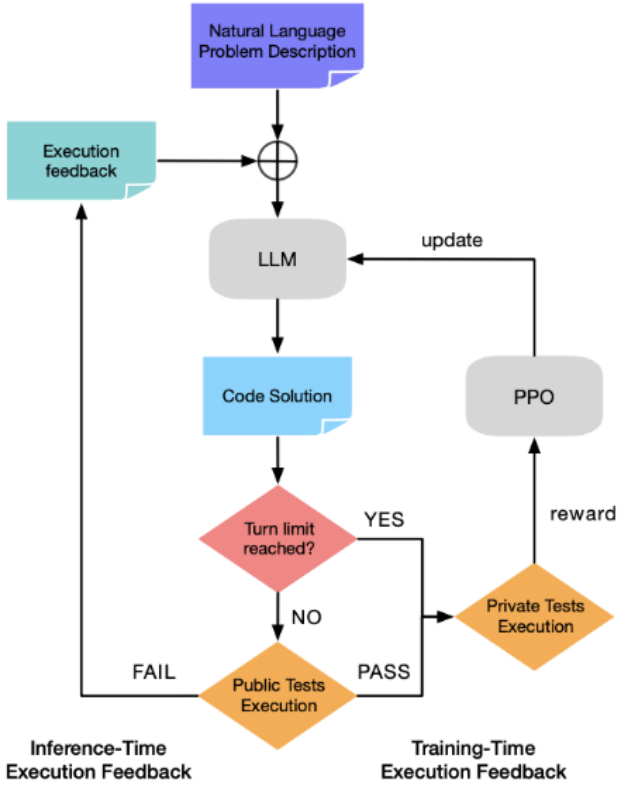
Methodology: Framework

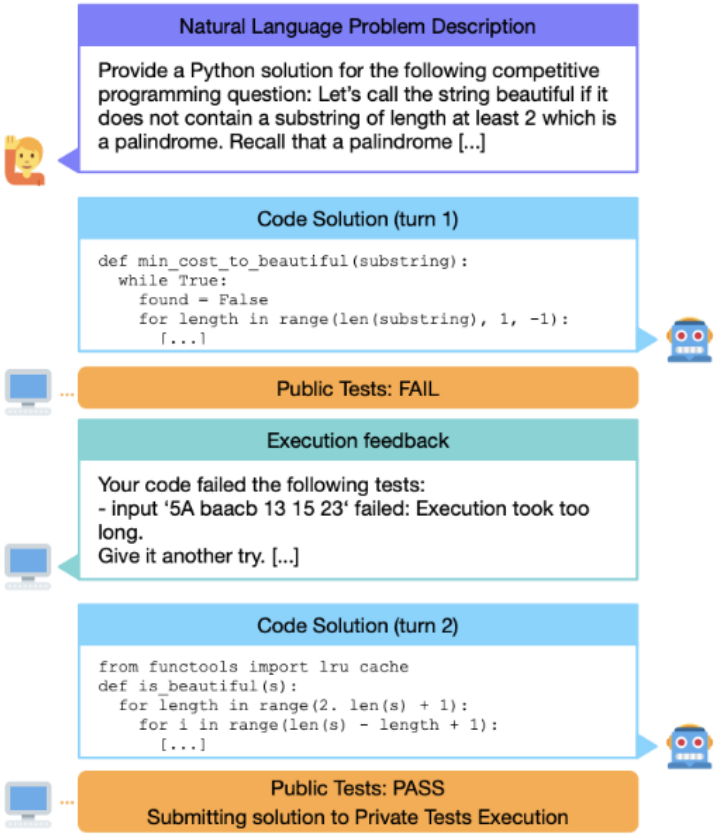
Exploit policy



Update policy

Methodology: Example

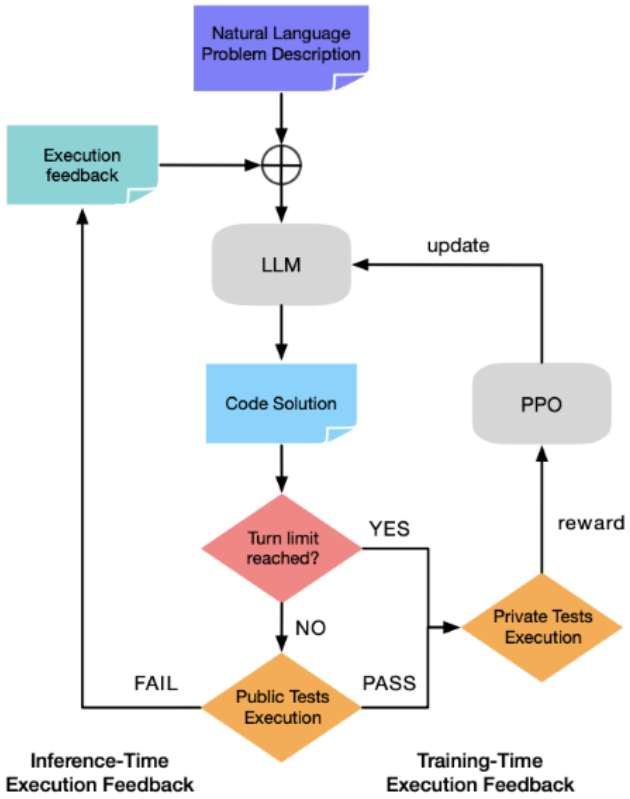




Technique 1: Two-tier Test Strategy

Public Tests

- Used during iteration process
- Provides execution feedback
- Limited set for faster iteration
- Guides solution selection



Avoid info leakage!!

Private Tests

- Hidden during generation
- Determines correctness
- Full test suite coverage
- Used for PPO training

Technique 2: PPO + Hybrid Token/Turn-level Policy

$$c_t = o_0, a_0, o_1, a_1, \ldots, o_t$$

$$R(s_t, a_t) = r(s_t, a_t) - \beta \log \frac{\pi(a_t|c_t)}{\rho(a_t|c_t)}, \quad r(s_t, a_t) = \begin{cases} 1, & \text{if end of episode and all tests pass} \\ -1, & \text{if end of episode and any test fails} \\ -0.2, & \text{if } a_t \text{ does not contain valid code} \end{cases}$$

KL divergence from original distribution

$$r_t(\theta) = \frac{\pi_\theta(a_t|c_t)}{\pi_{\theta_{\text{old}}}(a_t|c_t)} \text{stop_grad}\left(\min\left(\frac{\pi_\theta(a_t|c_t)}{\pi_{\text{b}}(a_t|c_t)}, 1\right)\right)$$

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right] \qquad A_t = R(s_t, a_t) - V(c_t)$$

Policy (Token Level)

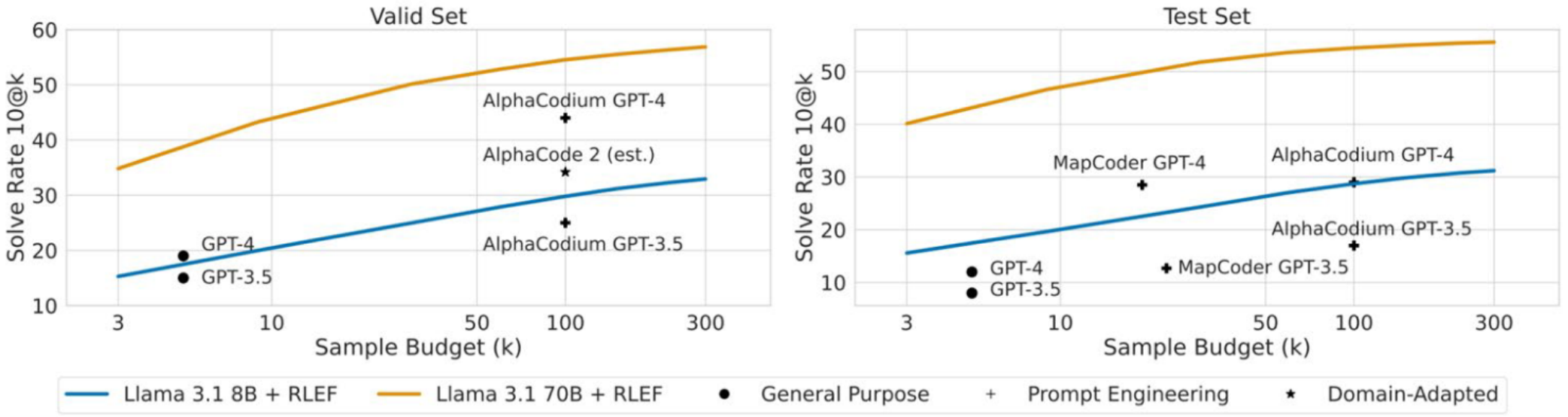
- Generates code token by token
- Finer control over generation process

Value Function (Turn Level)

- Evaluates complete responses
- Uses last token of prompt
- Single advantage value for all tokens in response

Augmenting Code LLMs with Execution Feedback

Solve rates of Llama 3.1 Models after RLEF training on CodeContests scale linearly with execution feedback



Inference-time Behavior

Model	CC. Test		HumanEval+		MBPP+	
	ST	MT	ST	MT	ST	MT
Llama 3.1 8B Instruct	11.8	10.5	65.3	63.9	58.3	60.5
+ RLEF	9.7	16.0	67.5	69.5	57.0	63.1
Llama 3.1 70B Instruct	26.2	27.4	73.2	75.0	66.9	70.2
+ RLEF	30.3	40.1	78.6	80.4	67.6	72.2
gpt-4o-2024-05-13	25.3	24.3	82.8	80.7	68.8	71.7

1@3 solve rates in single-turn (ST) and multi-turn (MT) setups for base and RLEF models.

- Base models rarely benefit from access to faulty solutions and execution feedback
- Training on CodeContest also generalize to other code generation benchmarks

Inference-time Ablation w/ Execution Feedback

Behavior analysis of initial and RLEF-trained models on public test, for 8B (top) and 70B (bottom) models.

- RLEF produces fewer "wrong outputs" but more "timeout error" in their initial turn
- In subsequent turns RLEF significantly improves self-repair while RLEF with random feedback fixes less errors

Strengths & Weaknesses

Strengths:

- An end-to-end RL fine tuning framework that effectively incorporates execution feedback to iteratively refine code solutions
- Significantly reduces the sample budget needed for state-of-the-art performance in competitive programming tasks
- Generalizes well across multiple benchmarks and various model sizes

Weaknesses:

- Specifically optimized for code synthesis tasks and competitive programming
- Heavily relies on the availability and quality of execution feedback
- Single-turn performance is hurt by RLEF training. Also performance improvements are observed with up to 5 turns, but benefits plateau beyond this limit.

What We Will Cover

ReAct: Synergizing Reasoning and Acting in Language Models

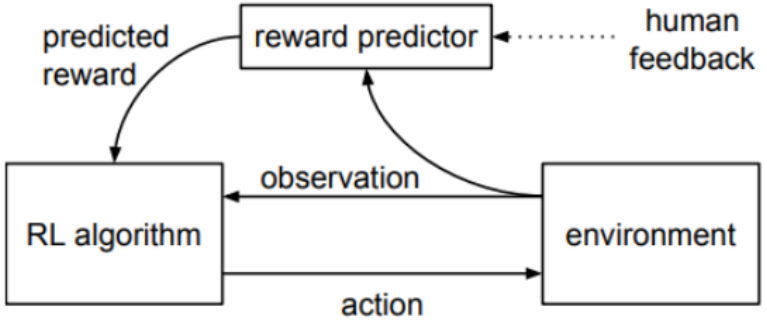
RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning

Constitutional AI: Harmlessness from AI Feedback

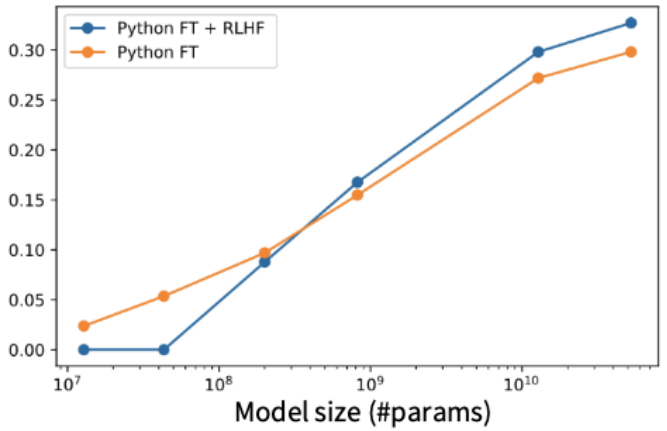
Scaling Finetuning with Human Preferences

- **We can approximate human preferences with a reward model and use Reinforcement Learning (RL) to finetune LLMs**

- **Humans are asked to rank the outputs of the model based on various criteria, for example:**
  - Correctness
  - Usefulness
  - Specificity



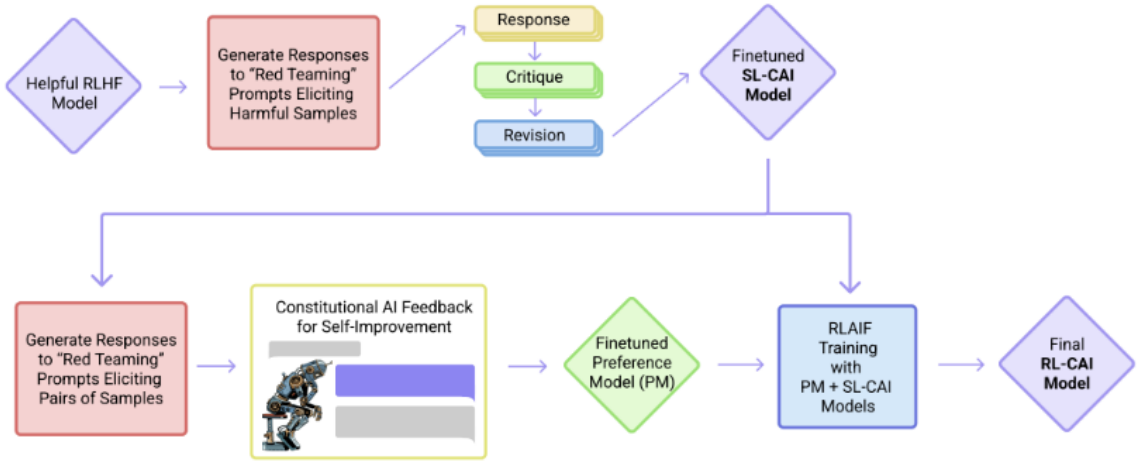
Performance on filling python function body given the context[2]



Christiano et al., Deep Reinforcement Learning from Human Preferences, 2017
Stiennon et al., Learning to summarize from human feedback, 2020
Bai et al., 2022, Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback

We Can Scale Finetuning Even More with AI Feedback Constitutional AI / RL from AI Feedback (RLAIF)!

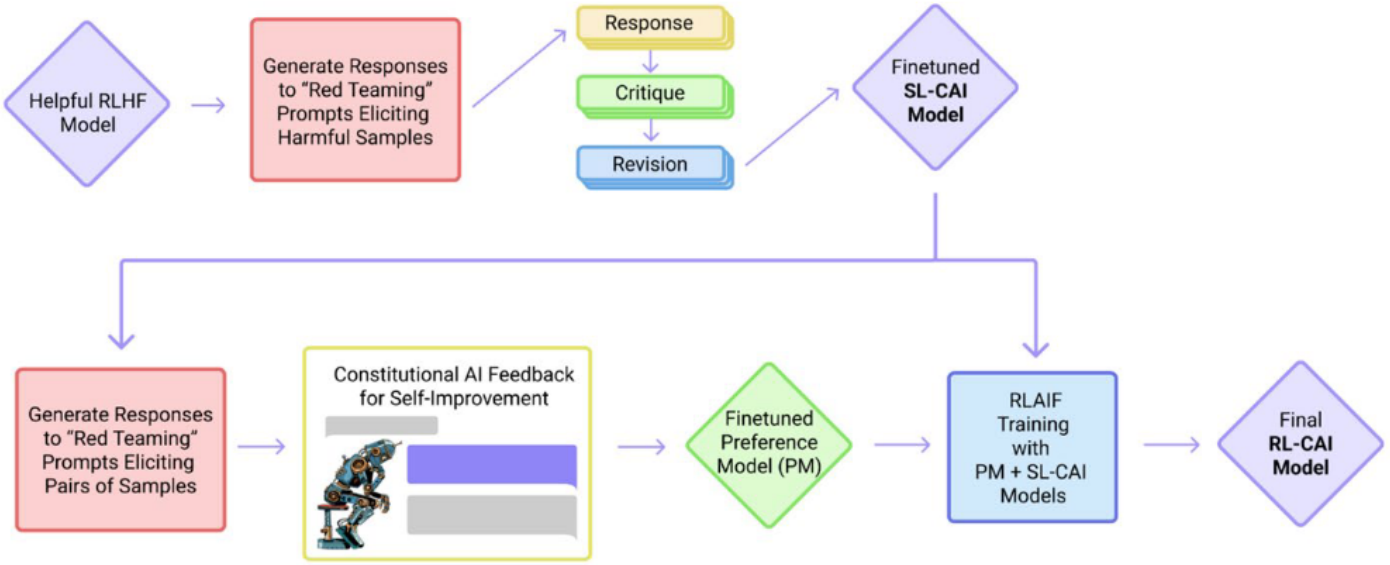
- Replaces ~10,000s of human labels with ~10 human-written principles (the “constitution”)!
- Perform RL-finetuning using AI feedback derived from a “constitution” describing desired behavior. Humans don’t need to be in the loop, except to write the constitution!



Bai, ..., Mirhoseini et al. Constitutional AI: Harmlessness from AI Feedback. arXiv 2022.

Constitutional AI

Use a set of 16 principles (~constitution) which governs AI behavior



An Example Constitution with 3 Principles

#1 CritiqueRequest: Identify specific ways in which the assistant's last response is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.
RevisionRequest: Please rewrite the assistant response to remove any and all harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.

#2 CritiqueRequest: Take a look at the assistant's last response and note if it was in any way misogynistic or otherwise gender biased. Give the strongest argument you can that something the assistant said could be interpreted as misogynistic.
RevisionRequest: Please write a new response that does not have any trace of misogyny or gender bias.

#3 CritiqueRequest: Did the assistant's preceding response include anything that may be inappropriate for young children? If so, discuss how it can be revised to be more appropriate.
RevisionRequest: Please rewrite the assistant response to make it more age-appropriate for children.

Bai et al., 2022, Constitutional AI: Harmlessness from AI Feedback

Constitutional AI: Self-Improving AI Using AI Feedback

In "Constitutional AI", the LLM follows a Constitution (set of principles written by a human) to generate feedback for self-improvement

1. **Supervised learning (SL):** Finetune LLM on data generated by self-critique and revisions

   Example:

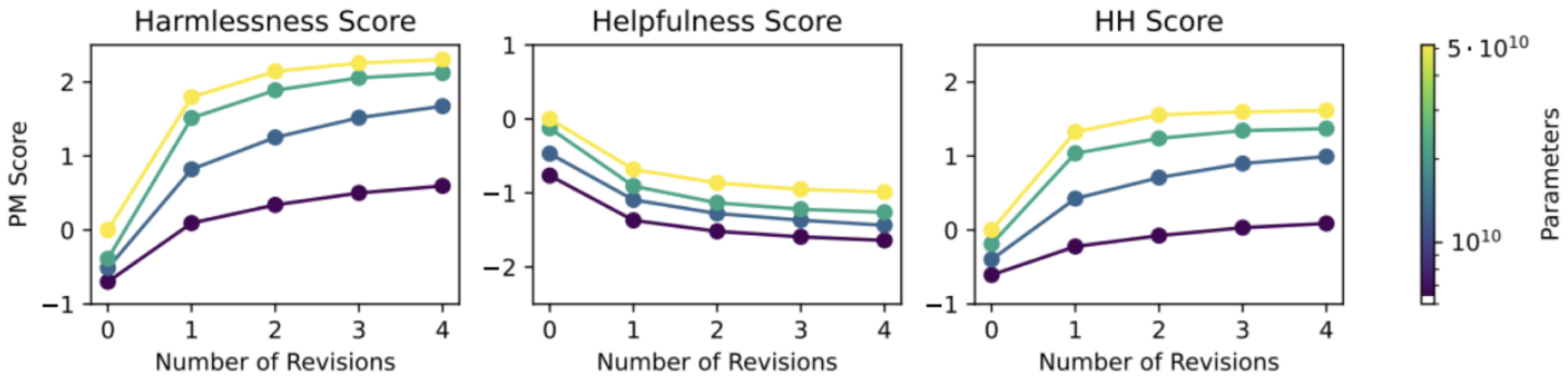
   Critique Request: *Identify specific ways in which the assistant's last response is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.*

   Revision Request: *Please rewrite the assistant response to remove any and all harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.*

Bai et al., 2022, Constitutional AI: Harmlessness from AI Feedback

Constitutional AI Supervised Learning Results

Harmlessness improves and helpfulness declines as a function of the number of revisions. Overall HH (Helpfulness + Harmlessness) improves monotonically.



Bai et al., 2022, Constitutional AI: Harmlessness from AI Feedback

Constitutional AI: Self-Improving AI Using AI Feedback

In "Constitutional AI", the LLM follows a Constitution (set of principles written by a human) to generate feedback for self-improvement

1. **Supervised learning:** Finetune LLM on data generated by self-critique and revisions

1. **Reinforcement learning from AI feedback (RLAIF):**
   a. Train a preference model based on LLM (from Step 1) responses and the constitution
   b. Finetune the LLM to maximize the preference model

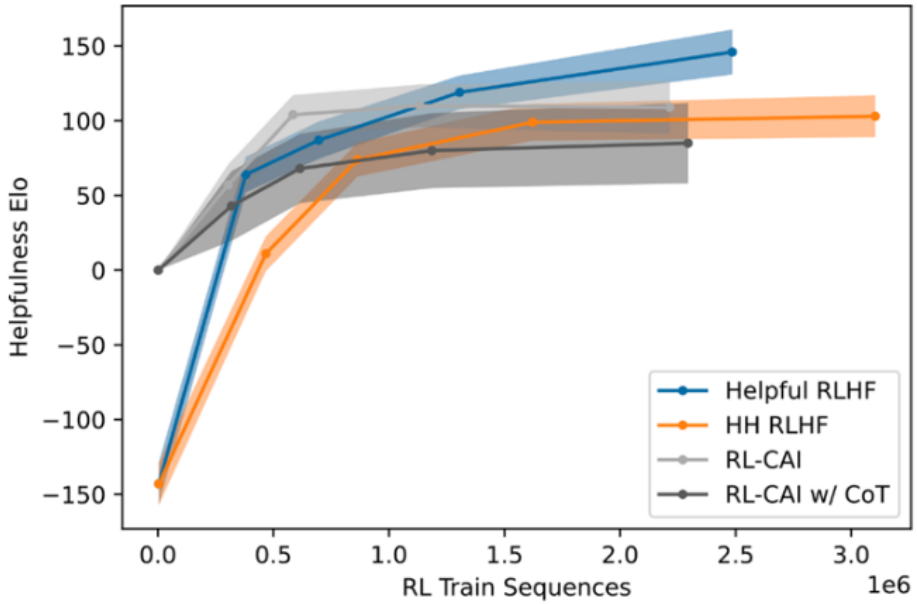
Example: *Choose the response that answers the human in the most thoughtful, respectful and cordial manner.*

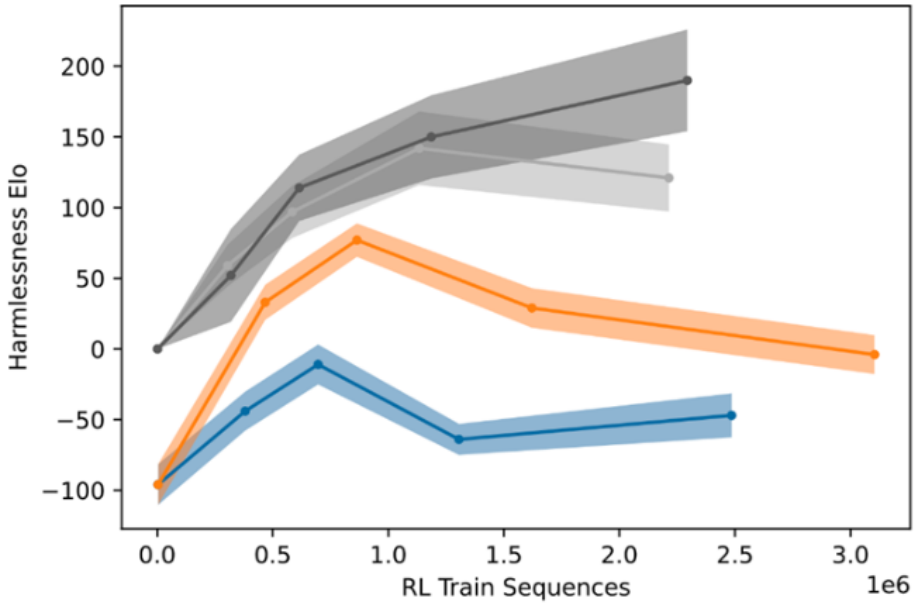
Bai et al., 2022, Constitutional AI: Harmlessness from AI Feedback

Constitutional AI Improves Scaling

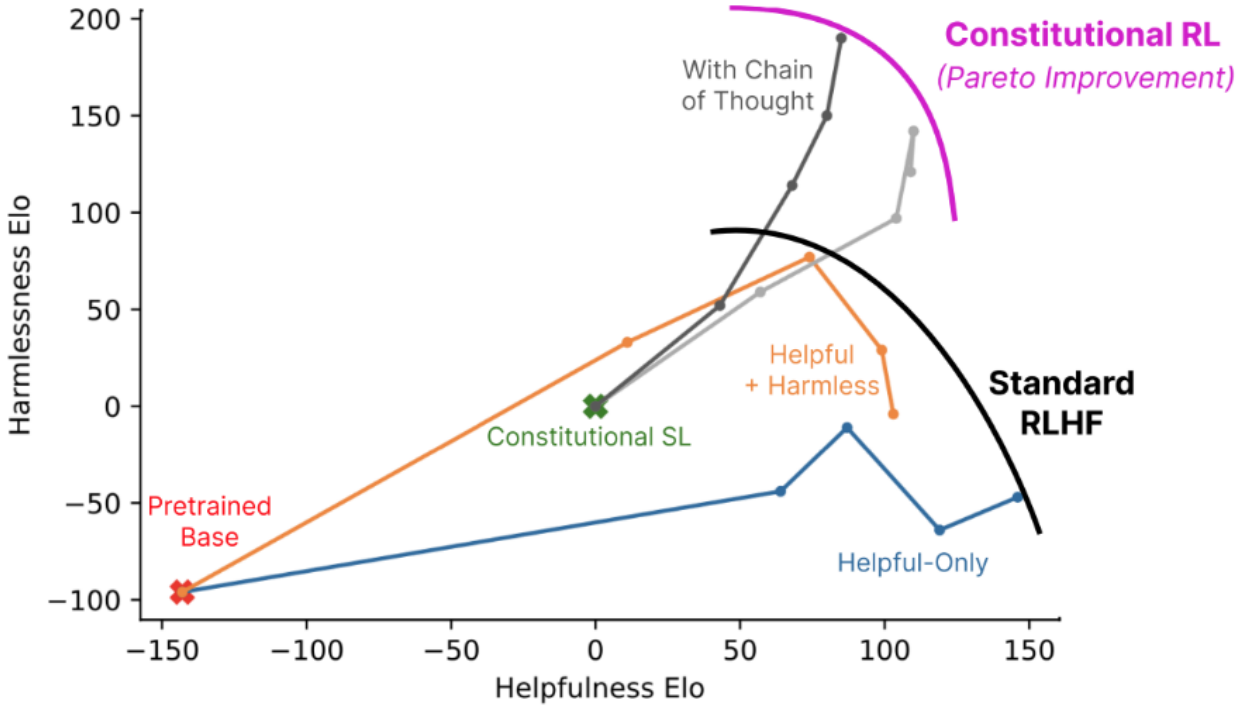
Similar to RLHF, except that we replace human preferences with AI feedback

The AI model itself evaluates responses according to a set of constitutional principles





Constitutional AI and RLAIF Further Improve Finetuning



Bai et al., 2022, Constitutional AI: Harmlessness from AI Feedback

Follow on work

2023

RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback [link]

Self-Refine: Iterative Refinement with Self-Feedback [link]

2024

Training Language Models to Self-Correct via Reinforcement Learning [link]

Recaps and Takeaways

ReAct: Synergizing Reasoning and Acting in Language Models

- Integrates reasoning and acting in large language models
- Demonstrates superior performance across QA, fact-checking, and decision-making tasks
- Generates interpretable decision traces

RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning

- incorporates execution feedback to iteratively refine code solutions
- Significantly reduces the sample budget needed for state-of-the-art performance in competitive programming tasks

Constitutional AI: Harmlessness from AI Feedback

- LLM follows a Constitution (set of principles written by a human) to generate feedback for self-improvement

Train Time Scaling/Scaling RL

What We Will Cover

STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al. 2022)

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

AIME Benchmark

AIME 2024

- GPT-3.5: ~5% accuracy
- DeepSeekMath-RL 7B: 51.7% accuracy
- DAPO on Qwen-32B: 50% accuracy

The question: How did smaller models catch up to larger ones?

Train-time scaling

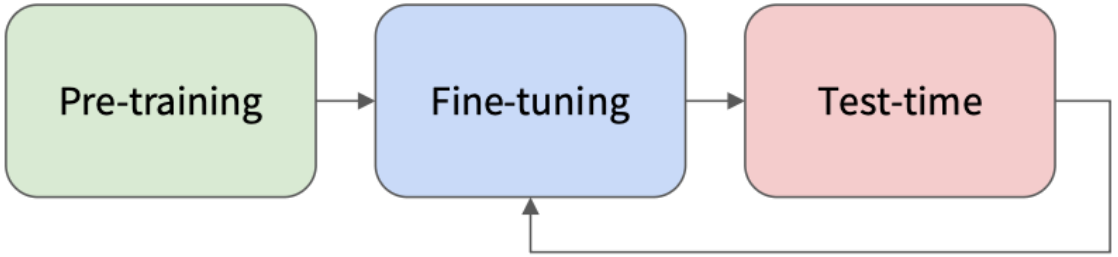
- Most attempts to do RL on reasoning fail catastrophically
- The devil is in implementation details
- Success requires understanding *why* these methods work

Three Insights Drive This Field

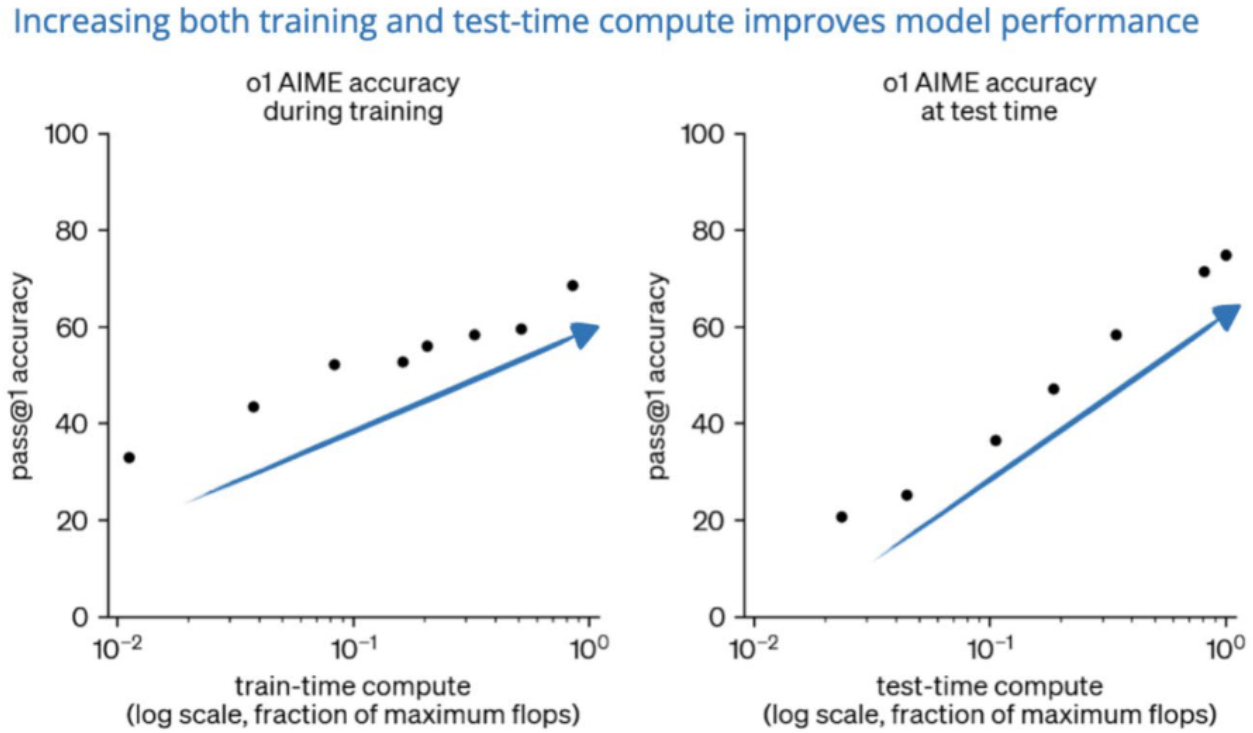
1. **Models can learn from their own outputs** (if you filter cleverly)
2. **Training compute can substitute for model parameters**
3. **Small fixes matter enormously at scale**

Connection to DeepSeek-R1 or Frontier Thinking/Reasoning:

From Test Time Scaling to Train-time Scaling



Scaling up Train-time compute in MATH domains has significant gains…



Competitive Programming with Large Reasoning Models, https://arxiv.org/abs/2502.06807

Reasoning Models: o series, Gemini Flash Thinking…

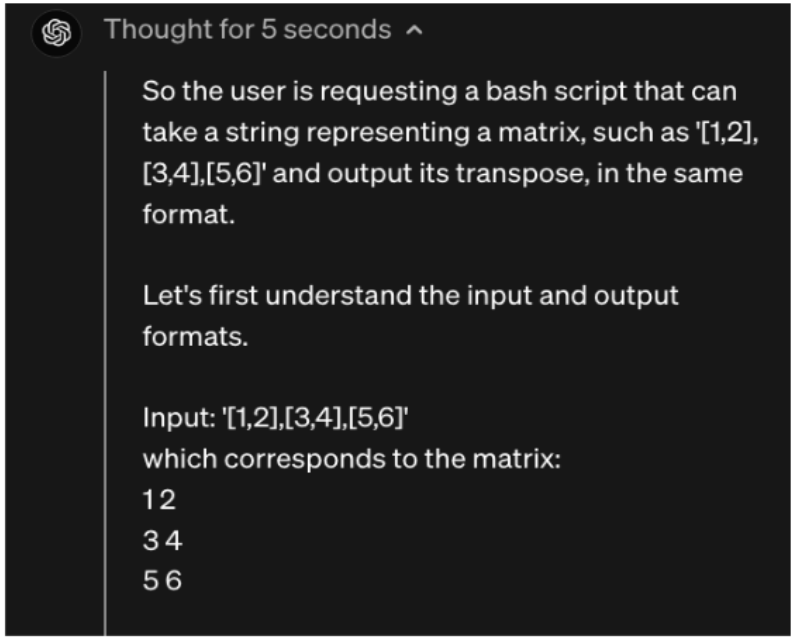
Motivation: For difficult problems, humans may think for a long time before responding. Thinking models use a chain of thought when attempting to solve a problem.

- Problem Analysis
- Task Decomposition: Learning to break down tricky steps into simpler ones
- Self-Evaluation: Through RL, thinking models learn to finetune their chain of thought and refine the strategies it uses
- Self-Correction: Learns to recognize and correct its mistakes
- Alternative Proposal: Learns to try a different approach when the current one isn't working

Reasoning Models: o1…

- Problem Analysis: So the user is requesting a bash script that. . . Let's first understand the input and output formats. . . So the requested output is '[1,3,5],[2,4,6]'

Write a bash script that takes a matrix represented as a string with format '[1,2],[3,4],[5,6]' and prints the transpose in the same format.



https://openai.com/index/learning-to-reason-with-llms/

Reasoning Models: o1…

- Task Decomposition:

Write a bash script that takes a matrix represented as a string with format '[1,2],[3,4],[5,6]' and prints the transpose in the same format.

Approach:

- Parse the input string to extract the matrix elements.
- Build the matrix as an array of arrays.
- Transpose the matrix.
- Output the transposed matrix in the same format.

https://openai.com/index/learning-to-reason-with-llms/

Reasoning Models: o1…

- Self Correction: Wait, the correct formula is:…

What is the pH of a 0.10 M solution of $\mathrm{NH_4F}$? The K_a of $\mathrm{NH_4^+}$ is 5.6×10^{-10} and the K_a of HF is 6.8×10^{-4}.

So let's compute:

pH = 7 + 0.5 \times \log\left(\frac{Kb - Ka}\right)

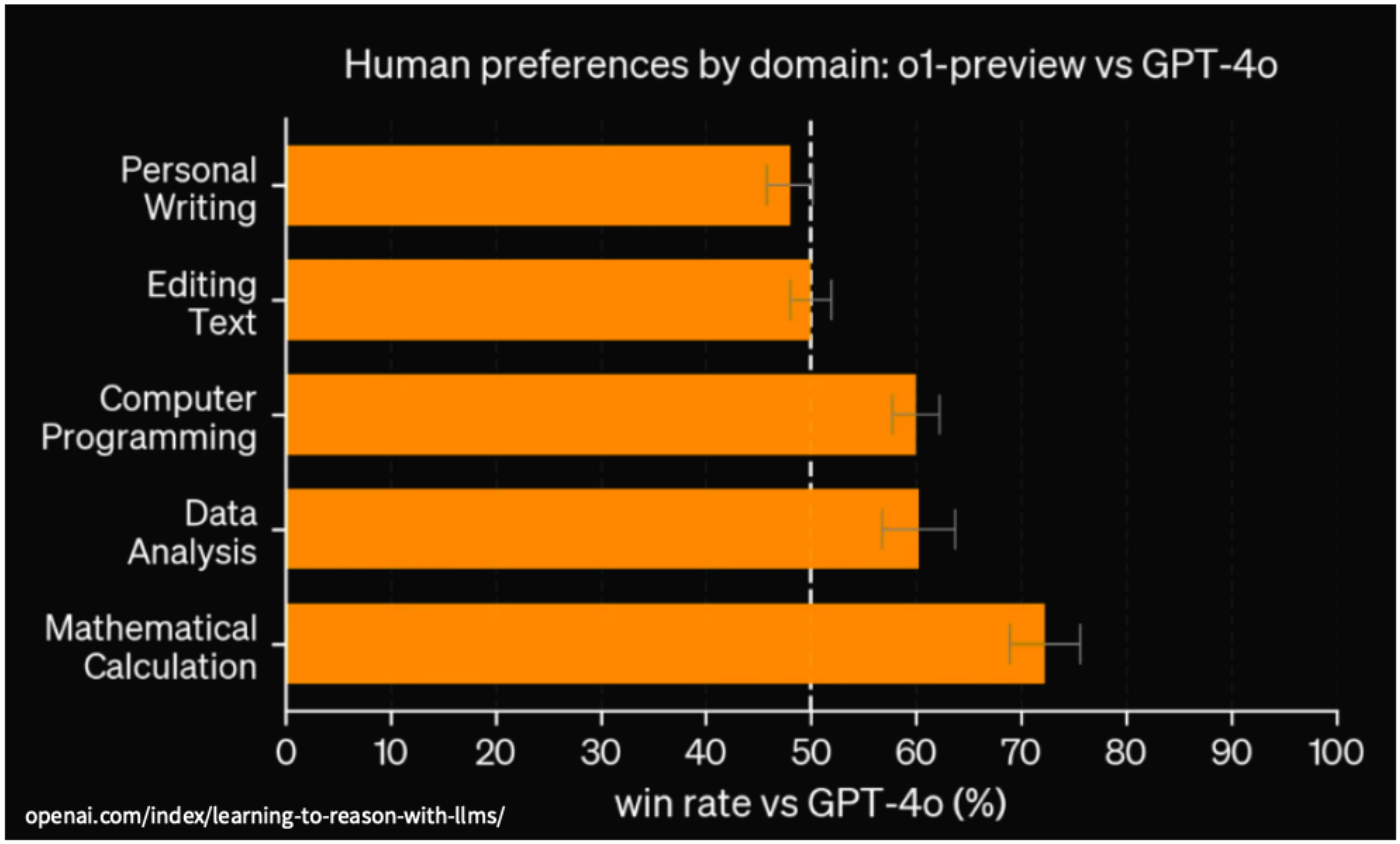
Wait, the correct formula is:

$$pH = 7 + 0.5 \times \log\left(\frac{Kb \text{ for base}}{Ka \text{ for acid}}\right)$$

So first compute Kb for F^- and Ka for $NH4^+$.

https://openai.com/index/learning-to-reason-with-llms/

Reasoning Models: o1…



What We Will Cover

STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al. 2022)

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

What We Will Cover

STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al. 2022)

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

Introduction

Motivation

Chain-of-thought in LMs provides interpretability, performance gains (Wei et al, 2022)

Objective

Facilitate strong step-by-step reasoning capabilities

Challenges with Existing Approaches

- Manual annotation of reasoning steps is expensive, doesn't scale (Rajani et al., 2019)
- Template-based automatic rationale generation only works for problems with known solution patterns (Nye et al., 2021)
- Few-shot prompting with limited example rationales underperforms models directly fine-tuned on larger datasets without rationales (Wei et al, 2022)

STaR: The Bootstrapping Insight

Start: 10 examples with reasoning steps

├—> Generate solutions to 10K problems

├—> Keep only correct answers

├—> Fine-tune on these

└—> Repeat

The Key Innovation: Rationalization

Problem: Model gets stuck—can't solve new problems, no training signal

Solution: Give the model the answer, ask it to explain backward

- Human: "The answer is 42. Now show your work."
- Model generates reasoning that leads to 42
- Fine-tune *without* the hint (as if it solved it alone)

Why this works: Expands the training set to include hard problems

Contributions

Autonomous bootstrapping to develop reasoning abilities from limited examples via iteratively:

1. Generating reasoning attempts
2. Learning from successful reasoning paths
3. Rationalizing failed attempts *a posteriori*
4. Using LM's generated rationales as training data

Assumptions

Autonomous bootstrapping mechanism to iteratively develop reasoning abilities from limited examples via:

1. Generating reasoning attempts
2. Learning from successful reasoning paths
3. Rationalizing failed attempts *a posteriori*
4. Using LM's generated rationales as training data

Correctness is proxy for *reasoning quality*

- Filtering incorrect answers may discard insightful reasoning paths, but it maintains higher-quality training data

Assumes LM can generate valid backwards reasoning when given answer as hint

Assumes initial LM is strong enough to enable bootstrapping from few-shot examples

Problem Setup

Prompt Dataset

(question, rationale, answer)

$(\tilde{x}_1, \tilde{r}_1, \tilde{y}_1), \ldots, (\tilde{x}_K, \tilde{r}_K, \tilde{y}_K)$

Small: $K \ll N$

Question $\tilde{x}_i$: `Where do you put your grapes just before checking out?`
`(a) mouth`
`(b) grocery cart`
`(c) super market`
`(d) fruit basket`

Rationale $\tilde{r}_i$: `The answer should be the place where grocery items are placed before checking out. Of the above choices, grocery cart makes the most sense for holding grocery items. Therefore, the answer is grocery cart (b).`

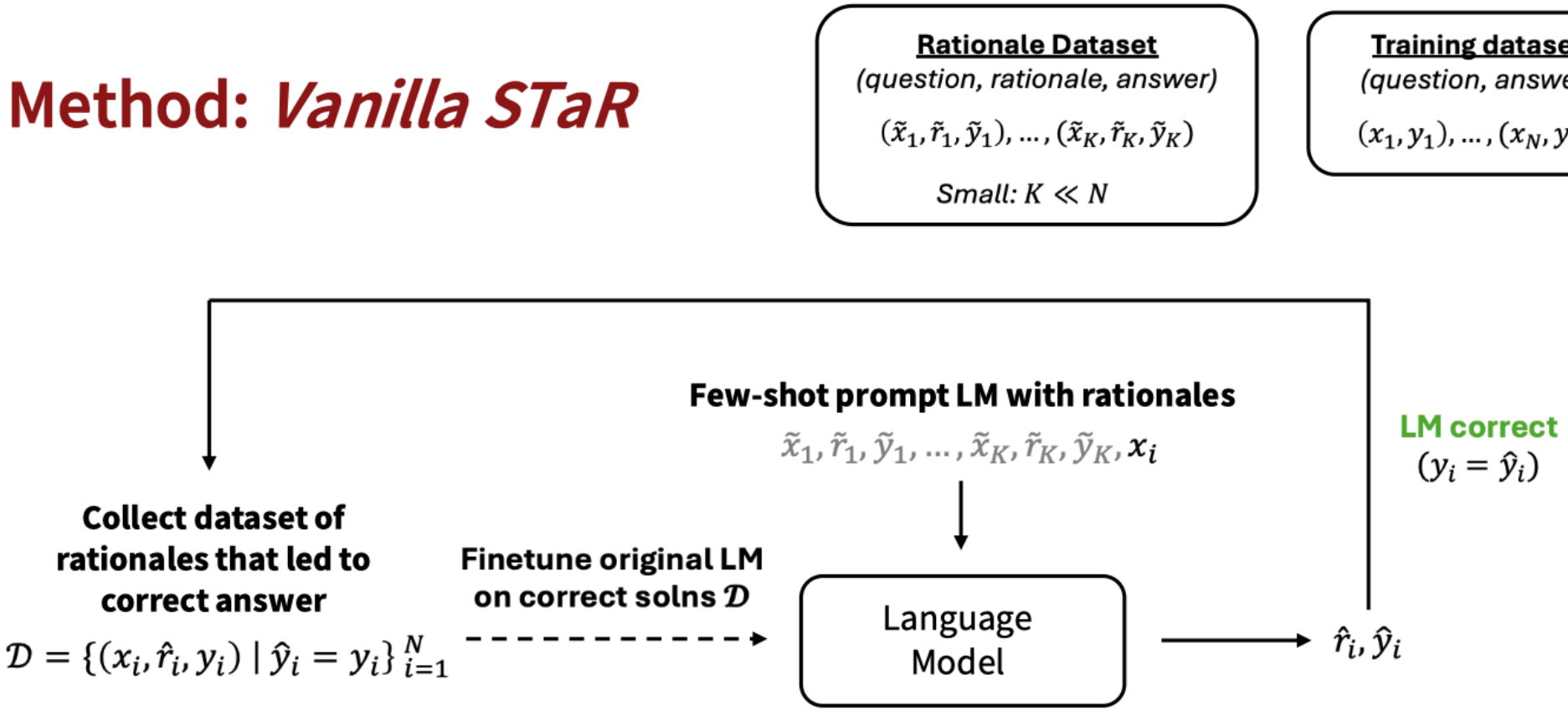
Answer $\tilde{y}_i$: `(b) grocery cart`

Training dataset

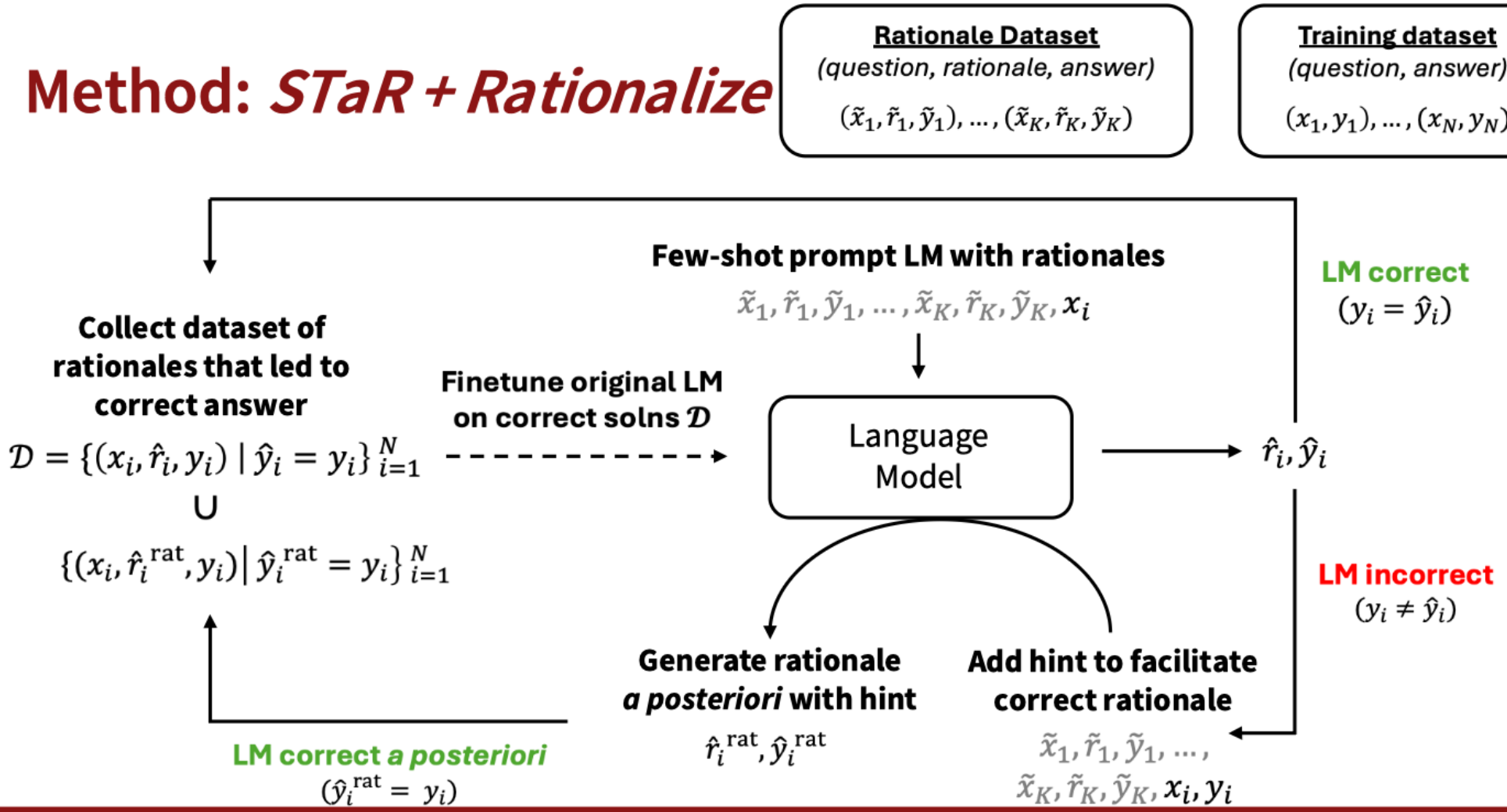
(question, answer)

$(x_1, y_1), \ldots, (x_N, y_N)$

Method: *Vanilla STaR*



Method: *STaR + Rationalize*



Experiments: Protocol

(6B-parameter)

100-step learning rate warm up, followed by a constant learning rate

Algorithm 1 STaR

Input M: a pretrained LLM; dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{D}$ (w/ few-shot prompts)

$M_0 \leftarrow M$ # Copy the original model
for n **in** $1 \ldots N$ **do** # Outer loop
  $(\hat{r}_i, \hat{y}_i) \leftarrow M_{n-1}(x_i) \quad \forall i \in [1, D]$ # Perform rationale generation
  $(\hat{r}_i^{\text{rat}}, \hat{y}_i^{\text{rat}}) \leftarrow M_{n-1}(\text{add_hint}(x_i, y_i)) \quad \forall i \in [1, D]$ # Perform rationalization
  $\mathcal{D}_n \leftarrow \{(x_i, \hat{r}_i, y_i) \mid i \in [1, D] \wedge \hat{y}_i = y_i\}$ # Filter rationales using ground truth ans
  $\mathcal{D}_n^{\text{rat}} \leftarrow \{(x_i, \hat{r}_i^{\text{rat}}, y_i) \mid i \in [1, D] \wedge \hat{y}_i \neq y_i \wedge \hat{y}_i^{\text{rat}} = y_i\}$ # Filter rationalized rationa
  $M_n \leftarrow \text{train}(M, \mathcal{D}_n \cup \mathcal{D}_n^{\text{rat}})$ # Finetune the original model on correct solutions - inner l
end for

40 steps at the 1st outer loop with 20% increase per outer loop

Obs. A slower start at training time benefits performance

Experiments: Datasets

GSM8K
~9k samples

10 few-shot prompts of answering with explanations

Problem: Beth bakes 4, 2 dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?
Solution: Beth bakes 4 2 dozen batches of cookies for a total of 4*2 = <<4*2=8>>8 dozen cookies
There are 12 cookies in a dozen and she makes 8 dozen cookies for a total of 12*8 = <<12*8=96>>96 cookies
She splits the 96 cookies equally amongst 16 people so they each eat 96/16 = <<96/16=6>>6 cookies
Final Answer: 6

prompt
model generates
rationalization (if applicable)

CommonsenseQA
~12k samples, human perf 89%

10 few-shot prompts of answering with explanations

```
Q: What do people use to absorb extra ink from a fountain pen?
Answer Choices:
(a) shirt pocket
(b) calligrapher's hand
(c) inkwell
(d) desk drawer
(e) blotter
A: The answer must be used to absorb extra ink. Blotters are designed to absorb
liquids. Therefore, the answer is blotter (e).
```

prompt
model generates
rationalization (if applicable)

Arithmetics
~50k samples

10 few-shot prompts for 1 to 5 digits

```
Input:
6 2 4 + 2 5 9
Target:
<scratch>
6 2 4 + 2 5 9 , C: 0
2 + 5 , 3  C: 1
6 + 2 , 8 3  C: 0
, 8 8 3  C: 0
0 8 8 3
</scratch>
8 8 3
```

prompt
model generates
rationalization (if applicable)

Results

Results

- MATH: 46.8% (instruction tuning) → 51.7% (with RL)
- Uses only 70-87% of data (correct solutions)

Limitation

- Resets to base model each iteration (wasteful)
- No true RL—just sophisticated filtering
- Plateaus after 16-36 iterations

Takeaway: Bootstrapping works, but we can do better with real RL.

Results: Language Reasoning on CQA

Table 1: We evaluate several baselines, including a few-shot GPT-J evaluation both with and without scratchpads, a GPT-J baseline finetuned to directly predict the answer, and STaR with and without rationalization applied to GPT-J. We use CoT to denote non-STaR models outputting rationales, and Direct to indicate those directly predicting the final answer. Note the final STaR model is trained on 78.2% of the training dataset with rationale generation, and an additional 8.5% from rationalization.

STaR is 30x less in size →

STaR trains on less data →

	CQA Dev Set Accuracy (%)	Train Data Used (%)
GPT-3 Direct Finetuned [29]	*73.0*	100
Few-shot Direct GPT-J	20.9	~0
Few-shot CoT GPT-J [3]	36.6	~0
Few-shot CoT LaMDA 137B [6]	55.6	~0
GPT-J Direct Finetuned	60.0	100
STaR without rationalization	68.8	69.7
STaR with rationalization	**72.5**	86.7

Obs. **Rational is hard to evaluate for natural language reasoning**, so the authors **qualitatively analyze** STaR (1) generates reasonable rationale and (2) sometimes improves upon few-shot examples (e.g. authors conduct a **human study** where the participants 30% more likely to rank the STaR-generated rationales higher than the few-shot rationales)

Results: Mathematical Reasoning on GSM8K

Obs. **Here, the use of rationalization does not substantially improve performance.** Can you think of why?

Table 2: We find that STaR substantially improves GSM8K performance over the baselines, despite training on only 25.0% of the data for the model without rationalization, and 28.7% of the dataset (with 0.5% from rationalization) for the model with rationalization.

	GSM8K Test Accuracy (%)	Train Data Used (%)
Few-shot Direct GPT-J	3.0	~0
Few-shot CoT GPT-J	3.1	~0
GPT-J Direct Finetuned	5.8	100
STaR without rationalization	10.1	25.0
STaR with rationalization	**10.7**	28.7

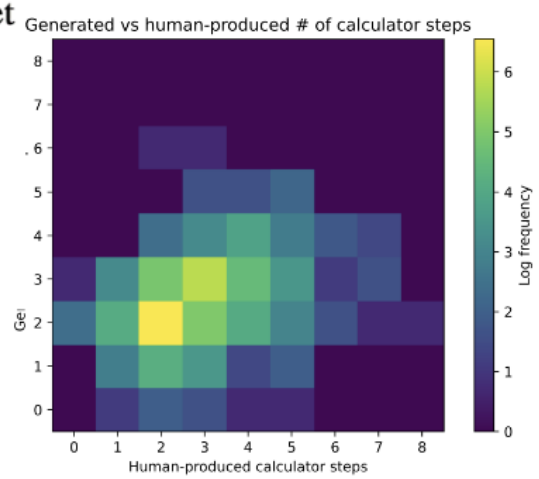


Figure 6: A comparison of the number of calculator steps generated by the model in order to solve examples in the training set relative to the number of steps used in the ground truth.

Obs. The **number of calculation steps** generated by the model matches the number of steps taken by humans. Is # of calculation steps a good indicator of performance for mathematical reasoning tasks?

Challenges

The Impact of Rationalization

✅ Rationalization conditions on answer, so with-rationalization distribution $p(r|x, y)$ might be a better search space for rationales than without-rationalization distribution $p(r|x)$

❓ Outputs without rationalization are examples where the model is most confident in answering, and they provide a weaker gradient signal than the rationalization examples.

❓ How to augment "hint" might be non-trivial for some tasks.

Few-shot Prompting dramatically reduces "drift" of later rationales from initial ones:
When should we stop including few-shot prompts over iterations of training?

✅ model is less constrained by initial rationales, potentially more generalizable

❌ style of rationale becomes dissimilar to the initial rationales

Strengths and Weaknesses

Strengths

1. Bootstrap mechanism for generating rationale dataset realizes **high sample efficiency while keeping human out of the loop**
2. Show **diversity in applicable problems types**: symbolic, natural language, and mathematical reasoning

Weaknesses

1. **Unclear evaluation metric for rationales** for some tasks
2. **True Negatives and False Positives**. Filtering based on correct answers may
   a. discard valid reasoning patterns
   b. acknowledge invalid reasoning patterns
3. Rationalization bootstrapping may **amplify biases**
   Q: Who's better at football: Russell Wilson or Jeremy Lin?
   A: Jeremy Lin
   ❌ biased rationale: Black people are better at football. Russell Wilson is black, whereas Jeremy Lin is asian. Therefore, the answer is Russell Wilson.

What has been done since

V-STaR: Adding a verifier to STaR

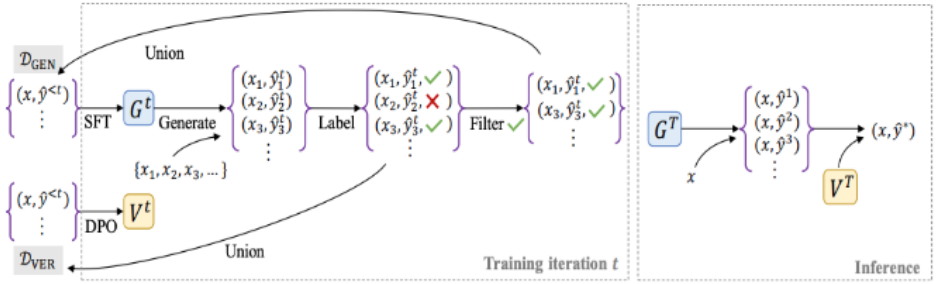


Figure 1: **Generator and verifier training in V-STaR. Left:** In each training iteration, the generator G^t is fine-tuned (from a pretrained LLM) on the current buffer of problem instances and correct solutions $\mathcal{D}_{\text{GEN}}$. Generated solutions that yielded a correct answer are added to $\mathcal{D}_{\text{GEN}}$ to be used in future iterations, and all the generated solutions (correct and incorrect) are added to $\mathcal{D}_{\text{VER}}$. The verifier V^t is trained using DPO with a preference dataset constructed from pairs of correct and incorrect solutions from $\mathcal{D}_{\text{VER}}$. **Right:** At test time, the verifier is used to rank solutions produced by the generator. Such iterative training and inference-time ranking yields large improvements over generator-only self-improvement.

Quiet-STaR: Additional internal "thinking"

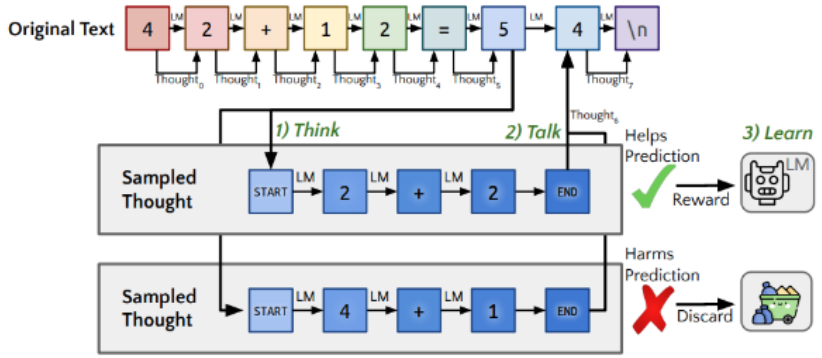


Figure 1: **Quiet-STaR**. We visualize the algorithm as applied during training to a single thought. We generate thoughts, in parallel, following all tokens in the text (think). The model produces a mixture of its next-token predictions with and without a thought (talk). We apply REINFORCE, as in STaR, to increase the likelihood of thoughts that help the model predict future text while discarding thoughts that make the future text less likely (learn).

What We Will Cover

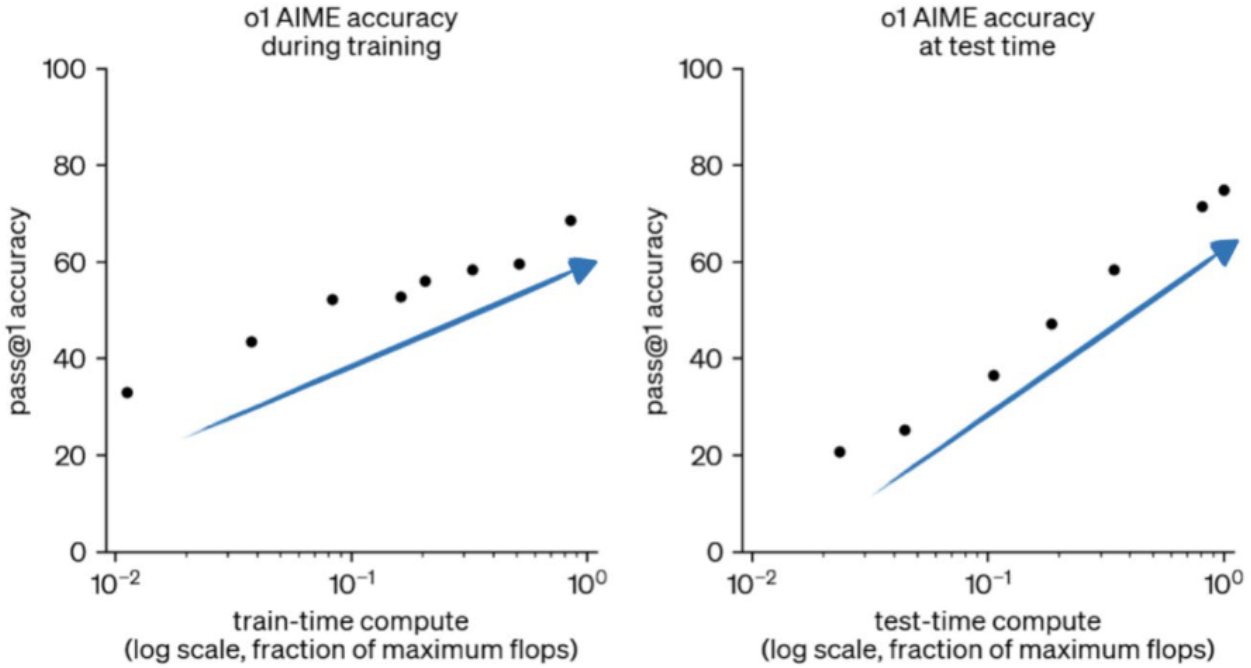
STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al. 2022)

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

Scaling up RL in MATH domains shows significant gains…

Increasing both training and test-time compute improves model performance



Competitive Programming with Large Reasoning Models, https://arxiv.org/abs/2502.06807

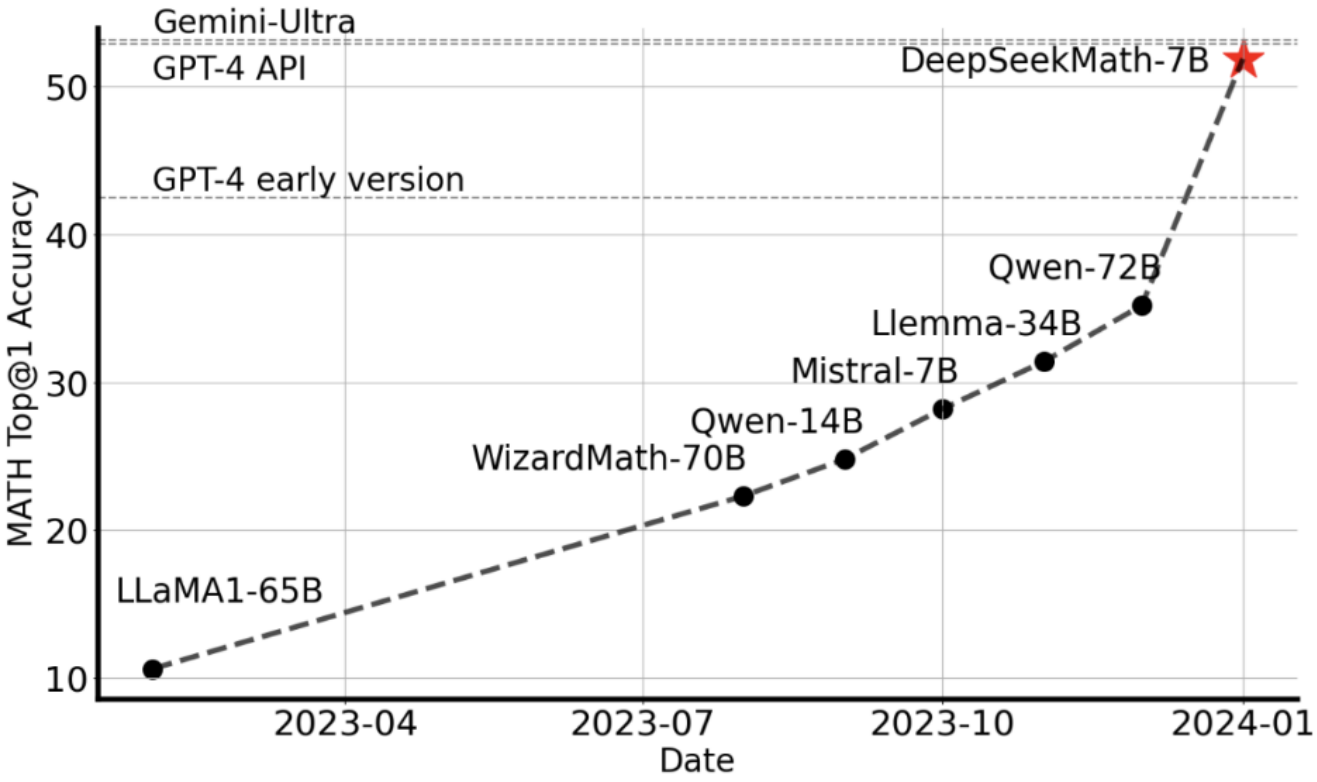
Train-time scaling

- Most attempts to do RL on reasoning fail catastrophically
- The devil is in implementation details
- Success requires understanding *why* these methods work

Three Insights Drive This Field

1. **Models can learn from their own outputs** (if you filter cleverly)
2. **Training compute can substitute for model parameters**
3. **Small fixes matter enormously at scale**

DeepSeekMath



DeepSeekMath: Data + Practical RL

Data Innovation

The Conventional Wisdom: Train on arXiv papers

DeepSeekMath's Finding: Common Crawl web pages > arXiv papers

How They Did It:

1. Start with OpenWebMath (13.6B tokens)
2. Train fastText classifier: "Does this look like math?"
3. Mine Common Crawl → find 120B tokens (9× bigger)
4. Iterate: Find new math domains, retrain classifier

DeepSeekMath: The Data Innovation

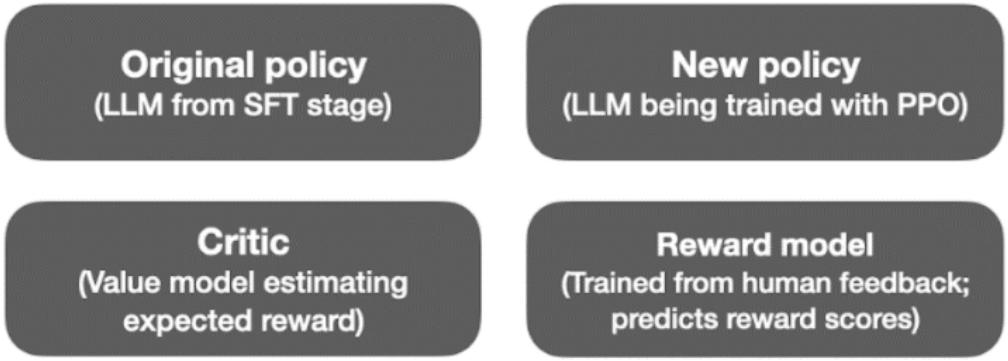
Surprising Results:

- ArXiv training: Sometimes *hurts* performance
- Code→Math sequence: Significantly helps both tool-use AND reasoning
- Multilingual data: Improves Chinese benchmarks without harming English

Key Lesson: Data curation matters more than data source prestige

What makes scaling RL challenging?

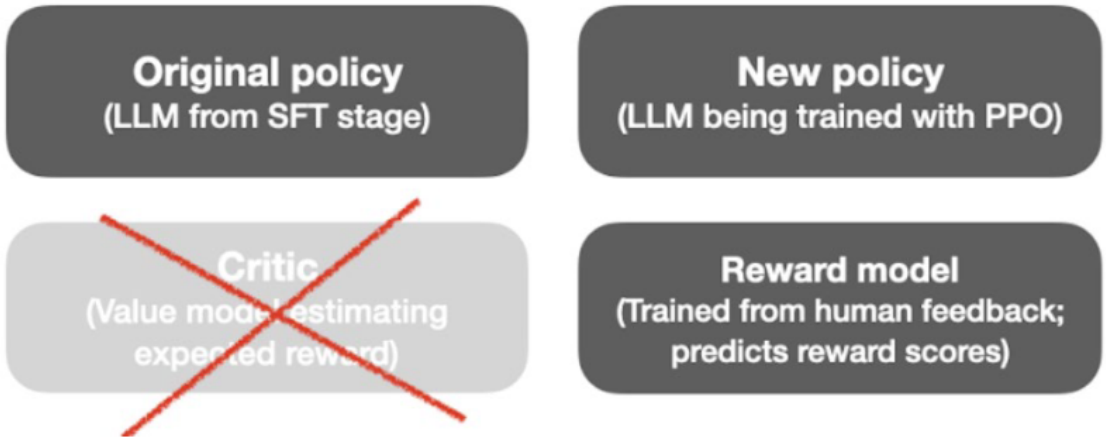
ML system for RLHF with PPO is challenging in memory footprint.



Source: https://sebastianraschka.com/blog/2025

What makes scaling RL challenging?

ML system for RLHF with GRPO is still challenging in memory footprint.



Source: https://sebastianraschka.com/blog/2025

GRPO - RL Without the Critic

The PPO Problem:

- Needs actor model (7B parameters)
- Needs critic model (7B parameters)
- 2× memory, 2× training cost

GRPO's Insight: Use the group for the baseline

GRPO - RL Without the Critic

GRPO's Insight: Use the group for the baseline

How It Works (simplified):

For each question:

1. Sample 64 different answers

2. Score them: [0.8, 0.2, 0.9, -1.0, ...]

3. Normalize: reward - mean(rewards) / std(rewards)

4. This is your advantage—no critic needed!

GRPO - RL Without the Critic

Why This Makes Sense:

- Reward models trained on comparisons anyway
- Group context gives natural baseline
- Saves 50% memory

The Formula (don't memorize, just intuition):

```
Advantage = (my_reward - average_reward) / spread_of_rewards
```

Results: MATH 46.8% → 51.7% (first open-source model >50%)

GRPO - RL Without the Critic

DeepSeekMath's Key Insight: All methods are variants of this:

$$\text{gradient} \propto \text{GradientCoefficient} \times \nabla \log p(\text{output}|\text{question})$$

Compare approaches:

STaR	Generated once	1 if correct, 0 if wrong
GRPO	Generated online	Advantage (continuous)
Online RFT	Generated online	1 if correct, 0 if wrong

DeepSeekMath: Online RL Loop

The Discovery: Online > Offline (sampling from current model beats sampling from old model)

Critical Finding:

- RL improves **Maj@K** (best of 32 tries)
- RL does NOT improve **Pass@K** (at least one correct in 32)

What this means: RL makes the model more consistent, not fundamentally smarter. It fixes "misalignment" where the model knows the answer but doesn't always produce it.

What We Will Cover

STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al. 2022)

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

DAPO: An Open-Source LLM Reinforcement Learning System at Scale

DAPO: Scaling to Long Reasoning

Naive GRPO on Qwen-32B: **30% on AIME**

- Entropy collapses (model becomes too confident)
- Training becomes unstable
- Response length explodes uncontrollably

But we know it should work—DeepSeek got 47%. What's missing?

Standard PPO clip treats increases and decreases the same

Clip-Higher: Asymmetric Exploration

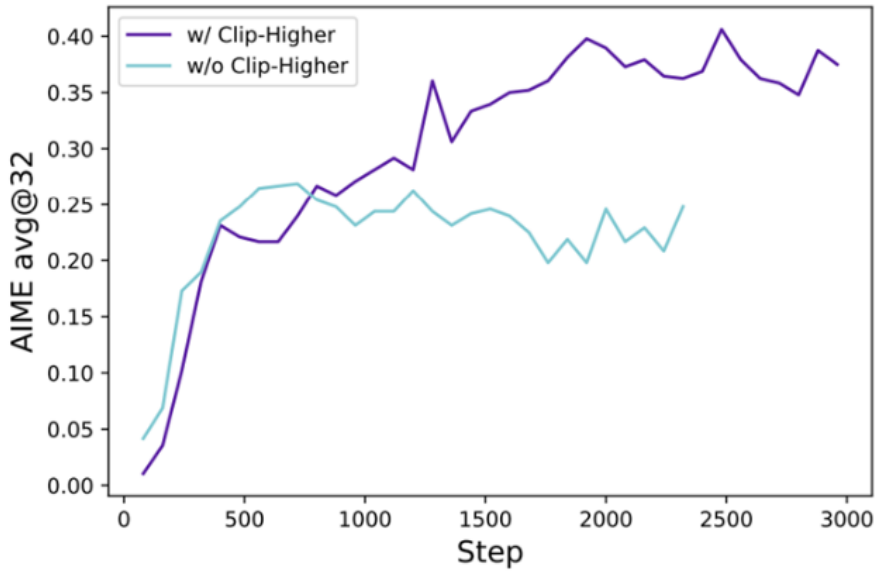
The problem: Standard PPO clip treats increases and decreases the same

- Low-probability token (0.01) can only reach 0.012 (clipped)
- High-probability token (0.9) can reach 1.08 (not clipped in practice)
- Result: Exploration dies

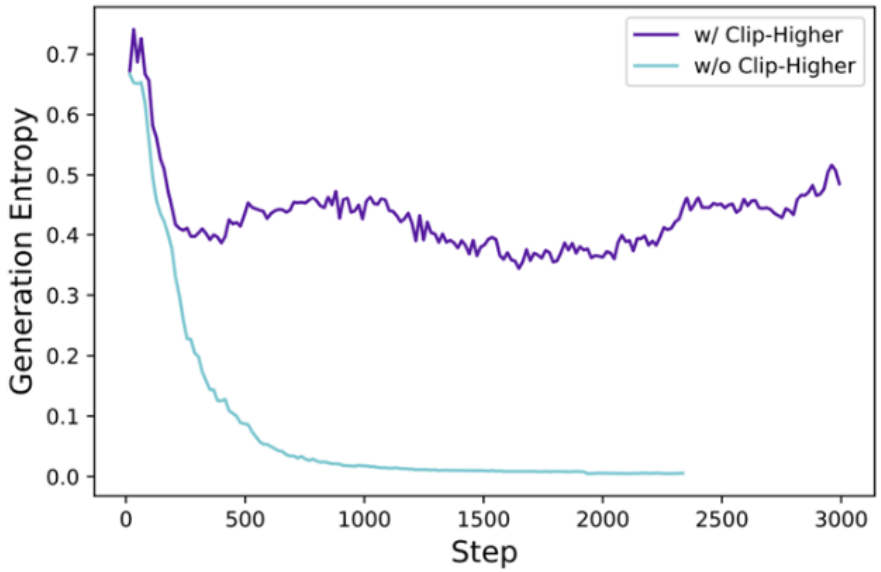
The fix: Allow bigger increases than decreases

- Impact: 30% → 38% AIME

Clip-Higher: Asymmetric Exploration



(a) Accuracies on AIME.



(b) Entropy of actor model.

Dynamic Sampling: Keep the Gradient Signal

The problem:

- 60% of questions eventually get all 64 samples correct
- All correct → advantage is zero → no gradient → wasted compute

The fix: Over-sample, filter out all-correct and all-wrong groups

- Keep only: 0 < correct_count < 64
- Maintains effective batch size

Token-Level Loss: Weight by Length

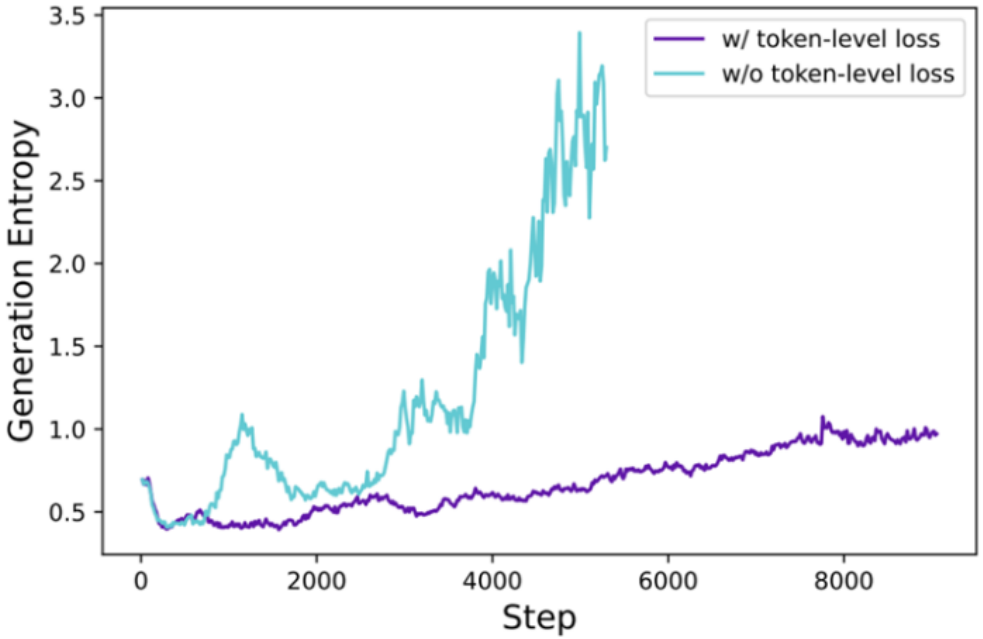
The problem:

- Sample-level: Each answer counts equally
- Long garbage answer = same weight as short good answer
- Result: Unhealthy length explosion

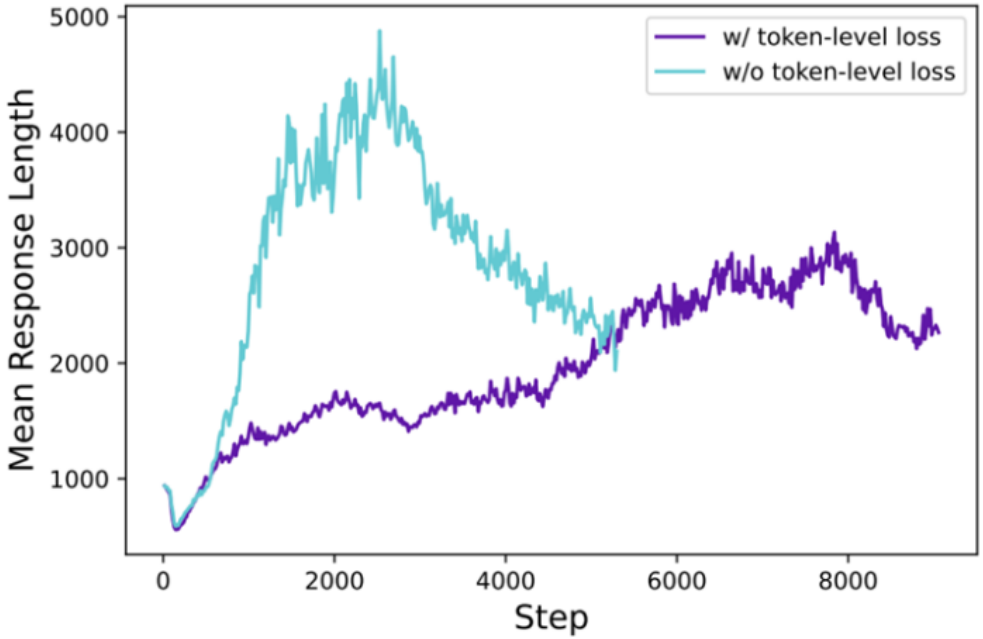
The fix: Weigh tokens, not samples

- Impact: Better length control, 41% → 42%

Token-Level Loss: Weight by Length



(a) Entropy of actor model's generation probabilities.



(b) Average length of actor model-generated responses

Soft Overlong Punishment

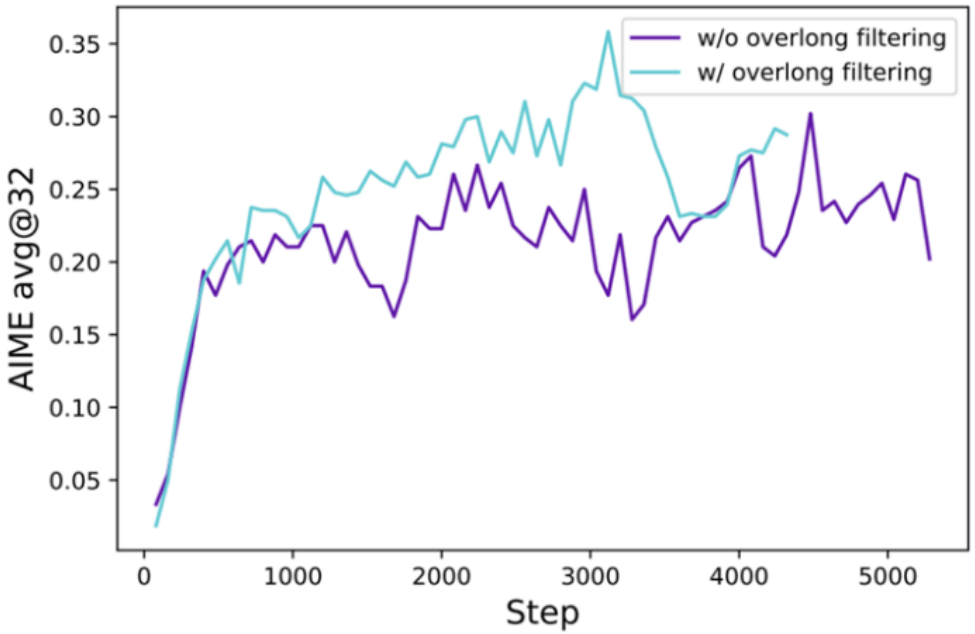
The problem: Hard cutoff at max_length adds noise

- Good reasoning punished just for being long

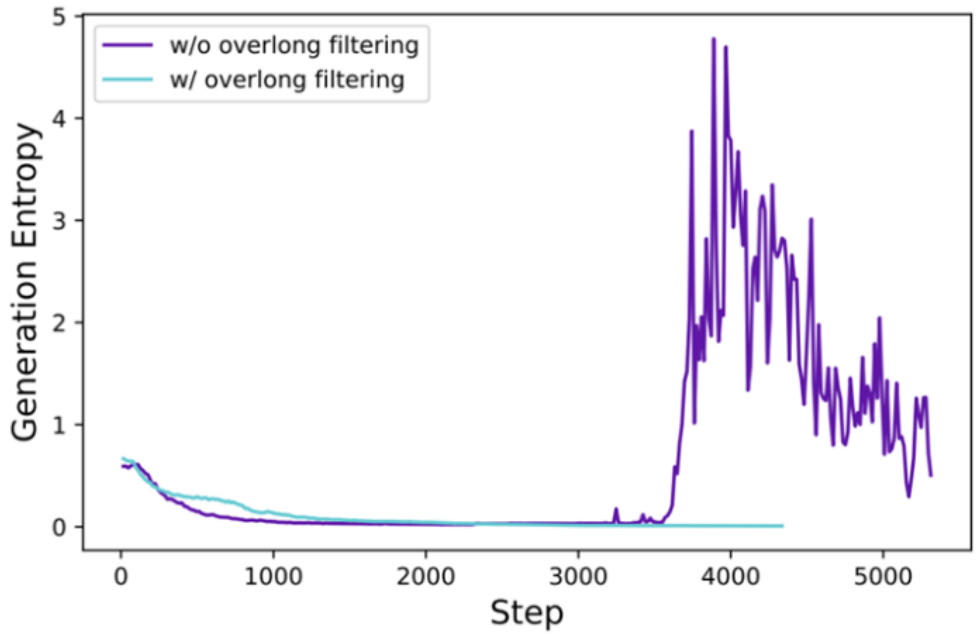
The fix: Gradual penalty in last 4K tokens (cache interval)

- Impact: Stability, 36% → 41%

Soft Overlong Punishment



(a) Performance on AIME.



(b) Entropy of actor model.

DAPO

Table 1 Main results of progressive techniques applied to **DAPO**

Model	AIME24$_{\text{avg@32}}$
DeepSeek-R1-Zero-Qwen-32B	47
Naive GRPO	30
+ Overlong Filtering	36
+ Clip-Higher	38
+ Soft Overlong Punishment	41
+ Token-level Loss	42
+ Dynamic Sampling (**DAPO**)	**50**

Scaling RL in Practice

Don't watch loss—watch these:

1. **Response length**: Should grow to ~4K tokens
2. **Entropy**: Keep in 0.4-0.6 range (too low = collapse, too high = gibberish)
3. **% samples with acc=1**: Tells you when to increase sampling

Red flags:

- Length suddenly explodes → check token-level loss
- Entropy drops to 0.2 → entropy collapse, increase ε_{hi}
- No improvement after 2K steps → reward model may be saturated

Comparative Analysis

Use STaR if:

- You have <100 examples with reasoning
- You need interpretability (education, research)
- Limited compute (no RL infrastructure)
- Task: Simple reasoning (GSM8K-level)

Use DeepSeekMath GRPO if:

- You have good base model + instruction data
- Memory-constrained (can't fit critic)
- Task: Standard math reasoning

Comparative Analysis

Use DAPO if:

- Reasoning chains >1K tokens
- Need state-of-the-art performance
- Have RL infrastructure
- Task: Competition-level problems (AIME, IMO)

What is expected to improve?

All three improve:

- Maj@K (consistency/robustness)
- Answer formatting
- Multi-step coherence

None improve:

- Pass@K (fundamental capability)
- Novel problem-solving
- Out-of-distribution generalization (much)

Implication: These are alignment/elicitation methods, not capability expansion methods.

What makes scaling RL challenging?

Neural reward models may suffer from reward hacking in the large-scale RL.

However, autonomous coding applications have access to verifiable rewards

- Correct final answer for given inputs
- Execution feedback
- Unit Tests

…

Open Problems & Future

What We Still Don't Understand

1. **Why Maj@K but not Pass@K?** (Fundamental question about what RL does)
2. **Are emergent behaviors real?** (Reflection, backtracking—learned or statistical?)
3. **How to learn from failures?** (Current: just filter them out)

Promising Directions

1. **Data**: Tree search during generation, advanced sampling
2. **Algorithms**: Noise-robust RL (reward models are ~20% wrong)
3. **Rewards**: Process rewards, uncertainty quantification
4. **Combination**: Can we merge STaR's rationalization + DAPO's techniques?

Self improvement with Search & Deep Research Agents

What We Will Cover

Competition-level code generation with AlphaCode (Li et al. 2022)

AlphaCode 2 Technical Report (2023)

Search-o1: Agentic Search-Enhanced Large Reasoning Models

What We Will Cover

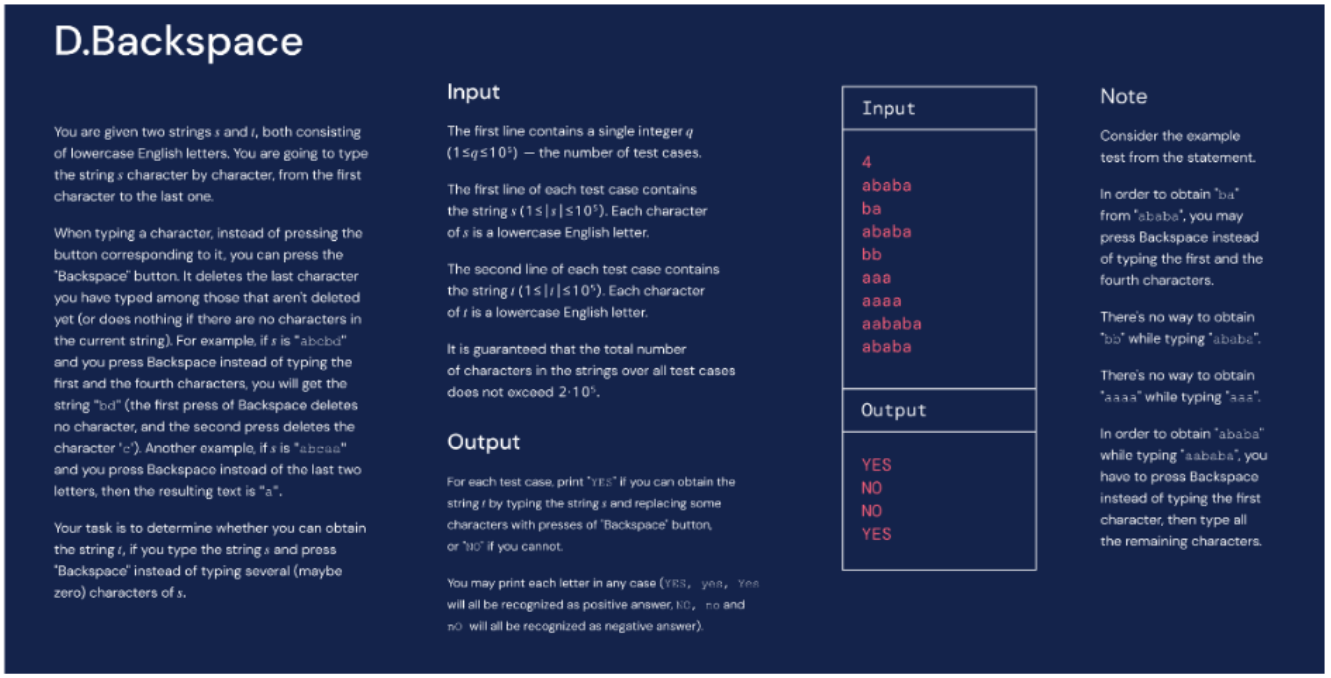
Competition-level code generation with AlphaCode (Li et al. 2022)

AlphaCode 2 Technical Report (2023)

Search-o1: Agentic Search-Enhanced Large Reasoning Models

Problem

Automating the process of writing competitive programming solutions



D.Backspace

You are given two strings s and t, both consisting of lowercase English letters. You are going to type the string s character by character, from the first character to the last one.

When typing a character, instead of pressing the button corresponding to it, you can press the "Backspace" button. It deletes the last character you have typed among those that aren't deleted yet (or does nothing if there are no characters in the current string). For example, if s is "abcbd" and you press Backspace instead of typing the first and the fourth characters, you will get the string "bd" (the first press of Backspace deletes no character, and the second press deletes the character 'c'). Another example, if s is "abcaa" and you press Backspace instead of the last two letters, then the resulting text is "a".

Your task is to determine whether you can obtain the string t, if you type the string s and press "Backspace" instead of typing several (maybe zero) characters of s.

Input

The first line contains a single integer q ($1 \le q \le 10^5$) — the number of test cases.

The first line of each test case contains the string s ($1 \le |s| \le 10^5$). Each character of s is a lowercase English letter.

The second line of each test case contains the string t ($1 \le |t| \le 10^5$). Each character of t is a lowercase English letter.

It is guaranteed that the total number of characters in the strings over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print "YES" if you can obtain the string t by typing the string s and replacing some characters with presses of "Backspace" button, or "NO" if you cannot.

You may print each letter in any case (YES, yes, Yes will all be recognized as positive answer, NO, no and nO will all be recognized as negative answer).

Input

```
4
ababa
ba
ababa
bb
aaa
aaaa
aababa
ababa
```

Output

```
YES
NO
NO
YES
```

Note

Consider the example test from the statement.

In order to obtain "ba" from "ababa", you may press Backspace instead of typing the first and the fourth characters.

There's no way to obtain "bb" while typing "ababa".

There's no way to obtain "aaaa" while typing "aaa".

In order to obtain "ababa" while typing "aababa", you have to press Backspace instead of typing the first character, then type all the remaining characters.

Problem

Automating the process of solving competitive programming

Applications:

- Make existing programmers more productive
  - Code completions
  - Optimizing blocks of code
- Allow developers operate at a higher level of abstraction that hides details

Results Preview

AlphaCode ranks among the top 54% among contest participants over 10 contests

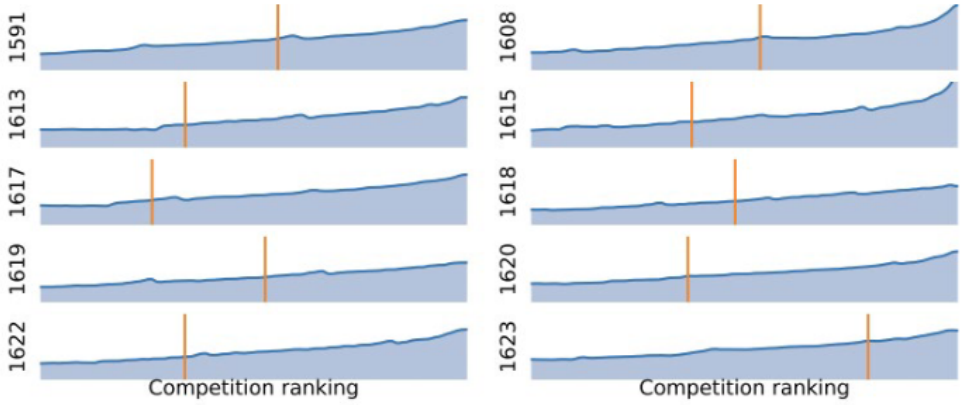
It demonstrates AI's ability to generalize beyond narrow tasks to creatively solve diverse problems.

Solution (output)

First AlphaCode reads the two phrases.

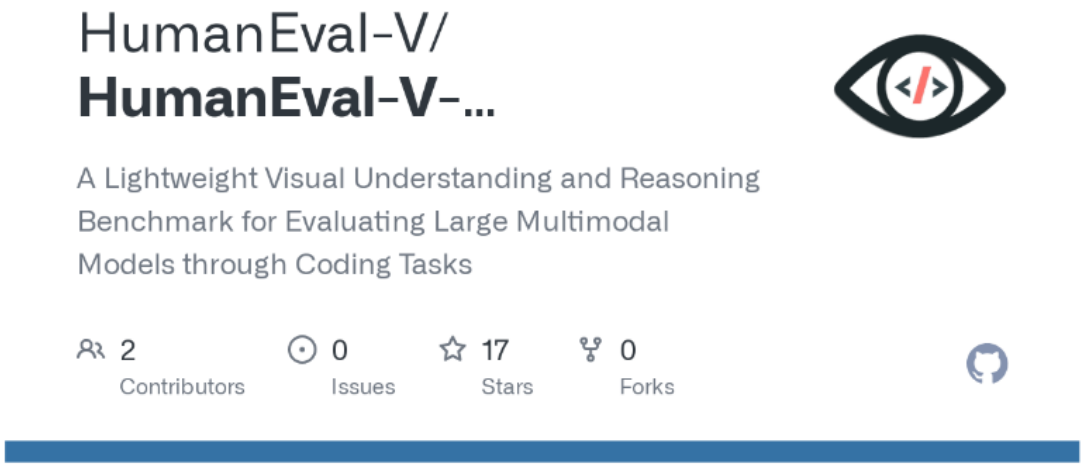
Backspace deletes two letters. The letter you press backspace instead of, and the letter before it.

```
t=int(input())
for i in range(t):
    s=input()
    t=input()
    a=[]
    b=[]
    for j in s:
        a.append(j)
    for j in t:
        b.append(j)
    a.reverse()
    b.reverse()
    c=[]
    while len(b)!=0 and len(a)!=0:
        if a[0]==b[0]:
            c.append(b.pop(0))
            a.pop(0)
        elif a[0]!=b[0] and len(a)!=1:
            a.pop(0)
            a.pop(0)
        elif a[0]!=b[0] and len(a)==1:
            a.pop(0)
    if len(b)==0:
        print("YES")
    else:
        print("NO")
```



(a) AlphaCode's ranking in 10 contests

Prior work has focused on simpler and shorter problems



measure functional correctness for synthesizing programs from docstrings

Codeforce problems are **4x longer** in terms of description length and solution length.

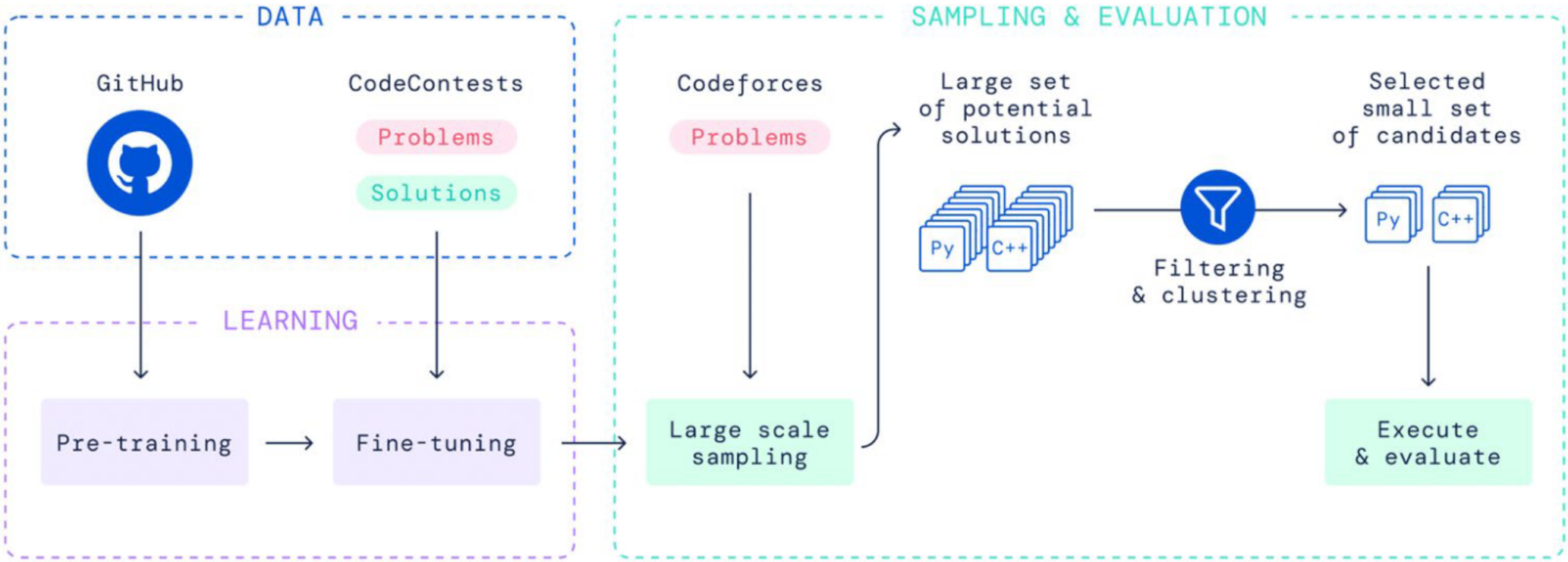
HumanEval problems tend to include instructions about **exactly what to implement** (competitive programming problems have no suggested implementation)

Prior work has focused on simpler and shorter problems

Prior code completion systems only generated suggestions for at most 1 line

Easier to get high success rate in completing hand-specified Python functions given the function signature and docstring

Method Overview

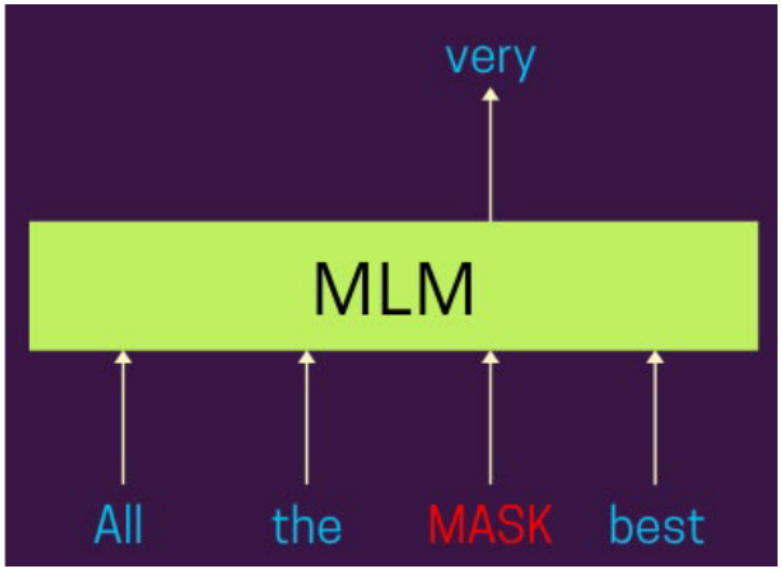


Pretraining

Dataset: Scrape Github for Code (715.1 GB)

Decoder: standard cross entropy next token prediction loss

Encoder: masked LM loss



Fine-tuning

Tempering: Regularization technique to sharpen the training distribution

- Model learns to assign higher probabilities to the most meaningful patterns rather than spreading probability across irrelevant options.

Value Conditioning & Prediction: insert whether or not a submission was correct

GOLD: A single problem can have many valid solutions due to differences in algorithm choice, implementation style, or language constructs. GOLD helps focus on *precision* by:

1. Focusing on high-likelihood tokens: Reinforce predictions it already handles well
2. Ignoring low-likelihood tokens: Avoid focusing on irrelevant or unlikely tokens

Large Scale Sampling

Generate **1M** ‼️diverse sample programs programs per question:

- ½ in Python, ½ in C++
- Randomized the problem tags and ratings in the natural language prompt
- High sampling temperature

Filter and Cluster to Generate a Candidate Set of Solutions

Filter: filtering samples to only those that pass the example tests given in the problem statement (removes approximately 99% of model samples)

Cluster to differentiate syntactically different but semantically equivalent programs

- Use a separate test input generation model
- Trained to predict test inputs from problem descriptions, using example, hidden, and generated test inputs as training data
- Creates new test inputs for unseen problems

Results

Evaluation on Codeforces (actual competition platform)

- Checks program correctness more thoroughly
- Benchmarks against the best performers on this task: human competitors

Evaluation on CodeContests (dataset created by authors)

- Contains false positives
- Accepts algorithmically inefficient solutions
- Lower variance and easier to measure (no submitting to an external site)

Evaluation on Codeforces platform

- 10 competitions with >5000 participants each
- Ran AlphaCode live, generated samples, filtered with example tests, clustered to get candidate submissions, and submitted to platform
- Top 54.3% average ranking (limiting to 10 submissions per problem)
- Top 28% of Codeforces competitors in the last 6 months

Contest ID	1591	1608	1613	1615	1617	1618	1619	1620	1622	1623	Average
Best	43.5%	43.6%	59.8%	60.5%	65.1%	32.2%	47.1%	54.0%	57.5%	20.6%	**48.4%**
Estimated	44.3%	46.3%	66.1%	62.4%	73.9%	52.2%	47.3%	63.3%	66.2%	20.9%	**54.3%**
Worst	74.5%	95.7%	75.0%	90.4%	82.3%	53.5%	88.1%	75.1%	81.6%	55.3%	**77.2%**

Estimated % ranking of AlphaCode in 10 competitions (lower is better)

Evaluation on CodeContests dataset

pass@k

% of problems solved when we take k samples from the model for each problem and submit all of them for evaluation on hidden tests

Measures the search aspect of the sampling process

10@k

% of problems solved when we take k samples from the model for each problem but only submit 10 for evaluation on hidden tests

Measures the filtering process and how models behave at a very large number of samples

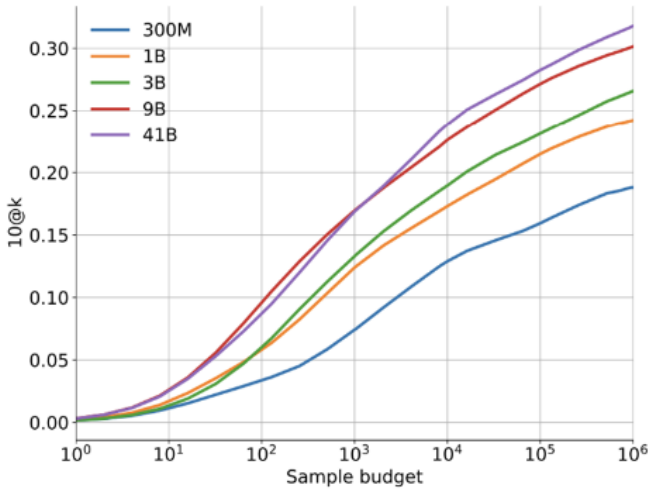
Evaluation on CodeContests dataset

Approach	Validation Set				Test Set		
	10@1k	10@10k	10@100k	10@1M	10@1k	10@10k	10@100k
9B	16.9%	22.6%	27.1%	30.1%	14.3%	21.5%	25.8%
41B	16.9%	23.9%	28.2%	31.8%	15.6%	23.2%	27.7%
41B + clustering	21.0%	26.2%	31.8%	34.2%	16.4%	25.4%	29.6%

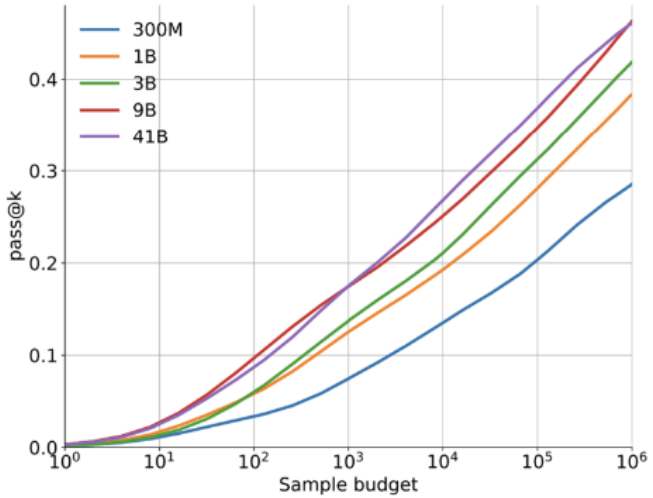
Table 5 | **Solve rates of our best systems on the validation set and test set .**

- 41B consistently outperforms 9B model
- Clustering consistently provides an improvement
- Context: competitive coding problems are quite difficult!

Ablations with # of samples



(a) 10 attempts per problem



(b) Unlimited attempts per problem

Solve rate scaling vs. number of samples

- Solve rates scale log-linearly with more samples (and compute)
- Better models have higher slopes in the scaling curve
- Even when only 10 samples are submitted, scaling up total # of samples → drastically improves solve rate
  - Average of 2.4 submissions for each problem solved in Codeforces evaluation

Strengths vs. Weaknesses

- Sampling, filtering, and clustering approach leverages high coverage and stronger reasoning
- Robust evaluation framework
- Interpretability of generated code
- Model does not copy existing solutions; actually reasons
- Reduces FPR compared to prior works

- Reliance on hidden test case coverage
- Loss is poor proxy for solve rate (solving problem is one-of-many task)
- Solve rate as a metric has high variance and compute cost
- Worse performance on DP and constructive algorithms
- Computational Intensity: Large scale sampling (1m+/problem) yet filter 99% of samples filter
- Test set dataset size is small (n=165)

Additional Experiments + Reproducibility

- Examining accuracy vs. efficiency (most optimal solution)
  - There is a time limit bound, but further ranking acceptable solutions
- Further experiments on more difficult problem domains
  - DP, constructive algorithms, graphs

- Large sampling amount requires massive compute (even compared to LLMs)
  - 1 million samples drawn per problem per model

What We Will Cover

Competition-level code generation with AlphaCode (Li et al. 2022)

AlphaCode 2 Technical Report (2023)

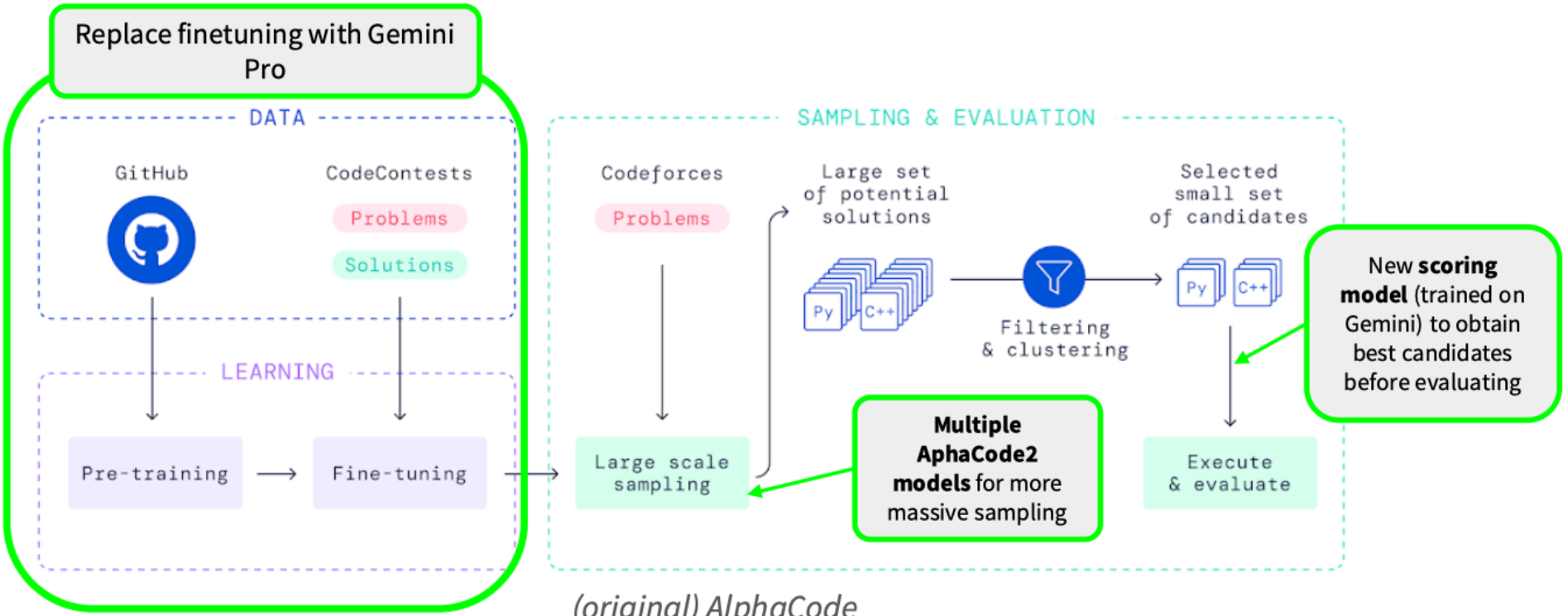
Search-o1: Agentic Search-Enhanced Large Reasoning Models

AlphaCode 2: Problem Statement, Motivation

Gemini Pro

Can using Large Language Models (Gemini Pro) improve AlphaCode (specifically sampling and scoring) for a better AlphaCode 2?

AlphaCode 2: New Ideas and Contributions



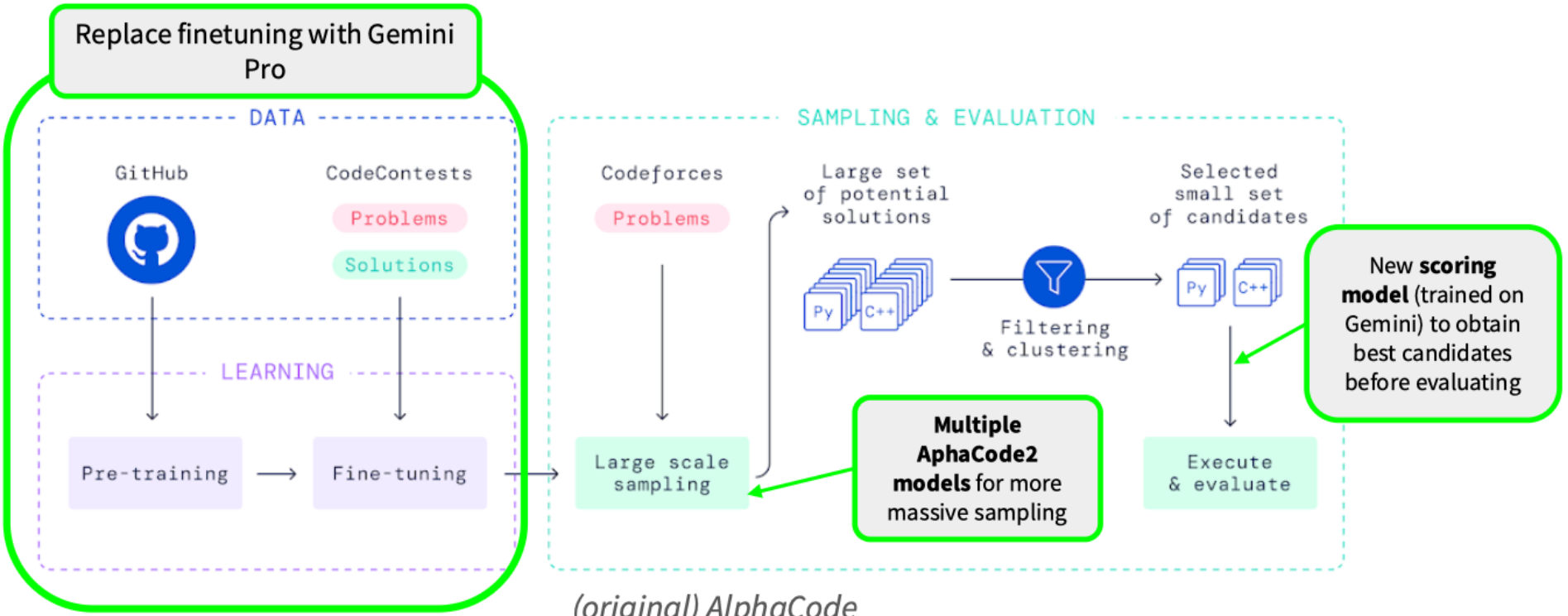
(original) AlphaCode

AlphaCode 2: New Ideas and Contributions

- ⭐ Fine-tune another Gemini Pro model to **score "correctness"** before submitting
- ⭐ **"Family of models"** – Generate several fine-tuned models w/ varied hyperparameters (e.g. difficulty, tags) → maximise diversity

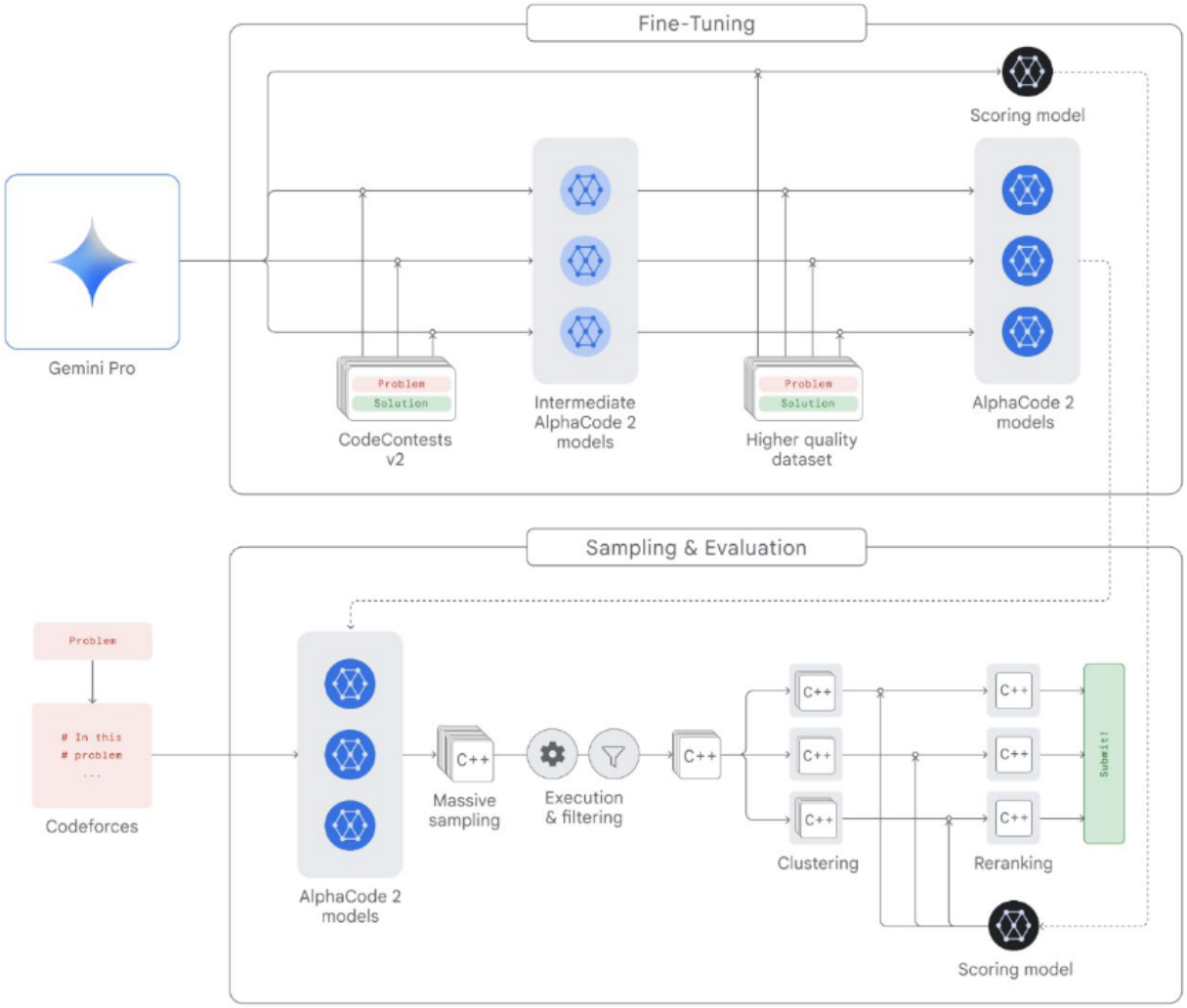
- Using an **LLM** (Gemini Pro) to fine-tune instead of pre-training on GitHub
- CodeContestsV2 dataset and second "higher quality dataset"

Methodology - AlphaCode2 (almost)

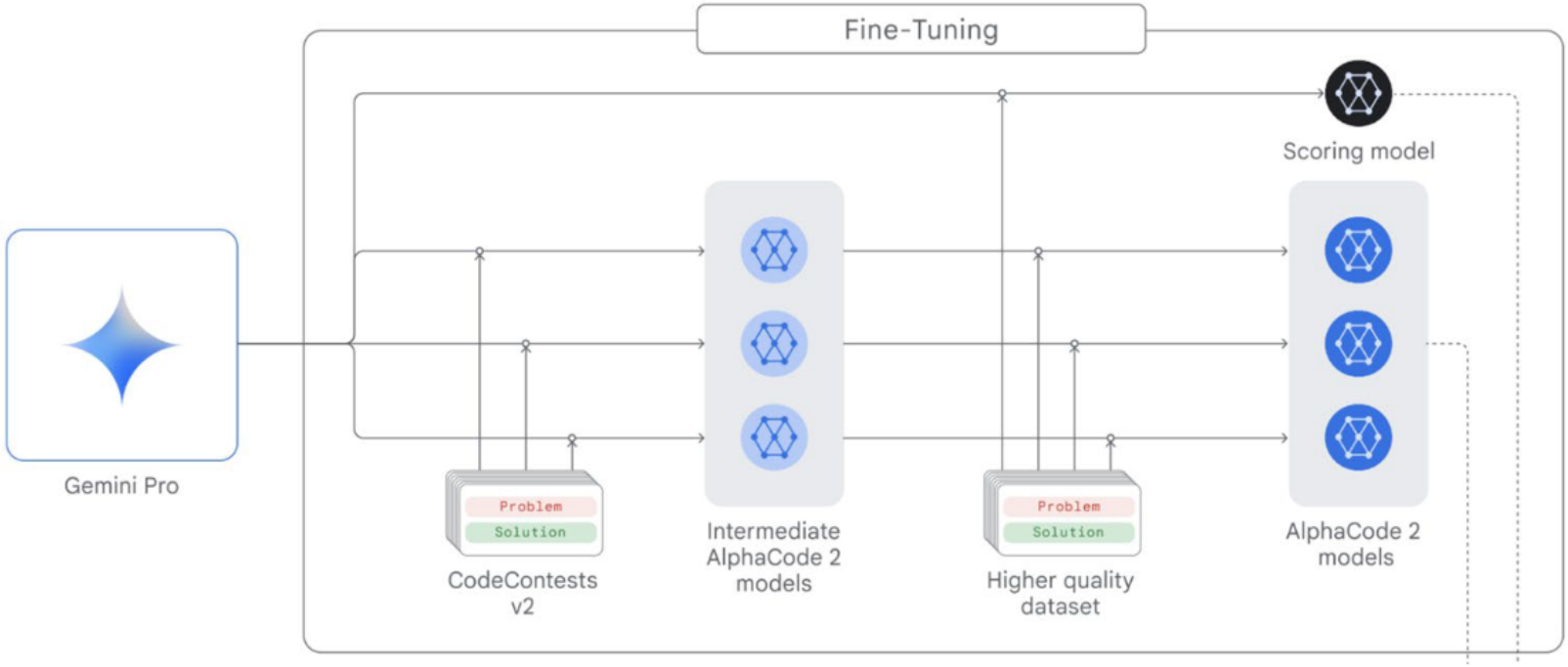


(original) AlphaCode

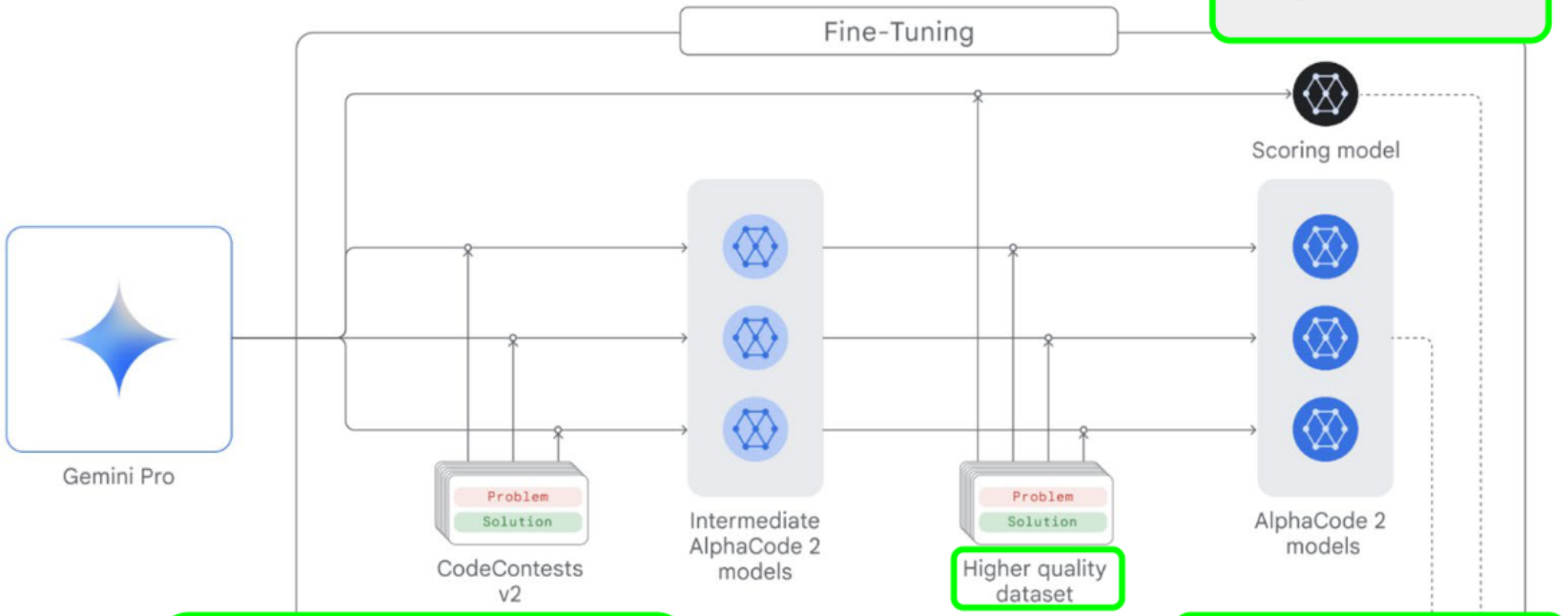
AlphaCode2


Fine-Tuning
Scoring model
Gemini Pro
Problem
Solution
CodeContests v2
Intermediate AlphaCode 2 models
Problem
Solution
Higher quality dataset
AlphaCode 2 models
Sampling & Evaluation
Problem
In this
problem
...
Codeforces
AlphaCode 2 models
C++
Massive sampling
Execution & filtering
C++
C++
C++
C++
Clustering
C++
C++
Reranking
Submit!
Scoring model

AlphaCode2


Fine-Tuning
Scoring model
Gemini Pro
Problem
Solution
CodeContests v2
Intermediate AlphaCode 2 models
Problem
Solution
Higher quality dataset
AlphaCode 2 models

AlphaCode2



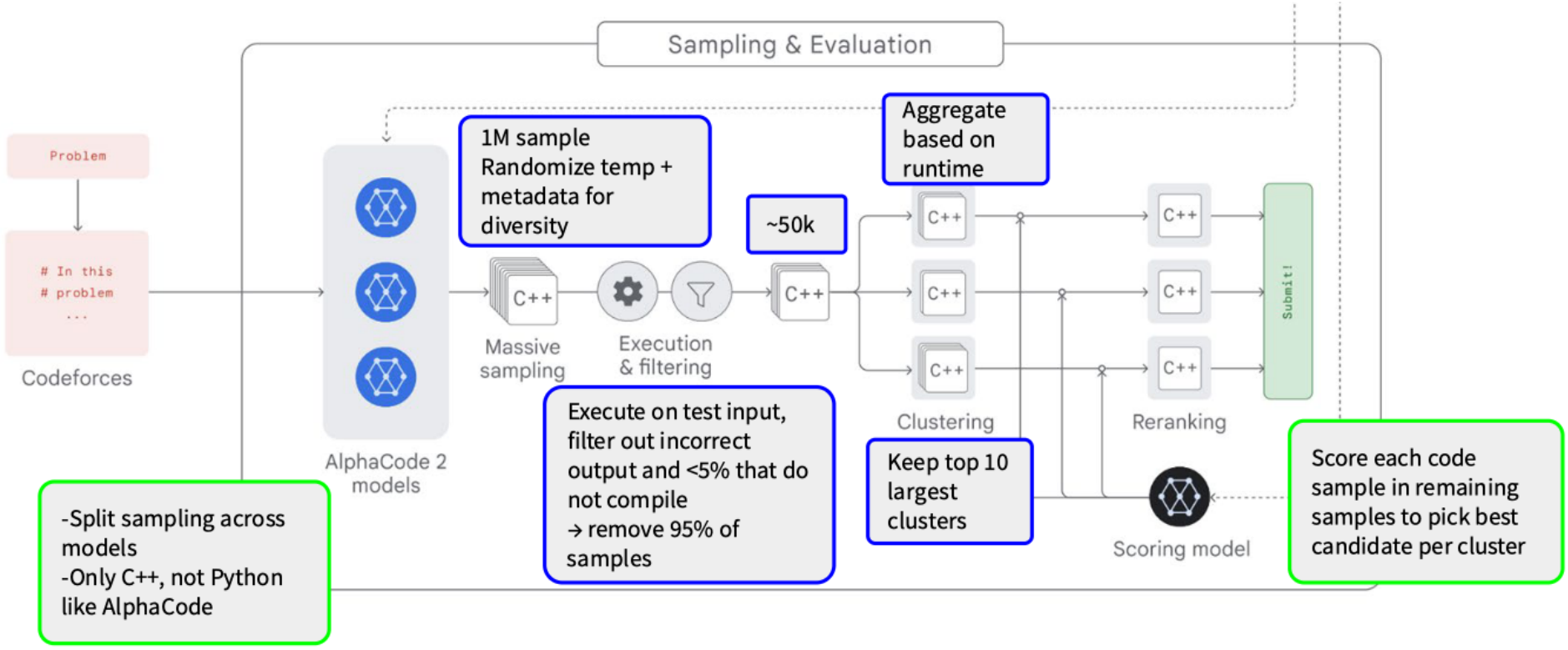
Estimated correctness of code sample between 0-1

more problems + solutions of higher quality

manually-curated tests on the validation set

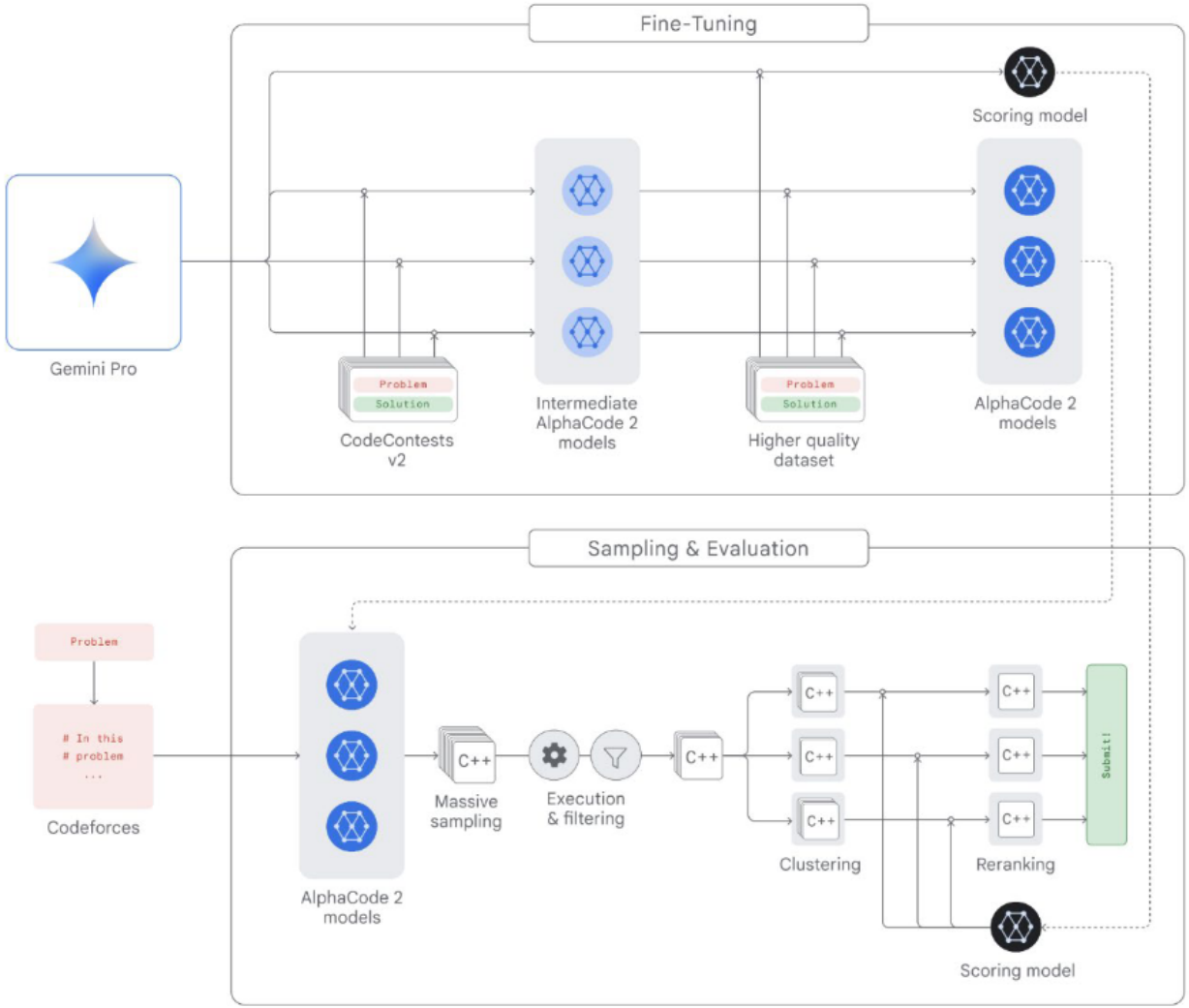
"Family of models" instead of one
→ maximise diversity

AlphaCode2

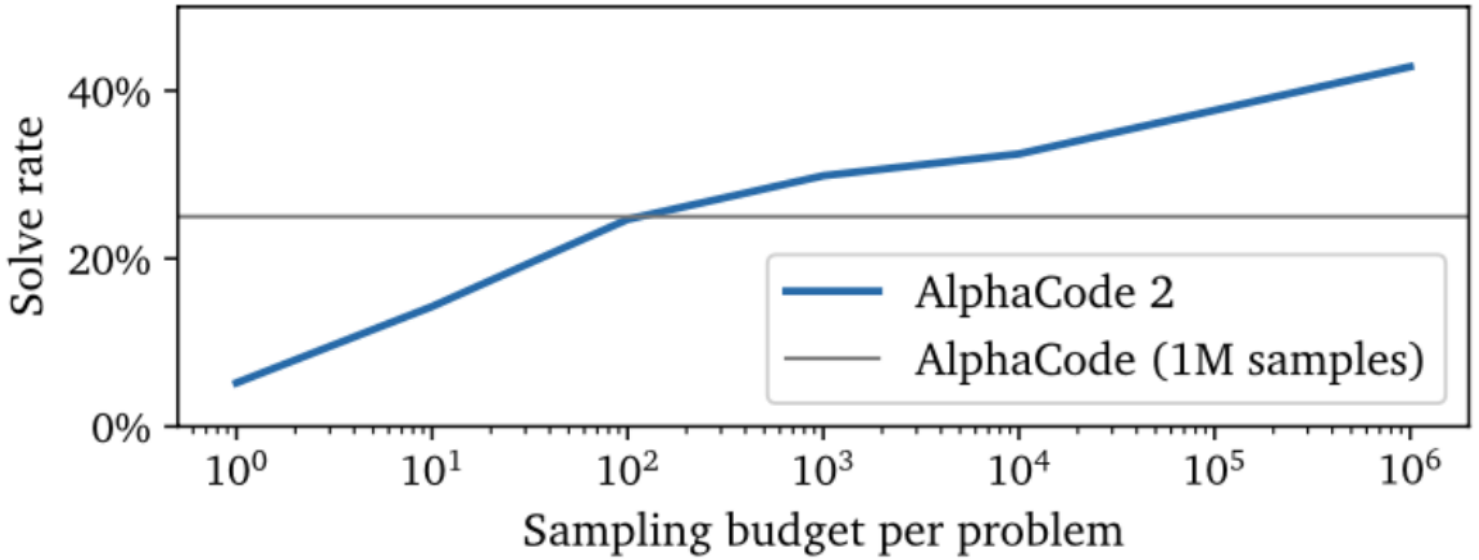

Sampling & Evaluation
Problem
In this
problem
...
Codeforces
AlphaCode 2 models
1M sample
Randomize temp + metadata for diversity
Massive sampling
Execution & filtering
~50k
Execute on test input, filter out incorrect output and <5% that do not compile
→ remove 95% of samples
Aggregate based on runtime
C++
Clustering
Keep top 10 largest clusters
Reranking
Scoring model
Submit!
Score each code sample in remaining samples to pick best candidate per cluster
-Split sampling across models
-Only C++, not Python like AlphaCode

AlphaCode2

**** Eval is same as AlphaCode***

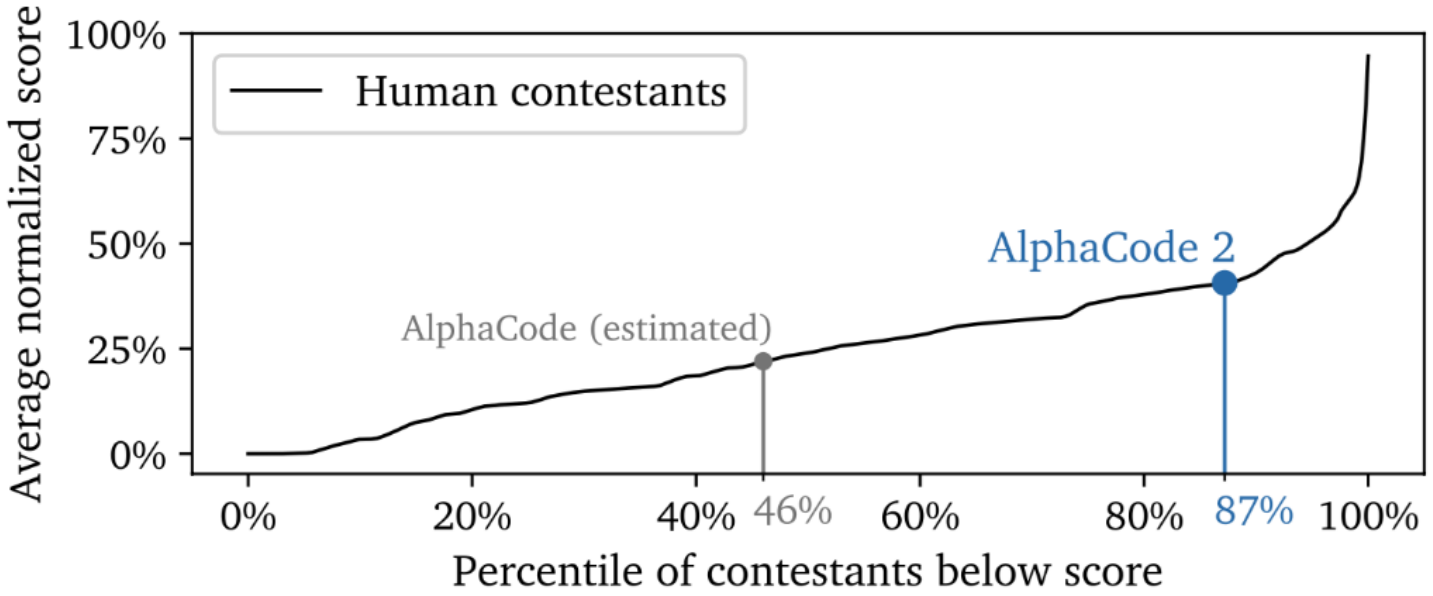


Experimental Results - AlphaCode2



- 43% solved (2x improvement over AlphaCode's 25%)
- AlphaCode 2 requires 100 samples to reach level of performance of AphaCode with a million samples

Experimental Results - AlphaCode 2



- Human competitors (divided by best human score per contest) averaged on the 12 contests
- AlphaCode 2 at 85th percentile between Expert and Candidate Master
- AlphaCode outperforms 46%, AlphaCode 2 outperforms 99.5% (for top 2 performance on qs)

Strengths vs. Weaknesses - AlphaCode2

✅

- Increases performance with less samples (100 samples to meet 1M samples)
    - Promising prospects for future large foundation models
- Scoring model to select best candidate

❌

- Experimentation is costly to operate at scale
- Very specific to code (e.g. being able to filter out bad code sample)

What We Will Cover

Competition-level code generation with AlphaCode (Li et al. 2022)

AlphaCode 2 Technical Report (2023)

Search-o1: Agentic Search-Enhanced Large Reasoning Models

Search-o1: Agentic Search-Enhanced Large Reasoning Models

Key Innovation: Enhancing Large Reasoning Models (LRMs) with autonomous retrieval capabilities

LRMs like OpenAI-o1 exhibit impressive reasoning but suffer from knowledge insufficiency during extended reasoning chains

Problem: Knowledge Gaps in Reasoning

Challenge: Long-form reasoning means more opportunities for knowledge gaps

Evidence from Research:

- Word "perhaps" appears **30+ times** per reasoning output on GPQA dataset
- Uncertain phrases indicate knowledge insufficiency:
  - "perhaps," "alternatively," "wait," "likely"

Example Uncertainty:

> "Wait, perhaps it's referring to dimethyl sulfone, but that doesn't seem right..."

Impact: Any knowledge gap can propagate errors through the entire reasoning chain

What if we use Standard RAG

Traditional RAG Limitations:

1. **Single Retrieval:** Retrieves once at the beginning, problem-oriented
2. **Diverse Knowledge Needs:** Each reasoning step may need different information
3. **Insufficient for Complex Reasoning:** Cannot adapt to evolving information needs

Performance Gap:

- Standard RAG shows minimal improvement over direct reasoning
- Cannot address knowledge gaps that emerge during multi-step reasoning

Search-o1 Solution

Two Core Components:

1. Agentic RAG Mechanism

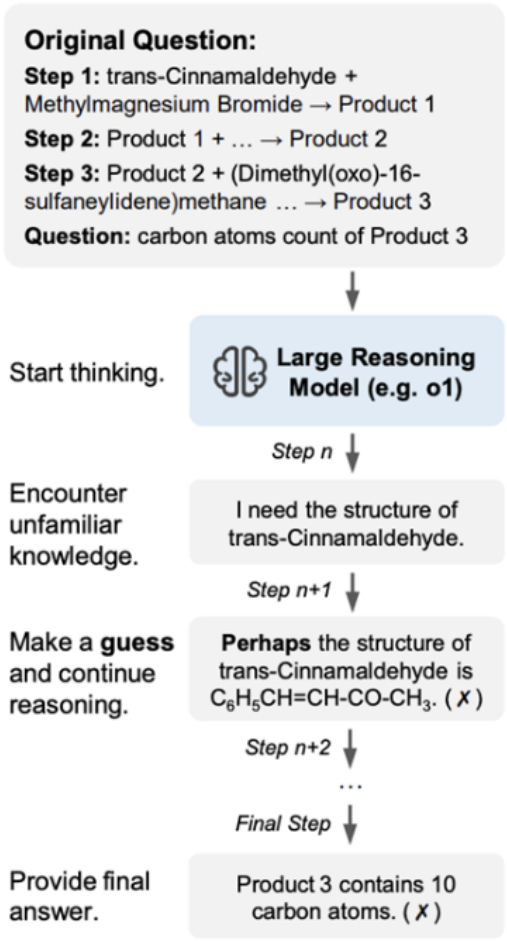
- Model **autonomously** generates search queries
- Triggered when encountering knowledge gaps
- **Multiple iterations** within single reasoning session

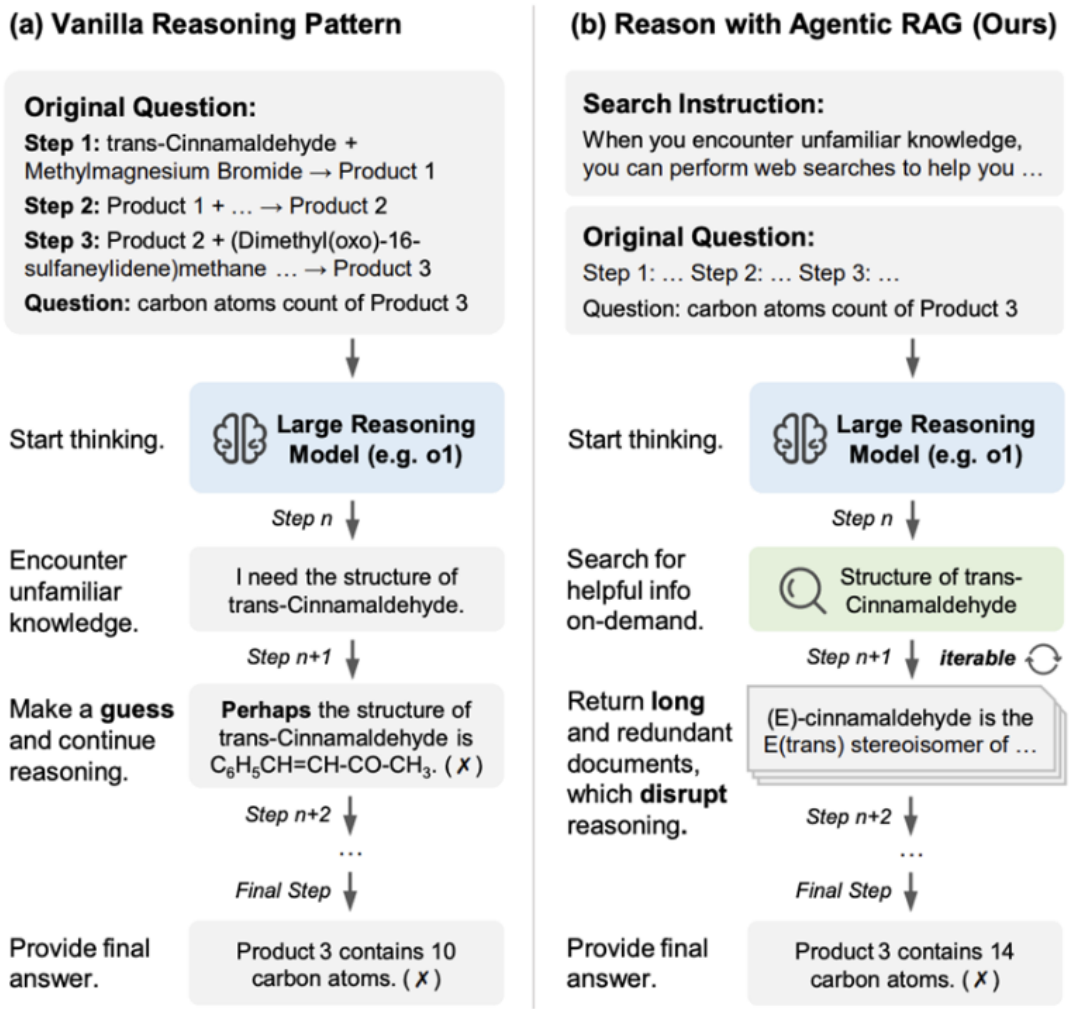
2. Reason-in-Documents Module

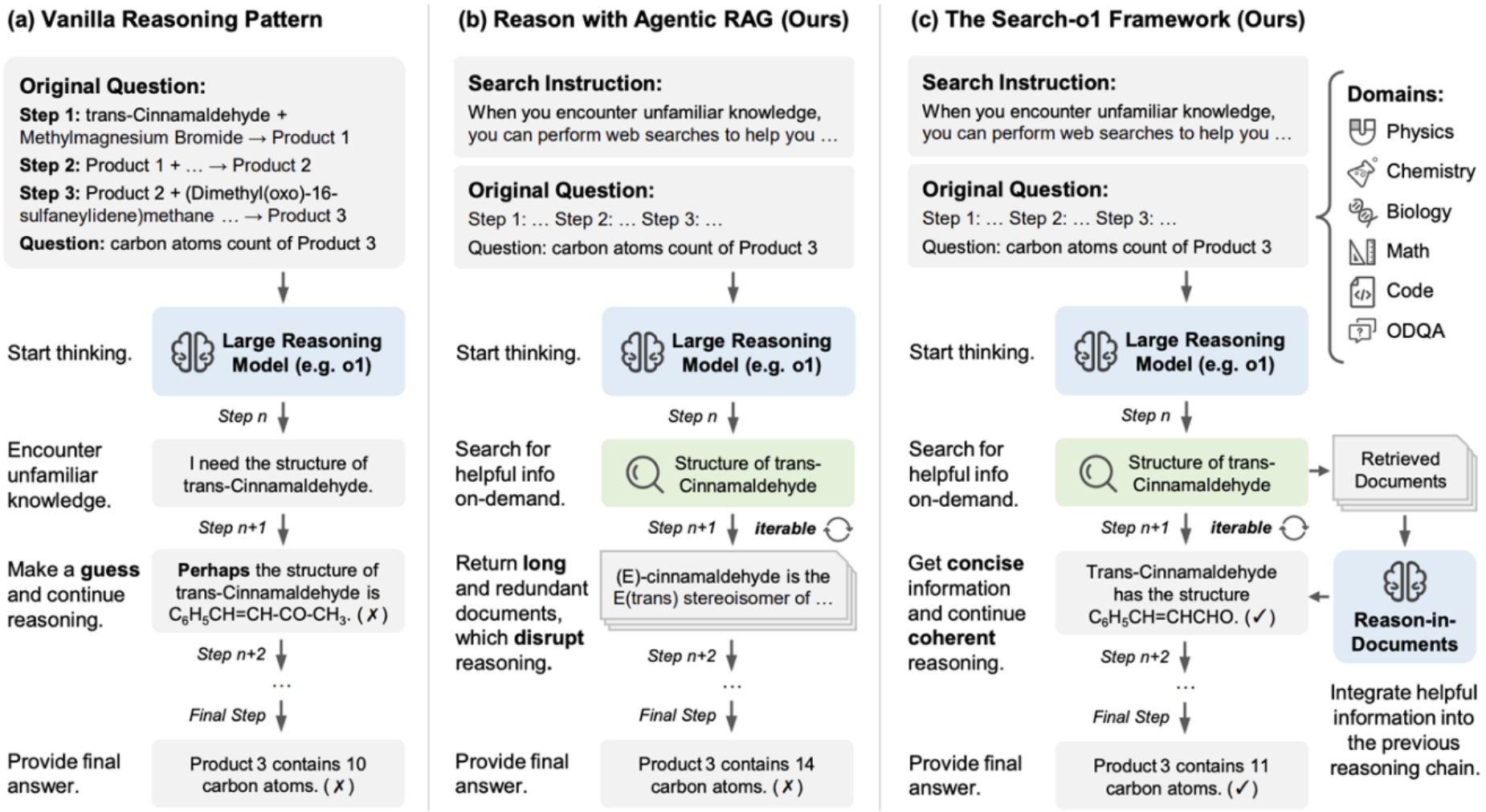
- Analyzes retrieved documents separately
- Extracts relevant information
- Integrates seamlessly into reasoning chain

Key Advantage: Maintains reasoning coherence while incorporating external knowledge

(a) Vanilla Reasoning Pattern




(a) Vanilla Reasoning Pattern
Original Question:
Step 1: trans-Cinnamaldehyde + Methylmagnesium Bromide → Product 1
Step 2: Product 1 + … → Product 2
Step 3: Product 2 + (Dimethyl(oxo)-16-sulfaneylidene)methane … → Product 3
Question: carbon atoms count of Product 3
Start thinking.
Large Reasoning Model (e.g. o1)
Step n
Encounter unfamiliar knowledge.
I need the structure of trans-Cinnamaldehyde.
Step n+1
Make a guess and continue reasoning.
Perhaps the structure of trans-Cinnamaldehyde is $C_6H_5CH=CH-CO-CH_3$. (✗)
Step n+2
…
Final Step
Provide final answer.
Product 3 contains 10 carbon atoms. (✗)
(b) Reason with Agentic RAG (Ours)
Search Instruction:
When you encounter unfamiliar knowledge, you can perform web searches to help you …
Original Question:
Step 1: … Step 2: … Step 3: …
Question: carbon atoms count of Product 3
Start thinking.
Large Reasoning Model (e.g. o1)
Step n
Search for helpful info on-demand.
Structure of trans-Cinnamaldehyde
Step n+1
iterable
Return long and redundant documents, which disrupt reasoning.
(E)-cinnamaldehyde is the E(trans) stereoisomer of …
Step n+2
…
Final Step
Provide final answer.
Product 3 contains 14 carbon atoms. (✗)


(a) Vanilla Reasoning Pattern
Original Question:
Step 1: trans-Cinnamaldehyde + Methylmagnesium Bromide → Product 1
Step 2: Product 1 + … → Product 2
Step 3: Product 2 + (Dimethyl(oxo)-16-sulfaneylidene)methane … → Product 3
Question: carbon atoms count of Product 3
Start thinking.
Large Reasoning Model (e.g. o1)
Step n
Encounter unfamiliar knowledge.
I need the structure of trans-Cinnamaldehyde.
Step n+1
Make a guess and continue reasoning.
Perhaps the structure of trans-Cinnamaldehyde is $C_6H_5CH=CH-CO-CH_3$. (✗)
Step n+2
…
Final Step
Provide final answer.
Product 3 contains 10 carbon atoms. (✗)
(b) Reason with Agentic RAG (Ours)
Search Instruction:
When you encounter unfamiliar knowledge, you can perform web searches to help you …
Original Question:
Step 1: … Step 2: … Step 3: …
Question: carbon atoms count of Product 3
Start thinking.
Large Reasoning Model (e.g. o1)
Step n
Search for helpful info on-demand.
Structure of trans-Cinnamaldehyde
Step n+1
iterable
Return long and redundant documents, which disrupt reasoning.
(E)-cinnamaldehyde is the E(trans) stereoisomer of …
Step n+2
…
Final Step
Provide final answer.
Product 3 contains 14 carbon atoms. (✗)
(c) The Search-o1 Framework (Ours)
Search Instruction:
When you encounter unfamiliar knowledge, you can perform web searches to help you …
Original Question:
Step 1: … Step 2: … Step 3: …
Question: carbon atoms count of Product 3
Domains:
Physics
Chemistry
Biology
Math
Code
ODQA
Start thinking.
Large Reasoning Model (e.g. o1)
Step n
Search for helpful info on-demand.
Structure of trans-Cinnamaldehyde
Retrieved Documents
Step n+1
iterable
Get concise information and continue coherent reasoning.
Trans-Cinnamaldehyde has the structure $C_6H_5CH=CHCHO$. (✓)
Reason-in-Documents
Step n+2
…
Final Step
Integrate helpful information into the previous reasoning chain.
Provide final answer.
Product 3 contains 11 carbon atoms. (✓)

Agentic RAG Mechanism

How It Works:

Special Tokens:

- `<|begin_search_query|>` ... `<|end_search_query|>` → Trigger search
- `<|begin_search_result|>` ... `<|end_search_result|>` → Inject results

Iterative Process:

1. Model generates reasoning tokens
2. When uncertain, generates search query between special tokens
3. System detects tokens, extracts query
4. Retrieval function returns documents
5. Documents injected back into reasoning chain
6. Repeat as needed

$$P(q_{\text{search}}^{(i)} \mid I, q, \mathcal{R}^{(i-1)}) = \prod_{t=1}^{T_q^{(i)}} P\left(q_{\text{search},t}^{(i)} \mid q_{\text{search},<t}^{(i)}, I, q, \mathcal{R}^{(i-1)}\right)$$

Reason-in-Documents Module

Two Key Challenges:

1. **Redundant Information:** Retrieved docs are lengthy, contain noise
2. **Limited Long-Context Understanding:** LRMs suffer "catastrophic forgetting" of general capabilities

Solution:

- **Independent analysis** of retrieved documents
- Considers: current query + previous reasoning + retrieved content
- Produces **refined information** that integrates seamlessly

Benefit: Maintains coherent reasoning flow while adding external knowledge

Example

Question: Carbon atom count in Product 3 from multi-step reaction?

Vanilla Reasoning:

- Guesses structure → Wrong answer (14 atoms) ✗

Agentic RAG (without refinement):

- Retrieves documents → Too lengthy, disrupts reasoning ✗

Search-o1:

1. Searches: "structure of trans-Cinnamaldehyde"
2. Refines to: "C6H5CH=CHCHO structure"
3. Integrates cleanly into reasoning
4. **Correct answer: 11 carbon atoms** ✓

Results

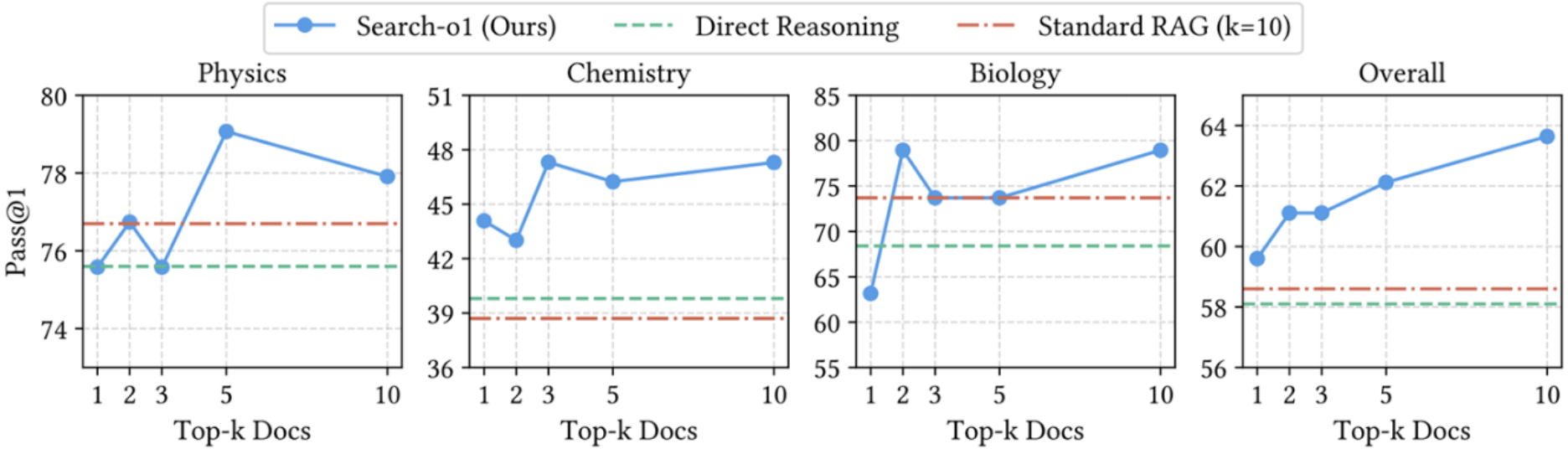


Figure 3: Scaling analysis of top-k retrieved documents utilized in reasoning. All results are based on QwQ-32B-Preview model.

Results

+5.5% over direct reasoning

+24% average improvement over RAG baselines (Qwen2.5-7B)

Outperforms human experts in physics (68.7% vs 57.9%)

Table 2: Performance comparison with human experts on the GPQA extended set [52].

Method	GPQA Extended Set			
	Physics	Chemistry	Biology	Overall
Human Experts				
Physicists	57.9	31.6	42.0	39.9
Chemists	34.5	**72.6**	45.6	48.9
Biologists	30.4	28.8	<u>68.9</u>	37.2
Reasoning Models				
QwQ-32B	61.7	36.9	61.0	51.8
RAG-QwQ-32B	<u>64.3</u>	38.3	66.7	<u>54.6</u>
Search-o1 (Ours)	**68.7**	<u>40.7</u>	**69.5**	**57.9**

Scaling Analysis - Retrieved Documents

Search-o1 effectively leverages increasing documents

<cite index="9-9">Performance improves with more retrieved documents (top-k from 1 to 10)</cite>

Key Insight:

- Even **1 document** with Search-o1 > 10 documents with Standard RAG
- Demonstrates effectiveness of agentic search + document refinement

Performance by Subject (with increasing k):

- Physics: 74% → 78%
- Chemistry: 36% → 48%
- Biology: 55% → 85%

Multi-Hop QA Performance

Average Improvement:

- **+29.6%** over standard RAG
- **+5.3%** over agentic RAG without refinement

Demonstrates: Value of clean knowledge integration in complex reasoning

Method	Single-hop QA				Multi-hop QA							
	NQ		TriviaQA		HotpotQA		2WIKI		MuSiQue		Bamboogle	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Direct Reasoning (w/o Retrieval)												
Qwen2.5-32B	22.8	33.9	52.0	60.3	25.4	34.7	29.8	36.3	8.4	18.0	49.6	63.2
QwQ-32B	23.0	33.1	53.8	60.7	25.4	33.3	34.4	40.9	9.0	18.9	38.4	53.7
Qwen2.5-72B	27.6	41.2	56.8	65.8	29.2	38.8	34.4	42.7	11.4	20.4	47.2	61.7
Llama3.3-70B	36.0	48.7	68.8	76.8	37.8	49.1	46.0	54.2	14.8	23.6	54.4	67.8
Retrieval-augmented Reasoning												
RAG-Qwen2.5-32B	33.4	49.3	**65.8**	**79.2**	38.6	50.4	31.6	40.6	10.4	19.8	52.0	66.0
RAG-QwQ-32B	29.6	44.4	65.6	77.6	34.2	46.4	35.6	46.2	10.6	20.2	55.2	67.4
RAgent-Qwen2.5-32B	32.4	47.8	63.0	72.6	44.6	56.8	55.4	69.7	13.0	25.4	54.4	66.4
RAgent-QwQ-32B	33.6	48.4	62.0	74.0	43.0	55.2	**58.4**	71.2	13.6	25.5	52.0	64.7
Retrieval-augmented Reasoning with Reason-in-Documents												
Search-o1 (Ours)	**34.0**	**49.7**	63.4	74.1	**45.2**	**57.3**	58.0	**71.4**	**16.6**	**28.2**	**56.0**	**67.8**

Key Technical Insights

1. Uncertainty Analysis:

- Extended reasoning → more uncertain words
- Search-o1 reduces uncertainty from 30.4 to 15.8 occurrences of "perhaps"

2. Document Quality Matters:

- Standard RAG retrieves once → may miss crucial info
- Agentic approach adapts queries → better recall

3. Knowledge Refinement Critical:

- Direct document insertion disrupts coherence
- Separate analysis maintains logical flow

4. Multi-Turn Capability:

- Average of 1.4-2.0 valid searches per problem
- Enables iterative knowledge acquisition

Impact on Reasoning Quality

Before Search-o1:

- Frequent hedging: "perhaps," "might be," "possibly"
- Guesswork when knowledge gaps appear
- Error propagation through reasoning chain

With Search-o1:

- Definitive statements backed by retrieved evidence
- Self-correction when initial assumptions wrong
- Verification through additional searches

Example Reduction in Uncertainty:

- "Perhaps" usage: 30.4 → 15.8 occurrences
- "Alternatively": 27.1 → 11.9 occurrences
- "Wait": 7.8 → 8.2 (minimal change)
- "Likely": 2.8 → 2.6

Takeaways

1. **Extended reasoning creates knowledge gaps** → Need for dynamic retrieval
2. **Simple retrieval is insufficient** → Requires intelligent integration
3. **Outcome-based RL works** → No complex reward engineering needed
4. **Agentic approach outperforms fixed retrieval** → Adaptability is key
5. **Document refinement is critical** → Raw documents disrupt coherence
6. **Scales with model size** → Larger models show greater benefits

Bottom Line: Search-o1 successfully bridges the gap between powerful reasoning and necessary external knowledge

Search-R1 Contributions:

- Retrieved token loss masking for stable RL
- Multi-turn search with outcome rewards
- 41% improvement (Qwen2.5-7B) on QA tasks